

Airborne Laser Scanning for Machine Learning in Ecology

Draft

Contents

Part I — Sensors and Signals	8
Orientation to Part I	8
Chapter 1. Ranging Physics	9
1.1 Time of flight	9
1.2 The lidar equation, and why target size matters more than you expect	9
1.3 Atmospheric transmission	10
1.4 Wavelength	11
1.5 Beam divergence and footprint	12
1.6 The energy-rate trade-off, and why higher PRF is not simply “more data”	12
1.7 Detection thresholds and why “number of returns” is partly a software choice	13
Further reading and visual resources for Chapter 1	13
Chapter 2. Beam Steering and Scan Geometry	14
2.1 The five mechanisms	14
2.2 Ground pattern algebra	15
2.3 Scan angle: the most under-modelled covariate in ALS	15
2.4 Sidelap, overlap zones, and hidden density structure	16
2.5 Multiple pulses in air	17
2.6 Aircraft attitude and platform effects	17
Further reading and visual resources for Chapter 2	17
Chapter 3. Detection, Digitization, and the Discrete-Return Pipeline	18
3.1 The receiver chain	18
3.2 Discrete-return detection: discriminators and range walk	18
3.3 Full-waveform digitization	19
3.4 From waveform to points: decomposition	19
3.5 Dead time, minimum separable range, and the missing understory	20
3.6 Maximum returns per pulse	21
3.7 What “intensity” means in a LAS file	21
Further reading and visual resources for Chapter 3	22
Chapter 4. Sensor Heterogeneity as a First-Class Problem	22
4.1 The three layers of variation	22
4.2 A worked example: NEON is a single program and still heterogeneous	23

4.3 The collinearity problem, stated honestly	23
4.4 What each variation does to a learned representation	24
4.5 Four strategies, and my recommendation	25
4.6 The provenance schema	26
Further reading for Chapter 4	27
Bridge to Part II	27
Part II — Acquisition in Practice	27
Orientation to Part II	28
Chapter 5. Flight Planning and Mission Geometry	28
5.1 The coupled variables	28
5.2 A worked mission	29
5.3 Coverage patterns and calibration geometry	30
5.4 What “2 points per square metre” actually means — and the trap it sets	31
5.5 Environmental windows	32
5.6 Ground control and check points	32
5.7 What a flight plan leaves in the data	33
Further reading and visual resources for Chapter 5	33
Chapter 6. USGS 3DEP: The Specification-Driven Model	34
6.1 What 3DEP is	34
6.2 Quality levels	34
6.3 What the specification requires — and what it declines to require	35
6.4 Classification, bit flags, and the withheld regime	36
6.5 The accuracy regime, and the transition now in progress	38
6.6 Deliverables	38
6.7 Reading a project’s metadata	39
6.8 What all of this means for a pretraining corpus	40
Further reading for Chapter 6	40
Chapter 7. NEON AOP: The Research-Observatory Model	41
7.1 Design logic: colocation, not coverage	41
7.2 Flight box and season	41
7.3 Payload and lidar specifics	42
7.4 The calibration and QA regime	42
7.5 The product ladder	42
7.6 The ground: TOS plot design	43
7.7 Known biases	44
7.8 What NEON is and is not good for	44
Further reading and visual resources for Chapter 7	45
Chapter 8. Other Programs, and Other Platforms	45
8.1 Why you need them	45
8.2 IGN Lidar HD (France)	45
8.3 A survey of other national programs	46
8.4 Other platforms, and the temptation to mix them	47
8.5 A transfer taxonomy for your benchmark	48

Further reading for Chapter 8	48
Bridge to Part III	49
Part III — From Photons to the File You Downloaded	49
Orientation to Part III	49
Chapter 9. Georeferencing and Trajectory	50
9.1 The direct georeferencing equation	50
9.2 GNSS positioning	50
9.3 Inertial attitude	51
9.4 Lever arm and boresight	51
9.5 The error budget	52
9.6 Time	53
9.7 Datums, geoids, and a transition in progress	53
9.8 What survives into the file	55
Further reading for Chapter 9	55
Chapter 10. Post-Flight Adjustment and Classification	56
10.1 Strip and block adjustment	56
10.2 Absolute vertical calibration	57
10.3 Ground classification: the algorithm zoo	57
10.4 Ground classification failure modes	58
10.5 Hydro-flattening	59
10.6 Noise and outlier handling	59
10.7 Above-ground classification	60
10.8 Intensity normalization	60
10.9 Tiling	61
10.10 Trust or re-derive: a decision table	61
Further reading for Chapter 10	62
Chapter 11. What Has Already Been Done to Your File	62
11.1 A taxonomy of transformations	62
11.2 Category B: irreversible and total	63
11.3 Category C: lossy but detectable	63
11.4 Category D: silently inconsistent	64
11.5 The extra layer: cloud copies and derived distributions	65
11.6 The audit protocol	65
11.7 The provenance schema	66
11.8 What the audit buys you	68
Further reading for Chapter 11	69
Bridge to Part IV	69
Part IV — Format, Access, and Provenance Capture	69
Orientation to Part IV	69
Chapter 12. LAS and LAZ Internals	70
12.1 The container	70
12.2 Point data record formats	71
12.3 The classification byte and the flag byte	72

12.4 Fields worth preserving	73
12.5 LAZ	73
12.6 Extra Bytes	74
12.7 LAS 1.5	74
Further reading for Chapter 12	75
Chapter 13. Cloud-Native Organizations: COPC and EPT	75
13.1 The problem both solve	75
13.2 The octree	75
13.3 COPC	76
13.4 EPT	76
13.5 The resolution trap	77
13.6 A second trap: reader bounds versus crop	78
13.7 Which to use	78
Further reading for Chapter 13	78
Chapter 14. PDAL as the Data Layer	79
14.1 The model	79
14.2 Readers	80
14.3 Filters: the ones that matter	81
14.4 Writers	82
14.5 A worked extraction pipeline	83
14.6 Where PDAL ends	84
14.7 Alternatives worth benchmarking	84
Further reading for Chapter 14	85
Chapter 15. Discovery, Catalogs, and Provenance Capture	85
15.1 WESM: the index that makes provenance possible	85
15.2 RockyWeb and the project directory structure	86
15.3 The other discovery paths	86
15.4 Turning the schema into a database	87
15.5 Storage for training	88
15.6 The capture pipeline, end to end	88
15.7 What Part IV bought you	89
Further reading for Chapter 15	89
Bridge to Part V	89
Part V — Features, Tasks, and Evaluation	89
Orientation to Part V	90
Chapter 16. The Engineered-Feature Vocabulary	90
16.1 Why this chapter is not optional	90
16.2 Height distribution metrics	91
16.3 Cover, density, and gap metrics	91
16.4 Vertical arrangement	92
16.5 Neighbourhood geometric features	93
16.6 Intensity and multi-return metrics	93
16.7 Terrain and context features	94

16.8 What the vocabulary implies for architecture	94
Further reading for Chapter 16	94
Chapter 17. Forest-Structure Tasks	95
17.1 Area-based inventory	95
17.2 Canopy height	95
17.3 Individual tree detection and crown delineation	96
17.4 Species classification	96
17.5 Semantic and instance segmentation	97
17.6 Change and structural products	97
17.7 What forest tasks are for, in your benchmark	97
Chapter 18. Evaluation Data: Ground-Truth Sources	97
18.1 A two-tier framing	97
18.2 eBird — avian occurrence	98
18.3 iNaturalist — insect occurrence	99
18.4 Snapshot USA and camera traps — mammal occurrence	99
18.5 Tree identification — a correction	100
18.6 sPlotOpen — vegetation functional traits	100
18.7 NEON’s organismal suite — the rigorous tier you are missing	101
18.8 Other sources worth a paragraph	101
18.9 The source summary table	102
18.10 Task construction principles	103
Chapter 19. Evaluation Protocol	103
19.1 What is being measured	103
19.2 Frozen probing versus fine-tuning	104
19.3 Pooling: the underrated design decision	104
19.4 Multi-scale extents	105
19.5 The ablation ladder	105
19.6 Splitting	106
19.7 The paired sensor design	106
19.8 Metrics and reporting	107
19.9 Reporting negative results honestly	108
19.10 The protocol as a checklist	108
Further reading for Chapters 18–19	109
Bridge to Part VI	109
Part VI — Deep Learning on ALS	109
Orientation to Part VI	110
Chapter 20. What Makes ALS Different	110
20.1 The comparison	110
20.2 Property 1: 2.5D acquisition and systematic vertical occlusion	111
20.3 Property 2: unbounded horizontal extent, thin vertical extent	112
20.4 Property 3: anisotropic, non-stationary density	112
20.5 Property 4: gravity alignment and meaningful absolute z	113
20.6 Property 5: self-similar semantics and sparse indirect labels	113

20.7 Representation families	114
20.8 The design summary	114
Chapter 21. Hierarchical Scaling in Detail	115
21.1 The problem	115
21.2 PointNet: the foundation	115
21.3 PointNet++: set abstraction	116
21.4 KPConv and the convolution analogy	117
21.5 Point Transformer V1 and V2	118
21.6 Point Transformer V3: ordering instead of searching	118
21.7 Superpoint Transformer: partition instead of hierarchy	120
21.8 Comparison and recommendation	120
Further reading for Chapter 21	121
Chapter 22. Self-Supervised Pretraining	121
22.1 Why pretrain at all	121
22.2 The masked-modelling design space	122
22.3 The geometric shortcut	122
22.4 Sonata’s mitigations, and how they map to ALS	123
22.5 The height-normalization decision	124
22.6 Alternatives to masked modelling	124
22.7 A pretraining recipe	125
Further reading for Chapter 22	125
Chapter 23. Spatial Extent, Crop Size, and Stitching	126
23.1 The argument for choosing extent ecologically	126
23.2 The point-budget versus area trade	126
23.3 Density normalization	127
23.4 Overlapping crops and inference stitching	127
23.5 The experiment I would run	128
23.6 Multi-scale embeddings, revisited	128
Further reading for Chapter 23	128
Bridge to Part VII	129
Part VII — Building the Benchmark	129
Orientation to Part VII	129
Chapter 24. Corpus Construction	129
24.1 What “representative” has to mean	129
24.2 The stratification design	130
24.3 Deduplication and repeat coverage	131
24.4 The leaf-on / leaf-off problem	131
24.5 Corpus size and scaling curves	132
24.6 Sample geometry for pretraining	132
24.7 The construction pipeline	133
24.8 Reference implementations to study first	133
Chapter 25. Baselines and Evaluation	134
25.1 The baseline matrix	134

25.2 The evaluation matrix	134
25.3 What to fix before running anything	135
25.4 The results tables you should plan to produce	135
25.5 The honest failure modes	136
Chapter 26. Release	136
26.1 Licensing: three programs, three regimes	136
26.2 The release strategy that follows	137
26.3 Documentation	137
26.4 The frozen test split	138
26.5 A leaderboard protocol that does not rot	138
26.6 Reproducibility artifacts	138
Chapter 27. Prioritization	138
27.1 The problem with the plan	138
27.2 The genuinely novel contributions, ranked	138
27.3 What is necessary but not novel	139
27.4 What I would drop or defer	139
27.5 A staged plan	140
27.6 The result I would most like to see	140
Closing	140
Appendix A — Consolidated resource list	141
Appendices	141
Appendix B — Classification and Validity Flags: A Practical Reference	142
B.1 The production workflow	142
B.2 What the specification does and does not constrain	142
B.3 The class codes that matter for ecology	143
B.4 The four bit flags	144
B.5 The withheld flag in detail	144
B.6 The loader policy	146
Appendix C — Stratified Allocation: An Implementable Specification	147
C.1 The frame	147
C.2 Allocation across ecoregions	147
C.3 Allocation within ecoregion	147
C.4 Filling QL and phenology	148
C.5 Reporting the shortfall	148
C.6 Nested subsets	148
C.7 Exclusion geometry	149
Appendix D — Receptive Field in Metres: Measurement and the Serialization	
Experiment	149
D.1 Why the analytic estimate is not enough	149
D.2 The empirical measurement protocol	150
D.3 The serialization experiment	150
D.4 Reporting	151

Part I — Sensors and Signals

Airborne Laser Scanning for Machine Learning in Ecology

Orientation to Part I

Almost every ALS machine-learning paper begins at the LAZ file. That is a defensible place to begin if your task is engineering, and an indefensible one if your task is building a benchmark. The reason is that a LAZ file is not a measurement — it is the *fourth or fifth derived product* of a measurement, and most of the transformations between the photons and the file are lossy, vendor-specific, and undocumented in the file itself.

Concretely: by the time you read a 3DEP tile, an unknown discretization algorithm has decided how many of the returns in each waveform to report; an unknown gain control has decided what “intensity” means; an unknown discriminator has introduced a range bias correlated with target reflectance; and a receiver dead time has silently deleted most of the near-ground vegetation. None of this is recoverable from the file. All of it varies between projects. And in a benchmark spanning hundreds of 3DEP collections, all of it is confounded with geography, because a given contractor with a given sensor works a given region in a given year.

Part I is therefore not background. It is the chapter set that tells you which apparent ecological signals in your benchmark are actually hardware signatures. Four chapters:

- **Chapter 1** develops the ranging physics: how a range is measured, what governs whether a return is detected at all, and why the answer depends on target size in a way that makes intensity nearly uncomparable across missions.
- **Chapter 2** covers beam steering and mission geometry — the mechanisms that turn one laser into a swath, and the systematic within-tile non-stationarity they produce.
- **Chapter 3** covers detection and digitization: the receiver chain, waveform decomposition, and the specific artifacts (range walk, dead time, return-count limits) that shape the vertical profile you will ask a network to interpret.
- **Chapter 4** consolidates all of this into the central methodological problem of the book: sensor heterogeneity, and the four strategies available for handling it.

On the running case study. Throughout the book I use one concrete design as a worked example: pretraining a general-purpose ALS representation on a nationally stratified sample of USGS 3DEP, then evaluating it on a range of ecological ground-truth sources. Where a general principle has a specific consequence for that design, I flag it as **[Case study]**. The rest of the text is written to serve as a general reference.

On figures. This text links to external figures, animations, and videos rather than reproducing them. Where I have a verified stable URL I give it; where I don’t, I name the specific figure and source precisely enough to find. The single most useful companion for Part I is Lewis Graham’s *Airborne Lidar: A Tutorial for 2025* in LIDAR Magazine, which is figure-rich and written by someone who builds these systems. Read it alongside Chapters 1–3.

Chapter 1. Ranging Physics

1.1 Time of flight

A pulsed topographic lidar measures range by timing a light pulse's round trip:

$$R = \frac{c t}{2n}$$

where t is the round-trip time, c the vacuum speed of light, and n the group refractive index of air (≈ 1.0003 at optical wavelengths, altitude- and humidity-dependent; ignoring it introduces a range error of roughly 30 cm per kilometre of path, which is why atmospheric models enter the georeferencing chain).

The factor of two is the whole story of lidar timing precision. Because light travels 30 cm in one nanosecond, and the range is half the path, **1 ns of timing error is 15 cm of range error**. A system targeting 5 cm ranging precision needs timing precision better than about 330 ps. This is achievable — modern systems use time-to-digital converters or interpolate the digitized waveform to sub-sample precision — but it explains why the timing electronics are the expensive part of the instrument and why range precision, uniquely among lidar error sources, is *not* the dominant term in the total error budget. Trajectory and boresight errors dominate (Chapter 9).

An immediate corollary worth internalizing: **range precision and range resolution are different quantities and they are governed by different things**.

- *Range precision* — how repeatably you can locate a single target — is set by timing precision and SNR, and can be far finer than the pulse length.
- *Range resolution* — how close two targets can be and still produce two distinguishable returns — is set by the transmitted pulse duration τ and the receiver dead time:

$$\Delta R_{\min} \approx \frac{c \tau}{2}$$

A 5 ns pulse gives $\Delta R_{\min} \approx 75$ cm as a hard optical floor, and real systems do worse than this because of receiver recovery time (§3.4). This is the physical origin of one of the most consequential blind spots in ALS ecology: **the layer immediately above the ground is systematically under-sampled**, because low vegetation returns fall inside the same resolution cell as the ground return and are merged into it. Any benchmark task involving understory, shrub layer, regeneration, or fuel loading is fighting this limit. It is not a processing artifact you can filter away; the information was never recorded.

1.2 The lidar equation, and why target size matters more than you expect

The received optical power determines whether a return is detected. The standard form for a Lambertian target is:

$$P_r = P_t \cdot \frac{\rho}{\pi} \cdot \frac{A_r}{R^2} \cdot \eta_{\text{sys}} \cdot \eta_{\text{atm}}^2 \cdot F$$

where P_t is transmitted power, ρ the target reflectance at the laser wavelength, $A_r = \pi D_r^2/4$ the receiver aperture area, η_{sys} the optical/electronic system efficiency, η_{atm} the one-way atmospheric transmission (squared, because the path is traversed twice), and F the fraction of the illuminated footprint occupied by the target.

The term to dwell on is F , because it changes the range exponent:

- **Beam-filling target** (bare ground, a road, a building roof, a dense closed canopy top): $F = 1$, and $P_r \propto R^{-2}$.
- **Sub-footprint target** (a single leaf, a twig, a branch, a wire, a fence post): the target area is fixed while the footprint area grows as R^2 , so $F \propto R^{-2}$ and therefore $P_r \propto R^{-4}$.

Vegetation is made of sub-footprint targets. Ground is a beam-filling target. So when flying height increases, or when scan angle increases (which increases slant range), *canopy return power falls off twice as fast in log terms as ground return power*. Three consequences, all of which will show up in your data:

1. **Intensity is not comparable across missions flown at different altitudes**, and the usual “range normalization” correction — multiplying intensity by $(R/R_{\text{ref}})^2$ — is *correct for the ground and wrong by a factor of $(R/R_{\text{ref}})^2$ for the canopy*. Applying it makes ground intensities comparable and canopy intensities differently-incomparable. If a vendor applied it, you cannot undo it. Treat intensity as a nuisance channel by default and earn the right to use it per-project.
2. **The number of detected returns per pulse in vegetation is altitude-dependent**, because weak canopy returns drop below the detection threshold sooner at higher altitude. A collection flown at 2000 m does not simply have a coarser footprint than one flown at 1000 m — it has a *shallower vertical profile*, missing the weakest interior-canopy returns.
3. **Density and scan angle interact with detectability**, not just geometry. Edge-of-swath points are not merely sparser; they are biased toward strong scatterers.

For a graduate-level treatment of the radiometric side, Wagner’s work on radiometric calibration of full-waveform ALS is the canonical entry point (search: *Wagner “radiometric calibration of small-footprint full-waveform airborne laser scanner”*, ISPRS Journal 2010), along with the associated concepts of backscatter cross-section σ and backscatter coefficient γ .

1.3 Atmospheric transmission

$$\eta_{\text{atm}} = \exp(-aR)$$

with a the extinction coefficient (absorption plus scattering). The two-way term $\eta_{\text{atm}}^2 = \exp(-2aR)$ means atmospheric loss is exponential in altitude, and it is why haze, thin cloud, or smoke can reduce return density dramatically without producing any obvious flag in the data. In practice this appears as **density striping and dropout patches** that correlate with nothing in the landscape. It is one of the reasons acquisition specifications forbid flying through cloud or smoke, and one of the reasons a per-tile density statistic belongs in your provenance schema — anomalous low density is frequently atmospheric, not ecological.

Wildfire smoke is now a routine constraint on western US summer acquisition, which introduces a subtle temporal bias worth noting: **[Case study]** the availability of clear-air acquisition

windows is itself correlated with fire regime, so a nationally stratified 3DEP sample is not stratified on acquisition conditions unless you make it so.

1.4 Wavelength

Three wavelengths dominate:

λ	Source	Typical use	Vegetation ρ	Water	Snow
1064 nm	Nd:YAG fundamental, diode-pumped	Most topographic ALS, including nearly all 3DEP and NEON	High (NIR plateau, ~40–60% healthy leaf)	Absorbing; specular loss on smooth water	Highly reflective
532 nm	Frequency- doubled Nd:YAG	Bathymetric / topo-bathy	Moderate	Penetrates to several Secchi depths	Reflective
1550 nm	Er-doped fibre laser	UAV/mobile units, some airborne	Lower (~25–35%; leaf water absorption)	Strongly absorbed	Near-black

The choice is driven by three competing pressures.

Vegetation reflectance favours 1064 nm. The NIR reflectance plateau of healthy foliage sits right there, and the contrast between vegetation and most soils/rocks is good. At 1550 nm you are on the shoulder of a liquid-water absorption feature, so leaf reflectance is lower and — importantly for ecology — *reflectance becomes a function of leaf water content*, which is either a bug (uncontrolled variation) or a feature (drought stress signal), depending on whether you can calibrate it.

Eye safety favours 1550 nm, decisively. The retinal hazard region spans roughly 400–1400 nm; light in that band is transmitted by the ocular media and focused onto the retina. At 1550 nm, absorption by the cornea and vitreous prevents retinal focusing, so the maximum permissible exposure is roughly one to two orders of magnitude higher. For a given eye-safety class, a 1550 nm system can transmit far more energy per pulse — which, via §1.2, means more detected returns from weak sub-footprint scatterers. This is the single largest engineering argument for 1550 nm, and it is why the wavelength dominates in low-altitude UAV lidar where the nominal ocular hazard distance would otherwise be a problem.

Snow, ice, and water favour 1064 nm for topographic mapping. A 1550 nm system over fresh snow gets almost nothing back. Since a substantial fraction of the US is snow-covered during the leaf-off season that 3DEP prefers for terrain mapping, this is not academic.

[Case study] For your purposes, the practical upshot is that 3DEP and NEON are both overwhelmingly 1064 nm, which is a rare piece of good news — you are not fighting a wavelength domain gap within CONUS. But if you later evaluate transfer to European national collections or to UAV data (FOR-instance, drone-density benchmarks), you may be crossing 1064 → 1550,

which changes intensity semantics completely and changes return counts in wet canopy. Do not treat intensity as transferable across that boundary.

1.5 Beam divergence and footprint

The transmitted beam has an angular divergence γ (full angle, usually specified at the $1/e^2$ intensity points, sometimes at $1/e$ — check which, they differ by $\sqrt{2}$). The footprint diameter on a flat surface at nadir is

$$D_f \approx 2R \tan(\gamma/2) \approx R\gamma \quad (\gamma \ll 1)$$

plus the beam waist at the exit aperture, which is negligible at airborne ranges. Typical airborne values are $\gamma = 0.15\text{--}0.5$ mrad, giving footprints of roughly 15–50 cm at 1000 m AGL. Off-nadir, the range increases as $R = h/\cos\theta$ and the footprint becomes an ellipse elongated in the scan direction, with the major axis growing as $1/\cos^2\theta$.

Footprint size has ecological consequences that are easy to miss and hard to correct:

- **Large footprint** → **more multiple returns**. A 50 cm footprint is more likely than a 15 cm footprint to straddle a leaf and a gap simultaneously, producing two returns from one pulse. So *the mean number of returns per pulse is partly a footprint property, not a canopy property*. Any metric built on the ratio of first/intermediate/last returns is contaminated by this.
- **Large footprint** → **better ground detection under canopy**, because the probability that some part of the footprint finds a gap increases. This raises the ground-return fraction and improves DTM quality under dense canopy — but it also means a small-footprint sensor and a large-footprint sensor report different canopy *cover*, in opposite directions, over the same forest.
- **Small footprint** → **sharper horizontal detail**, better crown delineation and gap mapping, but noisier vertical profiles because each pulse samples a smaller, less representative slice of the canopy.
- **Footprint grows with scan angle**, so all of the above vary *within a single swath*.

There is a useful mental model here: a lidar pulse is not a ray, it is a **cone that integrates over an area**, and the discrete returns it produces are a lossy summary of the vertical distribution of intercepting material within that cone. Everything about vegetation lidar follows from that.

1.6 The energy–rate trade-off, and why higher PRF is not simply “more data”

The laser has an average power budget. To first order,

$$P_{\text{avg}} \approx E_{\text{pulse}} \times \text{PRF}$$

So doubling the pulse repetition frequency at fixed average power halves the energy per pulse. Via §1.2, halving pulse energy halves every return’s power, which pushes the weakest returns below the detection threshold.

This is the most commonly misunderstood specification in ALS. A 1 MHz sensor flown at the same altitude as a 200 kHz sensor does not give you the same vertical profiles at five times the density. It gives you five times as many *weaker* shots, each of which recovers fewer returns from the canopy interior. The point cloud is denser horizontally and *shallower* vertically. If your benchmark contains both and you treat “points per square metre” as the density covariate, you have mismodelled the difference.

Practical implications for a heterogeneous corpus:

- Record **pulse density** (shots per m²) separately from **point density** (returns per m²). Their ratio — returns per pulse — is an informative and often-omitted covariate that partly captures the energy/footprint regime.
- Expect the returns-per-pulse distribution to differ between projects over ecologically identical forest.
- When simulating degradation for augmentation (§4.5), thin *pulses*, not points, and truncate return sequences — thinning points at random destroys the pulse structure and produces clouds unlike any real sensor.

Some systems break the trade-off partially with multiple laser channels (dual- or quad-channel systems firing into different scan sectors), which raises total PRF without lowering per-pulse energy. Those systems’ `point_source_id` and channel bit fields matter; see §12 on the LAS channel/scanner-channel field.

1.7 Detection thresholds and why “number of returns” is partly a software choice

Whether a given echo becomes a point in your file depends on a threshold applied to the amplified detector signal. Set the threshold low and you gain weak canopy returns and also atmospheric/noise triggers; set it high and you get a clean but vertically truncated cloud. The threshold is a vendor and operator choice, sometimes adaptive.

The consequence stated plainly: **the vertical extent of your point cloud below the canopy top is a joint function of pulse energy, footprint, atmospheric transmission, detector sensitivity, discriminator threshold, and decomposition algorithm — six things that vary between projects and none of which are recorded in a LAZ file.** Chapter 4 is about what to do with that.

Further reading and visual resources for Chapter 1

- *Airborne Lidar: A Tutorial for 2025*, LIDAR Magazine — the best short figure-rich treatment of the transmit/receive chain.
- [NEON: Lidar data collection](#) — includes NEON’s own explanation of waveform vs. discrete return and an embedded explainer video.
- Wehr & Lohr (1999), *Airborne laser scanning — an introduction and overview*, ISPRS Journal 54(2–3) — still the clearest derivation of the system equations and the scanner taxonomy used in Chapter 2.
- For the radiometry: Wagner (2010), ISPRS Journal, on radiometric calibration and backscatter cross-section.

- Baltsavias (1999), *Airborne laser scanning: basic relations and formulas*, ISPRS Journal 54(2–3) — a compact reference for footprint, spacing, and swath geometry algebra; worth keeping open while working through Chapter 2.

Chapter 2. Beam Steering and Scan Geometry

A lidar measures along one line of sight. Turning that into a swath requires deflecting the beam, and the deflection mechanism determines the geometry of the sample. That geometry is *systematically non-uniform*, and its non-uniformity is imprinted on every point cloud you will train on.

2.1 The five mechanisms

Following the classical taxonomy (Wehr & Lohr 1999), extended with modern additions. YellowScan’s [scan-pattern explainer](#) has clean diagrams of the ground patterns for most of these; its Figures 1–3 are the ones to look at.

Oscillating (galvanometer) mirror. A flat mirror driven back and forth about one axis. Ground pattern: a zig-zag or sinusoid, bidirectional. This is the most common mechanism in airborne topographic lidar, used across the Leica ALS and Teledyne Optech ALTM/Gemini/Galaxy families.

Its defining artifact: the mirror must decelerate, stop, and reverse at each swath edge. Angular velocity is therefore roughly sinusoidal, and **point density piles up at the edges of the swath** where the beam dwells longest. Simultaneously, edge-of-swath points are the *least* geometrically reliable — mirror reversal introduces nonlinearity and hysteresis, slant range and footprint are maximal, and incidence angles are worst. Systems therefore often blank or discard a few degrees at each extreme, and specifications typically require an effective scan half-angle well inside the mechanical maximum. The net pattern is a density distribution across the swath that is high at the edges, lower in the middle, with the extreme edges cut off — a shape that does not resemble anything ecological but will correlate with everything if you don’t account for it.

Rotating polygon mirror. A multi-faceted mirror spinning continuously in one direction. Each facet sweeps one line. Ground pattern: **parallel, unidirectional, evenly-spaced lines** with constant angular velocity and therefore uniform along-scan point spacing. This is geometrically the cleanest option and the reason Riegl’s airborne units (Q680/Q780/VQ series) are prized for vegetation work. The cost is duty cycle: only the fraction of each facet’s rotation that points at the ground is useful, so some pulses are wasted or must be gated.

Rotating/nutating mirror (Palmer scanner). A mirror whose rotational axis is tilted relative to the optical axis, so the beam sweeps an ellipse. Ground pattern: overlapping ellipses producing a spiral. The distinctive property is that **every ground point is viewed twice per pass, from two different look directions** (forward-looking and backward-looking). For vegetation this reduces directional occlusion — you sample both sides of a crown — and for calibration it provides internal geometric redundancy useful in strip adjustment. The cost is a strongly non-uniform density pattern and complex point-spacing statistics.

Fibre-optic line scanner. No moving deflector; a bundle of fibres arranged in an arc, illuminated sequentially, projects a fixed set of look directions. The TopoSys Falcon was the notable implementation. Ground pattern: perfectly parallel lines with fixed across-track angular spacing. Rare now, but you may encounter early-2000s data from such systems.

Risley prism pairs / MEMS / non-repetitive patterns. Two counter-rotating wedge prisms deflect the beam into circles, rosettes, or (with incommensurate rotation rates) non-repeating patterns that fill in over time. Common in UAV and automotive units (Livox and similar), and increasingly in survey-grade UAV lidar. Ground pattern: circles, rosettes, or dense non-repeating scribbles. Important if your benchmark ever includes UAV data (e.g. FOR-instance), because the point-spacing statistics are unlike any airborne pattern and naive density normalization will not make them comparable.

2.2 Ground pattern algebra

For a scanner with scan rate f_{scan} (scan lines per second), aircraft ground speed v , PRF, altitude h , and scan half-angle θ_{max} :

Swath width: $W = 2h \tan \theta_{\text{max}}$

Across-track line spacing: $\Delta_{\text{across}} = v/f_{\text{scan}}$ (for a unidirectional scanner; $v/(2f_{\text{scan}})$ for bidirectional, since two lines are laid per cycle)

Along-scan point spacing at nadir, for a constant-angular-velocity scanner: $\Delta_{\text{along}} = h\omega/\text{PRF}$ where ω is angular scan velocity. For a sinusoidal scanner, $\omega = \omega(\theta)$ and spacing varies across the swath as described above.

Nominal pulse density: $\text{NPD} \approx \text{PRF}/(Wv)$ for a single pass, ignoring sidelap; **nominal pulse spacing** $\text{NPS} = 1/\sqrt{\text{NPD}}$. USGS specifies quality levels in terms of NPS/NPD , so this is the algebra that connects a flight plan to a spec compliance claim.

The design tension: a good pattern has $\Delta_{\text{across}} \approx \Delta_{\text{along}}$, i.e. isotropic sampling. Achieving that constrains speed, scan rate, and PRF jointly, and it is generally *not* achieved — most ALS point patterns are measurably anisotropic. If your model uses k-nearest-neighbour or ball-query neighbourhoods (Chapter 21), anisotropic sampling means the neighbourhood shape is a sensor property. Worth an ablation.

Baltsavias (1999) is the reference for these relations, with more cases worked than I've given here.

2.3 Scan angle: the most under-modelled covariate in ALS

Four distinct effects, all monotonic in $|\theta|$, all confounded with each other, and all confounded with position within the swath:

1. Path length through canopy. A pulse entering the canopy at off-nadir angle θ traverses a path length $\propto 1/\cos \theta$ through the same vertical canopy depth. More interception means: higher probability of a return in the upper canopy, lower probability of reaching the ground, and therefore a **falling ground-return fraction with increasing scan angle**. At 20° off nadir the path is $\sim 6\%$ longer; at 30° , $\sim 15\%$. That sounds small, but gap probability is

exponential in path length (Beer–Lambert, §16), so the effect on ground-return fraction is not small.

2. Slant range. $R = h / \cos \theta$ increases both the footprint (§1.5) and the range attenuation (§1.2). Weak canopy returns drop out preferentially at swath edge.

3. Incidence angle. For a Lambertian surface, returned power scales as $\cos(\text{incidence})$, and incidence depends on the *sum* of scan angle and terrain slope. So intensity has a terrain-dependent, scan-angle-dependent modulation. Over sloped forest, intensity and structure are geometrically entangled.

4. Occlusion geometry. Off-nadir, you see the side of a crown; at nadir, the top. Two pulses hitting the same tree at -15° and $+15^\circ$ sample materially different parts of it. This is *useful* if you have both (see Palmer scanners, and overlap zones), and a source of unmodelled variance if you don't know which you have.

The practical recommendation, which I will repeat in Chapter 15: **retain scan angle as a per-point feature and as a per-tile summary statistic**. LAS point formats carry a scan angle field (rank in formats 0–5, degrees at finer resolution in formats 6–10; see §12.2). Most ALS-ML pipelines discard it. It is one of the cheapest available nuisance covariates and it directly encodes three of the four effects above.

2.4 Sidelap, overlap zones, and hidden density structure

Adjacent flight lines overlap. Specifications mandate a minimum — NEON designs adjacent lines with [30% overlap](#); 3DEP's Lidar Base Specification sets its own requirements — for two reasons: to guarantee no gaps under aircraft attitude variation, and to provide the redundancy needed for strip adjustment (§10.1).

For machine learning, the overlap zone is a distinct data regime hiding inside your tiles:

- **Density is roughly doubled**, in stripes whose position is determined by flight-line layout, not by the landscape.
- **The scan-angle distribution is bimodal**, since the zone is at the edge of one swath and the edge of the next.
- **Points are separated in time** by minutes to hours, across which the aircraft, GNSS constellation, and atmosphere have all changed. Residual vertical offsets between strips are the norm, and are the thing strip adjustment is trying to eliminate.
- **Vegetation may have moved**. Wind-driven crown motion between two passes is real and produces genuine, irreducible disagreement in canopy-top elevation.

3DEP requires overlap points to be identified — the mechanism has changed across specification versions (older revisions used class code 12, current practice uses the dedicated Overlap bit flag introduced in LAS 1.4), and the [Lidar Base Specification revision status page](#) is where to check what applies to a given collection year. Some vendors deliver overlap points flagged; some deliver them removed; some deliver them neither flagged nor removed. **All three appear in 3DEP.**

[Case study] Decide early and document: does your pipeline drop overlap points, keep them, or keep them with a flag as a feature? Whichever you choose, the *fraction of a tile that is*

overlap varies from ~0% to ~100% depending on where the tile falls relative to flight lines, and it will correlate with local density in a way that has nothing to do with ecology. Recording overlap fraction per sample costs one float and saves you an unexplained variance component later.

2.5 Multiple pulses in air

At high PRF the next pulse is emitted before the previous one's echo returns. Round-trip time from 1000 m AGL is $2 \times 1000/c \approx 6.7 \mu\text{s}$; a 1 MHz PRF emits every $1 \mu\text{s}$. So roughly seven pulses are simultaneously in flight.

Systems resolve this with pulse-coding, gated range windows, or geometric constraints, and modern implementations are reliable over moderate relief. Over high relief — deep canyons, tall cliffs, abrupt elevation change within a swath — the range window can be violated and produce **ghost returns at systematically wrong elevations**, typically appearing as a coherent surface offset by one pulse interval's worth of range. These are usually caught by QC and classified as noise, but “usually” is doing work in that sentence. If your benchmark includes steep terrain, budget for outlier filtering (`filters.outlier`, `filters.elm`) and inspect the classification-7/18 fraction as a QC signal.

2.6 Aircraft attitude and platform effects

The scanner is bolted to a moving, rolling, pitching, yawing aircraft.

- **Roll** shifts the swath sideways. Most systems compensate by biasing the mirror, but compensation is imperfect and residual roll produces swath-edge scalloping.
- **Pitch** and **yaw/crab** (from crosswind) rotate the scan pattern relative to the flight path, producing along-track density modulation and, in extreme cases, gaps.
- **Turbulence** produces high-frequency attitude variation that shows up as **density striping** — bands of high and low point density perpendicular or oblique to the flight line.

These are the origin of a class of artifact you will see repeatedly in ALS tiles: coherent linear density structure at flight-line scale and sub-flight-line scale, with no landscape correlate. A convolutional or attention-based model asked to predict a spatially smooth ecological quantity from such a cloud has ample opportunity to key on the artifact. Two defences: include density as an explicit input channel (so the model can condition on it rather than confound it), and augment with synthetic density modulation during training (§4.5).

Further reading and visual resources for Chapter 2

- [YellowScan: what are the different scan patterns of lidar systems?](#) — diagrams of oscillating, polygon, rotating, and Risley ground patterns. The clearest free visual reference for §2.1.
- Baltsavias (1999), *Airborne laser scanning: basic relations and formulas*, ISPRS Journal 54(2–3) — the algebra of §2.2 worked out completely.
- For a moving-picture intuition on scan mechanisms, search for manufacturer animations from Riegl (polygon), Leica (oscillating), and Livox (non-repetitive Risley); each vendor publishes short clips of its own mechanism.

- To *see* the scan-angle and overlap structure in your own data: load a single tile in viewer.copc.io or CloudCompare and colour by `ScanAngleRank`, then by `PointSourceId`. The flight-line geometry becomes visible immediately, and this five-minute exercise will change how you think about tiles.

Chapter 3. Detection, Digitization, and the Discrete-Return Pipeline

This chapter covers the transformation from a returning optical signal to the rows of a LAZ file. It is where the most information is lost and where the most inter-project inconsistency is introduced.

3.1 The receiver chain

1. **Telescope / receiving optics** collect the return over aperture A_r . Aperture appears linearly in the lidar equation, so it is a first-order sensitivity parameter.
2. **Optical bandpass filter** rejects solar background. Narrower is better for SNR but constrains laser wavelength stability. Solar background is why acquisition is often restricted by sun angle and why daytime SNR is worse than nighttime — and why some collections are flown at night, notably for coastal and bathymetric work.
3. **Photodetector**. Silicon avalanche photodiodes (APDs) for 1064 nm; InGaAs APDs for 1550 nm; photomultiplier tubes for the green channel in bathymetric systems, where sensitivity matters more than linearity. Detector choice sets noise-equivalent power, dynamic range, and linearity — and therefore sets what “intensity” can possibly mean.
4. **Transimpedance amplifier**, possibly with automatic gain control (AGC) or logarithmic compression to handle the enormous dynamic range between a bright roof at nadir and a leaf at swath edge.
5. **Timing / digitizing** — the fork in the road (§3.2 vs §3.3).

Point 4 deserves emphasis because it is the origin of the intensity comparability problem. **If a system applies AGC, the reported intensity is a function of the gain state at the moment of the shot, and the gain state depends on the recent history of returns.** Teledyne Optech systems historically used AGC and recorded the gain value in a separate field; Riegl systems report calibrated amplitude and reflectance in dB. A LAS `Intensity` field from the first, with the gain field not carried through, is uninterpretable. From the second, it is interpretable but on a log scale. There is no field in LAS that tells you which situation you are in.

3.2 Discrete-return detection: discriminators and range walk

In a discrete-return system, the amplified signal is fed to a discriminator that emits a timing pulse when an echo is detected. Three common designs:

Leading-edge threshold. Fire when the signal crosses a fixed threshold. Simple, and afflicted by **range walk** (also “amplitude walk,” “time walk”): a strong return crosses the threshold earlier on its rising edge than a weak return of identical timing, so the reported range is

systematically short for bright targets. Magnitude is on the order of a few centimetres to a decimetre depending on pulse shape and threshold.

Consider what range walk means ecologically. Bright targets are high-reflectance, beam-filling targets — bare soil, rock, roads, dense grass. Dim targets are dark or sub-footprint — wet soil, conifer needles, shadowed understory. So **uncorrected range walk produces a vertical bias correlated with land cover and species composition**. That is precisely the kind of nuisance signal a high-capacity model will happily learn. Systems correct for it with an intensity-dependent range correction derived from calibration; whether a given vendor’s correction was applied and how well is not recorded in the file.

Constant-fraction discrimination (CFD). Trigger at a fixed *fraction* of peak amplitude rather than a fixed absolute level, using a delay-and-invert circuit. Amplitude-independent to first order, so range walk largely disappears. Better and more expensive.

Peak or derivative-zero-crossing detection. Trigger at the maximum, i.e. the zero crossing of the differentiated signal. Also amplitude-independent, and the natural analogue of what waveform decomposition does in software.

3.3 Full-waveform digitization

Instead of discriminating in analog, digitize the whole return. NEON’s systems [sample the return waveform at 1 ns intervals](#) — i.e. 1 GS/s, giving 15 cm range sampling — and a single waveform may contain hundreds of samples where the corresponding discrete product reports only one to eight targets. That ratio is the scale of the information loss in discretization.

What waveform data gives you that discrete returns cannot:

- **Sub-resolution-cell structure**, recoverable by deconvolution or model fitting below the naive $c\tau/2$ limit.
- **Echo width σ_i** , which broadens with within-footprint height variation and surface slope. Echo width is a direct measure of small-scale roughness and is a genuinely different signal from anything in a discrete cloud.
- **Calibratable amplitude**, from which backscatter cross-section and reflectance can be derived given calibration targets — the only route to intensity that is comparable across missions.
- **The complete vertical profile of intercepted energy**, which is what canopy-profile and LAD-inversion methods actually want (§16.3). Discrete returns are a thresholded, count-based approximation of it.

NEON retains waveform alongside discrete (the waveform product is separate from DP1.30003.001), which makes NEON sites the natural place to *quantify* what discretization costs — a study that, as far as I know, has not been done systematically for ecological tasks and which would make a strong standalone contribution.

3.4 From waveform to points: decomposition

The standard model treats the return as a superposition of scaled, shifted copies of the system waveform:

$$w(t) = \sum_{i=1}^N A_i g(t - t_i; \sigma_i) + \varepsilon(t)$$

with g typically Gaussian (justified because the transmitted pulse is approximately Gaussian and convolution of Gaussians is Gaussian; generalized Gaussian, lognormal, and Nakagami alternatives handle asymmetric returns better).

The estimation procedure:

1. **Smooth** the digitized waveform (Gaussian or Savitzky–Golay) to suppress digitizer noise.
2. **Detect candidate peaks** via zero crossings of the first derivative with a second-derivative sign test, or via a matched filter.
3. **Initialize** (A_i, t_i, σ_i) from the candidates.
4. **Nonlinear least squares** — Levenberg–Marquardt is standard — to refine jointly.
5. **Prune** components below an amplitude or width criterion; possibly merge close components.
6. **Emit** one point per surviving component, with range from t_i and intensity from A_i , or from the integrated area $A_i \sigma_i \sqrt{2\pi}$, or from something else.

Alternatives: Wiener or Richardson–Lucy deconvolution followed by peak picking; sparse/compressed-sensing formulations; and, recently, learned decomposition.

The critical observation for benchmark builders is in step 5, and it is worth stating as a boxed conclusion:

The number of discrete returns extracted from a waveform is not a physical quantity. It is the output of an algorithm with tunable thresholds. Two vendors processing the same waveform will produce different numbers of points, at different elevations, with different intensities. Nothing in the LAZ file records which algorithm was used or how it was tuned.

NEON documents its own algorithm, which is unusually transparent: the [L0-to-L1 Discrete Return Lidar ATBD \(NEON.DOC.001292\)](#) is the reference, and NEON reports [discretizing up to 5 points per shot from a minimum 4 shots/m²](#). For 3DEP, the equivalent documentation does not exist in aggregate — each project’s processing report describes its own workflow, and the specification governs deliverables rather than methods.

3.5 Dead time, minimum separable range, and the missing understory

After a detection, the receiver requires a recovery interval τ_{dead} before it can register another. Combined with the pulse-width limit of §1.1, real minimum separable ranges are typically **1–3.5 m**, varying by system and being one of the specifications most worth extracting from sensor datasheets.

The ecological consequence is severe and structural:

- **Vegetation within ~1–3 m of the ground is largely invisible**, because its return falls within the ground return’s shadow. Shrub layer, seedlings, regeneration, coarse woody debris, surface fuels — all systematically under-sampled.

- The magnitude of the effect **differs between sensors**, so “shrub cover” computed identically from two collections over the same site is not the same quantity.
- The standard mitigation in the ecology literature is to exclude returns below a height threshold (2 m is conventional) and simply not make claims about that stratum. Neon-TreeEvaluation, for example, [excludes non-ground points below 2 m](#) in constructing its point-cloud annotations.

[Case study] This directly constrains which benchmark tasks are honest. A task predicting canopy height, biomass, or overstory species composition is on solid ground. A task predicting understory-dependent quantities — small mammal occupancy, ground beetle assemblages, herbaceous functional traits — is asking the model to infer from a stratum the sensor barely measured. That does not make such tasks invalid; structural proxies from the overstory carry real information about understory conditions. But it does mean you should expect and *report* lower ceilings on those tasks, and you should not attribute the gap to model capacity. State the sensing limitation in the task description.

3.6 Maximum returns per pulse

Systems cap the number of returns extracted per pulse: two in older systems, four typical for Optech ALTM/Gemini-era instruments, up to eight for Riegl and Optech Galaxy-class systems, more in some recent designs. LAS point formats 0–5 allocate 3 bits to **NumberOfReturns** (max 7); formats 6–10 allocate 4 bits (max 15) — a format-level constraint that can itself truncate data.

Effects on the vertical profile:

- A 4-return cap over a multi-layered forest **truncates the middle of the profile**, since the algorithm typically retains the strongest components. Intermediate-canopy returns are lost preferentially.
- Ratios like (intermediate returns)/(total returns) are direct functions of the cap and are therefore among the *least* transferable engineered features. Treat any such metric with suspicion when comparing across projects.
- When augmenting to simulate a sparser sensor (§4.5), **truncating return sequences is as important as thinning pulses**, and is almost universally omitted.

3.7 What “intensity” means in a LAS file

Collecting the strands: the **Intensity** field can be, depending on vendor and processing chain, any of —

- Peak amplitude of the detected echo, in raw digitizer counts
- Integrated echo area
- Amplitude with AGC applied and gain not recorded
- Amplitude with a range normalization applied to an unstated reference range
- Calibrated amplitude in dB
- Calibrated reflectance
- Any of the above rescaled to 16 bits by an unstated linear or nonlinear transform

The Lidar Base Specification requires that intensity be [scaled linearly to 16 bits and expressly](#)

forbids the image-style stretches that vendors sometimes apply for visual appeal — a rule that exists precisely because the violation was common. Note that compliance with “linear to 16 bits” still leaves every other ambiguity above unresolved.

Recommended policy for a multi-project benchmark:

1. **Default: exclude intensity from the model input.** The cost is real but bounded; the benefit of eliminating a large, project-correlated nuisance variable is larger.
2. **Run the ablation.** Train with and without intensity, and evaluate both under leave-one-project-out splits. If intensity helps in-domain and hurts out-of-domain, you have measured the nuisance directly, and that measurement is a publishable result.
3. **If you use it, normalize per-project** (rank-transform or quantile-normalize within project), which removes monotone reparameterizations but not AGC-induced within-project variation.
4. **Never use raw intensity as a feature in a task where labels are project-correlated.** That is a shortcut waiting to be learned.

Further reading and visual resources for Chapter 3

- [NEON L0-to-L1 Discrete Return Lidar ATBD \(NEON.DOC.001292\)](#) — a complete, public specification of a discretization algorithm. There is no better single document for understanding what §3.4 actually looks like in production.
- [NEON: Lidar data collection](#) — waveform vs. discrete, sampling interval, sensor history.
- Mallet & Bretar (2009), *Full-waveform topographic lidar: state-of-the-art*, ISPRS Journal 64(1) — the review to read for §3.3–3.4.
- Wagner et al. (2006), on Gaussian decomposition of small-footprint waveforms, ISPRS Journal — the original methodological treatment.
- For visuals of waveform decomposition, the figures in the Mallet & Bretar review and in the NEON ATBD are the ones to study; they show a real waveform with fitted components overlaid, which is the single most clarifying image in this chapter’s material.

Chapter 4. Sensor Heterogeneity as a First-Class Problem

Chapters 1–3 established that a large number of ALS data properties are set by hardware, mission design, and processing rather than by the landscape. This chapter is about what that means for a machine-learning benchmark, and it is the methodological hinge of the book.

The thesis: **in a multi-project ALS corpus, sensor and acquisition characteristics are not noise to be ignored — they are a confounder that is substantially collinear with geography, date, and land cover, and a benchmark that does not handle them explicitly will report generalization numbers that are wrong in an unknown direction.**

4.1 The three layers of variation

Layer 1 — Hardware. Wavelength; beam divergence and footprint; PRF and pulse energy; number of laser channels; scanner mechanism; detector type; discriminator design; dead

time and minimum separable range; maximum returns per pulse; intensity encoding; whether waveform is retained.

Layer 2 — Mission. Flying height and speed; scan rate and scan-angle limit; sidelap; number of cross-strips; time of day and sun angle; season and phenological state (leaf-on vs leaf-off); snow and soil-moisture state; atmospheric conditions; target pulse density; ground control density.

Layer 3 — Processing. Trajectory processing (differential vs PPK vs PPP; IMU grade); boresight calibration; strip adjustment method and residuals; waveform decomposition algorithm and thresholds; range-walk correction; intensity normalization and AGC handling; ground classification algorithm and parameters; noise/outlier filtering; overlap handling (removed / flagged / neither); tiling scheme; CRS and vertical datum transformations applied.

There are on the order of thirty independently varying parameters here, of which a LAZ file records perhaps five, and the associated project metadata records perhaps fifteen more if you go looking for it.

4.2 A worked example: NEON is a single program and still heterogeneous

NEON is the most tightly controlled large ALS program in existence, and it is *still* a heterogeneous corpus. Per NEON’s own [data collection documentation](#), the airborne sensor history includes an Optech Gemini (2013–2020), a Riegl Q780 (from 2018), an Optech Galaxy Prime (from 2021), and a Riegl VQ-780 II-S beginning in 2025 — spanning oscillating-mirror and rotating-polygon mechanisms, different footprints, different maximum return counts, and different intensity conventions.

The consequence: **a NEON site-year is a confounded variable**. If your model performs differently at site A in 2019 and site B in 2022, you cannot attribute that to site or to year without knowing which payload flew. Fortunately NEON makes this recoverable — the [per-site QA reports in the Metadata folder](#) record the sensor used, acquisition parameters, processing parameters, and QA information. Extract those fields. They are your ground truth for Layer 1 and much of Layer 2.

If NEON — one operator, one payload team, annual calibration, [cal-val flights at season start and end plus nominal runway flights per domain to check accuracy drift](#) — is this heterogeneous, then 3DEP, comprising hundreds of independent contractor collections across a decade and a half under a specification that governs deliverables rather than methods, is heterogeneous on a different scale entirely.

4.3 The collinearity problem, stated honestly

Here is the uncomfortable structural fact about 3DEP as a training corpus. A contractor wins a work unit covering some region, flies it in some season of some year with the payload they own, and processes it with their software stack. Therefore:

$$\text{sensor} \approx f(\text{region}, \text{year}, \text{contractor})$$

Sensor identity, geography, acquisition date, quality level, and often land cover are **collinear**

by construction. You cannot fully separate “the model failed to generalize to the Southeast” from “the model failed to generalize to the sensor that flew the Southeast in 2017.”

Three responses, in increasing order of ambition:

1. **Acknowledge and report.** Include sensor/project/QL/year as reported strata in all results, and be explicit that the axes are not orthogonal. This is the minimum standard and it is above what most of the literature does.
2. **Find the partial identifications.** Some regions have been flown more than once, by different contractors with different sensors, at different quality levels. Those repeat-coverage areas are where sensor and geography *do* separate, and they are worth locating explicitly and reserving as a dedicated evaluation stratum. They are the ALS equivalent of a natural experiment.
3. **Construct a paired design.** Where two different sensors covered the same footprint with the same reference labels, the difference in model performance isolates the acquisition effect. **[Case study]** This is the argument for evaluating over NEON footprints using *both* the NEON AOP cloud and the 3DEP cloud covering the same ground: the difference between those two numbers, with labels and geography held fixed, is a direct estimate of the sensor domain gap. I develop this design in Chapter 19.

4.4 What each variation does to a learned representation

A rough map from cause to effect, useful for predicting which of your tasks will be fragile:

Varying property	Direct effect on point cloud	Most-affected features/tasks
Pulse density	Sampling completeness of the vertical profile	Height percentiles (upper tail worst), cover metrics, gap detection, tree detection recall
Returns per pulse cap	Truncation of profile interior	Return-ratio metrics, LAD/foilage-profile inversion, stratification
Footprint size	Gap-vs-leaf discrimination; multiple-return rate	Canopy cover, gap fraction, crown delineation, ground-return fraction
Dead time / min. separable range	Near-ground blind zone	Understory, shrub cover, fuels, regeneration
Pulse energy / SNR	Depth of profile penetration	Mid- and lower-canopy density, LAD, biomass in dense canopy
Scan-angle distribution	Path length, occlusion geometry, footprint	Everything, with a swath-position gradient
Intensity convention	Semantics of the intensity channel	Species classification, land-cover semantics, anything using intensity
Ground classifier	DTM quality → height normalization	<i>All</i> height-above-ground features; catastrophic under dense canopy on slopes
Overlap handling	Local density doubling and bimodal scan angle	Density-sensitive features; also creates apparent spatial structure

Varying property	Direct effect on point cloud	Most-affected features/tasks
Leaf-on vs leaf-off	Presence of foliage	Deciduous cover, LAI, species discrimination, biomass

Two rows deserve special notice. **Ground classifier quality is the highest-leverage processing variable in the whole table**, because nearly every ecological feature is computed on height above ground, so a DTM error propagates into every feature simultaneously and coherently. And **leaf-on vs leaf-off is the largest single ecological confound in 3DEP**, because 3DEP’s terrain-mapping mandate favours leaf-off acquisition in deciduous regions while ecological interpretation usually wants leaf-on. A nationally stratified sample that ignores phenological state is not a representative sample of forests; it is a mixture of two different sensing regimes with a geographic gradient in the mixing proportion.

The empirical literature on density sensitivity is worth reading rather than reasoning from first principles. The broad finding from thinning experiments is that area-based height metrics are surprisingly robust down to roughly 1–2 pulses/m², with upper percentiles degrading first and cover/density metrics degrading faster than height metrics; individual-tree detection degrades much earlier. Search terms: *lidar point density thinning area-based approach forest attribute estimation*, and for a cross-sensor comparison, the sensor/resolution comparison literature cited in the [PyForestScan](#) references.

4.5 Four strategies, and my recommendation

Strategy 1 — Normalize away. Thin everything to the minimum common pulse density, truncate to the minimum common return count, discard intensity, restrict scan angle to a common range. Produces a homogeneous corpus at the cost of discarding most of the information in the good collections. Yields a lowest-common-denominator model. Appropriate as a *baseline*, not as the primary approach.

Strategy 2 — Condition on it. Feed acquisition characteristics to the model explicitly: per-point scan angle and return number, per-sample local pulse density, per-tile sensor embedding, QL, phenological state. Architecturally this can be input channels, a concatenated global vector, or FiLM-style conditioning of the encoder. The model can then learn *density-aware* features rather than density-confounded ones. Risk: at inference on a new sensor the conditioning vector is out of distribution, so you need a fallback and you should test it.

Strategy 3 — Enforce invariance. Domain-adversarial training against sensor/project labels, or contrastive objectives that treat two acquisitions of the same footprint as positive pairs. The latter is elegant and, in a corpus with repeat coverage, *free* — overlapping 3DEP collections over the same ground give you genuine multi-view positive pairs with no annotation cost. I think this is the most promising underexplored idea in this space, and it connects directly to the self-supervised pretraining design of Chapter 22.

Strategy 4 — Augment by simulated degradation. Since you can realistically degrade high-density data toward low-density but not the reverse, build an augmentation pipeline that samples a synthetic acquisition regime for each training sample:

- **Pulse thinning** to a randomly sampled target density. Critically, thin *pulses* (group points by GPS time / pulse, drop whole pulses), never individual points — random point dropout destroys the return structure and produces clouds unlike any real sensor.
- **Return truncation** to a random cap in $\{1, 2, 4, 8\}$, retaining the strongest or the first-N components.
- **Near-ground blanking**: drop non-ground returns within a randomly sampled 0.5–3 m of the ground surface, simulating varying dead time.
- **Scan-angle resampling**: reweight or subset points to simulate a different swath-position distribution; optionally simulate single-swath-only by filtering on `PointSourceId`.
- **Intensity corruption**: random monotone reparameterization, or channel dropout entirely.
- **Vertical noise**: small z jitter to simulate ranging and strip-adjustment residuals, and — worth trying — an *intensity-correlated* z bias to simulate uncorrected range walk (§3.2).
- **Systematic density modulation**: a low-frequency multiplicative field to simulate turbulence striping and overlap stripes.

Recommendation. Use Strategy 4 as the default because it is cheap, requires no architectural change, and directly simulates the deployment distribution; combine it with Strategy 2 for the covariates that are reliably available (scan angle, return number, local density); reserve Strategy 3 for a dedicated experiment on repeat-coverage pairs; and report Strategy 1 as a baseline so readers can see what the homogenization approach costs. Then evaluate all of it under **leave-one-sensor-out** and **leave-one-quality-level-out** splits alongside the geographic splits, and report all of them.

The augmentation set above is, I think, the most immediately actionable thing in Part I. It is a few hundred lines of code, it is not in any published ALS-ML pipeline I am aware of in this combined form, and if sensor robustness is a real problem it should show up as a large effect.

4.6 The provenance schema

Everything in this chapter argues for one concrete engineering artifact: **every sample in your dataset carries a provenance record**. Proposed minimum fields, to be finalized in Chapter 15:

Identity — project/work-unit ID; tile ID; sample geometry (WKT footprint); CRS and vertical datum, with the transformation history if any was applied.

Acquisition — acquisition date range; sensor make and model; wavelength; scanner mechanism; beam divergence; nominal flying height; PRF; maximum returns per pulse; minimum separable range if documented; scan-angle limit; number of channels.

Specification — USGS quality level; the LBS version in force at collection; nominal pulse spacing/density as specified.

Realized — measured pulse density and point density for this sample; returns-per-pulse distribution; scan-angle distribution summary (mean $|\theta|$, quantiles); overlap fraction; number of distinct `PointSourceId` values; class-code histogram; fraction flagged Withheld / Synthetic / Overlap; z range.

Ecological context — EPA Level III/IV ecoregion; land-cover class distribution; phenological state (leaf-on / leaf-off / indeterminate) inferred from date and ecoregion; terrain slope statistics.

Processing — ground classification source (vendor / re-derived and by which algorithm); height normalization method; whether intensity was retained and how normalized.

Six of these fields cost nothing (they come from the file header and a WESM lookup); a dozen require computing summary statistics you’re already reading the points for; a handful require parsing project metadata reports, which is genuinely tedious and genuinely worth it. **A benchmark whose negative results cannot be sliced by acquisition characteristics has produced uninterpretable negative results.** This schema is the difference.

Further reading for Chapter 4

- [USGS Lidar Base Specification](#), [online edition](#) and the [revision status page](#). Read the current version, then read what changed, because your corpus spans multiple versions.
- [NEON: Lidar data collection](#) for the sensor timeline and calibration regime.
- *Advancing Aerial Laser Scanning Applications with Large-Scale Pre-Training* — the closest prior work to a nationally-scaled 3DEP pretraining corpus. Read its sampling design and, more importantly, its negative results by terrain class.
- [FRACTAL](#) — a national ALS ML dataset built with explicit diversity sampling, whose source data was itself acquired with multiple sensor families (Leica, Riegl, Teledyne/Optech) at a target 10 pulses/m². The paper’s discussion of source heterogeneity and its companion sampler [pacasam](#) are directly applicable.
- For the sensor-comparison empirical literature, the most useful search framing is “*comparing sensors*” OR “*cross-sensor*” *airborne lidar forest structure metrics transferability*, which surfaces the studies that flew or thinned multiple configurations over common plots.

Bridge to Part II

Part I has been about the instrument. Part II is about the institution: how flight plans are designed, what USGS 3DEP and NEON AOP each specify and why their specifications differ in kind, and what the resulting documentation lets you recover about Layers 2 and 3 of §4.1. Chapter 5 covers flight planning and mission geometry; Chapter 6 works through 3DEP’s specification-driven model; Chapter 7 works through NEON’s research-observatory model; Chapter 8 surveys the other national programs you may need for transfer evaluation.

The through-line to keep in mind: 3DEP tells contractors *what to deliver*; NEON tells its own staff *what to do*. That single difference explains almost every downstream contrast between the two corpora, including why 3DEP is the better pretraining corpus and NEON the better evaluation corpus.

Part II — Acquisition in Practice

Airborne Laser Scanning for Machine Learning in Ecology

Orientation to Part II

Part I was about the instrument. Part II is about the *institution*: who decides where and how to fly, what they are contractually obliged to produce, and what documentation survives into the archive you will train on.

The organizing contrast of this Part is a difference in kind, not degree. **USGS 3DEP tells contractors what to deliver. NEON tells its own staff what to do.** A specification governing deliverables produces a corpus that is uniform in *format* and wildly variable in *method*. A protocol governing procedure produces a corpus that is uniform in method and narrow in extent. Nearly every downstream contrast between the two — including why 3DEP is the better pretraining corpus and NEON the better evaluation corpus — follows from that one sentence.

Four chapters:

- **Chapter 5** develops mission design: the coupled variables, a worked flight plan, coverage geometry, and the environmental windows that constrain when flying is possible at all. It also dismantles a specific and widespread misunderstanding about what a “2 points per square metre” specification means.
- **Chapter 6** works through 3DEP: quality levels, what the Lidar Base Specification requires, what it conspicuously declines to require, the classification and bit-flag regime (including a correction to Part I), and the accuracy overhaul now working through the system.
- **Chapter 7** works through NEON AOP, whose design logic is colocation rather than coverage, and whose known biases are documented well enough to reason about.
- **Chapter 8** surveys the other programs you will need for transfer evaluation, plus the non-ALS platforms whose data you may be tempted to mix in.

A note on how to read the specification material in Chapter 6. It is tempting to skim it as bureaucratic detail. Resist that. The specification is the only systematic description you will ever get of how your training data was made, and three or four of its clauses — the withheld-flag regime, the density-assessment protocol, the removal of overlap and scan-angle requirements — determine whether your pipeline is correct or silently broken.

Chapter 5. Flight Planning and Mission Geometry

5.1 The coupled variables

A lidar mission has roughly eight free parameters, and they are not independent. Choosing any five approximately determines the rest.

Parameter	Typical range (airborne topographic)	Primarily traded against
Flying height AGL	500–3000 m	Pulse energy budget, footprint, swath width, eye safety, airspace
Ground speed	100–180 kt	Line spacing, along-track density, project duration and cost
Field of view (full scan angle)	20°–60°	Swath width vs. edge-of-swath data quality
Scan rate	10–150 Hz	Along-track vs. across-track spacing balance
PRF	100 kHz–2 MHz	Per-pulse energy, pulses in air, achievable density
Sidelap	20%–60%	Cost (flight-line count) vs. gap risk and redundancy
Line spacing	derived	Swath width and sidelap
Target pulse density	0.5–40 pls/m ²	Everything above; usually the <i>specified</i> quantity

The tightest coupling is between height, PRF, and density. Halving flying height quarters the footprint area, roughly quadruples return power (§1.2), and halves swath width — so density per pass doubles while the number of passes needed also doubles. Total pulse count for a fixed area is conserved at fixed PRF and speed; what changes is footprint, penetration, and cost.

The relations from §2.2, restated for design use:

$$W = 2h \tan \theta_{\max} \quad \Delta_{\text{line}} = W(1 - s) \quad \text{ANPD} \approx \frac{\text{PRF}}{v \cdot \Delta_{\text{line}}}$$

where s is fractional sidelap. Note that ANPD in the last expression is the *aggregate* density over the full project including sidelap, which is not the same quantity USGS assesses — see §5.4, which is the most practically important section in this chapter.

5.2 A worked mission

Take a plausible QL1 forest collection.

- Flying height $h = 1200$ m AGL
- Full FOV = 40°, so $\theta_{\max} = 20^\circ$
- PRF = 500 kHz
- Ground speed $v = 140$ kt = 72 m/s
- Sidelap $s = 0.30$

Then:

- Swath width $W = 2(1200) \tan 20^\circ = 873$ m
- Line spacing $\Delta_{\text{line}} = 873 \times 0.70 = 611$ m
- Single-swath nadir density = $\text{PRF}/(vW) = 500,000/(72 \times 873) = 7.9$ pls/m²
- Aggregate density including sidelap = $500,000/(72 \times 611) = 11.4$ pls/m²
- Nadir footprint at $\gamma = 0.25$ mrad: $1200 \times 0.00025 = 30$ cm

- Edge-of-swath slant range = $1200 / \cos 20^\circ = 1277$ m; edge footprint major axis $\approx 30 / \cos^2 20^\circ = 34$ cm
- Pulses in air: round trip at nadir = $8.0 \mu\text{s}$, pulse interval = $2.0 \mu\text{s}$, so 4 pulses in flight
- Time to cover 100 km^2 : $100 \times 10^6 / (611 \times 72) \approx 2270$ s of on-line time ≈ 38 minutes, before turns, transits, and calibration lines — realistically a 2–3 hour sortie

Two things worth extracting. First, this mission comfortably exceeds QL1's 8 pls/m^2 *in aggregate* but only marginally meets it *in single swath* — and the second is the number that governs specification compliance. Second, the difference between nadir and edge footprint (30 vs. 34 cm) is modest at 20° , which is why moderate FOVs are preferred for vegetation work; at $\theta_{\max} = 30^\circ$ the edge footprint would be 40 cm and the slant range 1386 m, with correspondingly worse penetration.

5.3 Coverage patterns and calibration geometry

The Lidar Base Specification illustrates three collection geometries, and the distinction matters because it changes what is in your tiles:

Single coverage, no overlap. Adjacent swaths abut with minimal sidelap. Cheapest; no redundancy; any attitude excursion produces a gap. Rare in modern practice.

Designed double coverage, 50% overlap. Line spacing set to half the swath width, so every ground point is imaged twice from two different scan angles. This doubles aggregate density, doubles cost, and provides excellent redundancy and two-look geometry — which for vegetation means substantially reduced occlusion, since each crown is sampled from two directions. If you find a 3DEP project whose tile density is roughly twice what the QL implies, designed double coverage is the likely reason.

Cross-flights with 20% overlap. Two orthogonal (or oblique) sets of flight lines. The cross geometry is the strongest available constraint on boresight and strip-adjustment parameters, so cross-flights (or at least a few cross-strips) are standard practice for calibration even when the primary coverage is single-direction.

Calibration lines. Beyond project coverage, missions typically include: flights over a surveyed calibration site or runway at the start and end of a sortie or season; opposing-direction line pairs over a flat, hard surface to solve pitch and roll boresight; and lines at two different altitudes to separate scale-like errors. These lines exist in the swath deliverables and sometimes in the archive; they are not part of the tiled product.

[Case study] Coverage pattern is a per-project property that strongly affects tile density and occlusion structure, and it is *not* recorded in the LAZ header. It is usually described in the project's collection report. If you are sampling nationally, expect the coverage-pattern mixture to vary regionally with contractor and with the era of the collection. Recording it per-project requires reading reports; recording its *consequence* (measured tile density, number of distinct `PointSourceId` values per unit area, scan-angle distribution shape) requires only computing statistics you're already reading the points for. Do the latter always and the former where you can.

5.4 What “2 points per square metre” actually means — and the trap it sets

This section exists because the misunderstanding it addresses is nearly universal outside the surveying community, and because it will cause you to mislabel your data if you get it wrong.

The Lidar Base Specification’s density requirement reads, in the current [Collection Requirements](#): Aggregate Nominal Pulse Density shall be no less than 2 points per square metre for QL2, with **assessment made against single-swath, first-return data located within the geometrically usable centre portion (typically about 95%) of each swath**. Aggregate Nominal Pulse Spacing shall be no greater than 0.70 m for QL2, assessed the same way.

Unpack that. The specified number is:

- **per pulse, not per point** — it counts *first returns*, i.e. one per pulse, so multiple returns from vegetation do not count toward it;
- **single swath** — sidelap contributions do not count toward it;
- **central 95% of the swath** — the edge region where the oscillating-mirror density piles up (§2.1) is excluded.

Therefore, for a QL2 project flown with 30% sidelap over forest with an average of 2.5 returns per pulse, the density of your *delivered tile* could plausibly be:

$$2 \frac{\text{pls}}{\text{m}^2} \times \frac{1}{0.70} (\text{sidelap}) \times 2.5 \frac{\text{returns}}{\text{pulse}} \approx 7 \frac{\text{points}}{\text{m}^2}$$

more than three times the specified figure. Meanwhile a QL2 project over bare desert with 1.05 returns per pulse and minimal sidelap might deliver 2.2 points/m². **Two QL2 tiles can differ by a factor of three in point density, entirely legitimately.**

Consequences you should act on:

1. **Never use quality level as a proxy for point density.** Compute density per sample and store it (§4.6). QL is a *specification*, and specification is not realization.
2. **Report pulse density and point density separately**, and derive returns-per-pulse. Pulse density is recoverable: group by `GpsTime` (the specification requires timestamps precise enough to be unique per pulse — see §6.6), or count points with `ReturnNumber == 1`. The latter is cheaper and is what USGS itself uses.
3. **When comparing a NEON collection to a 3DEP collection, compare pulse densities**, because comparing point densities compares two different sensors’ return-extraction behaviour rather than their sampling.
4. **When thinning for augmentation, thin to a target pulse density**, for the same reason (§4.5).

A related definition worth knowing: a **data void** under the specification is any area at least $(4 \times \text{ANPS})^2$ in size, measured on first returns only. For QL2 that is a 2.8 m square. Voids larger than this are non-compliant unless caused by water or a legitimate absence of return. So the specification polices *gaps* tightly while leaving *excess* density unconstrained — an asymmetry that explains a lot about the shape of the archive.

5.5 Environmental windows

Phenology. The single largest ecological lever in mission design, and the axis on which 3DEP and NEON differ most sharply. Leaf-off collection maximizes ground returns and therefore DTM quality, which is 3DEP’s mandate; leaf-on collection characterizes the canopy, which is what ecological interpretation usually wants. In deciduous regions this is a genuine either/or. Note that “leaf-off” is not a binary: early leaf-off with senescent foliage still attached, mid-winter bare, and late leaf-off with buds breaking are three different sensing regimes, and conifer components are leaf-on year-round, so a mixed stand flown leaf-off yields a cloud in which the deciduous and coniferous fractions are measured under different conditions. Any species-discrimination task on leaf-off data is partly a task of detecting *which trees still have leaves*.

Snow. Snow cover is disqualifying for terrain mapping (the surface is not the ground) and for canopy work (branch loading alters structure). At 1064 nm snow is highly reflective and produces a clean bright surface, which makes the contamination easy to miss in QC if the snow is thin and continuous.

Soil moisture and standing water. Saturated soil and ephemeral standing water absorb strongly at 1064 nm, producing dropouts. Specifications typically require collection during low-water conditions and prohibit flying during or immediately after significant precipitation. Ecologically this means wetland collections are biased toward the dry end of the hydroperiod.

Wind. Crown motion during a pass, and between passes in sidelap, produces genuine disagreement in canopy-top elevation. There is usually a wind ceiling in the flight plan. For repeat-collection change detection, wind is a floor on detectable change.

Sun angle and solar background. High solar irradiance in the receiver band degrades SNR (§3.1). Some specifications constrain sun angle; night flying is used for some coastal and bathymetric work.

Smoke and haze. Two-way exponential attenuation (§1.3). Increasingly a binding constraint on western US summer acquisition, with the consequence noted in §1.3: acquisition-window availability is itself correlated with fire regime.

Aircraft and airspace. Restricted airspace, terrain clearance in mountains (which forces either higher flying or terrain-following with variable AGL and therefore variable density), and the practical requirement of a nearby airfield. Terrain-following is worth flagging: over high relief, a constant-*altitude* flight produces AGL variation of hundreds of metres within one line, and hence systematic density and footprint variation correlated with elevation. In mountains, elevation, density, and footprint are confounded.

5.6 Ground control and check points

Two distinct point sets with distinct roles, frequently conflated:

Control points are surveyed positions used *in* the calibration and adjustment process — to solve for a vertical bias, to constrain strip adjustment. They are inputs.

Check points are an independent set used *only* to assess accuracy after the fact. The specification is explicit that [control points used in the calibration process shall not be used as check](#)

points, which is the surveying equivalent of not evaluating on your training set.

The check-point protocol is unusually detailed and worth knowing because it defines what accuracy claims mean. Under the current specification: check points are surveyed separately for non-vegetated (NVA) and vegetated (VVA) assessment; they are located in areas with a minimum homogeneous extent of $(ANPS \times 5)^2$; on low-slope ground (under 10 degrees) with less than a third of the required RMSEz of deviation from a plane; with no obstructions above 15 degrees over the horizon except in forested and dense-urban cover; distributed across the project and among land-cover types in proportion to their areas; and every tested location photographed. Recent revisions raised the minimum from 20 to 30 check points and capped large projects at 120.

Read that list again with an ML eye. **The reported vertical accuracy of a 3DEP project is measured on flat, homogeneous, open, low-slope ground.** VVA extends it to vegetated cover but under the same flat-and-homogeneous siting constraints. Nothing in the accuracy statement tells you about vertical error on a 25-degree forested slope, which is precisely where DTM error is largest and where it propagates into every height-above-ground feature you compute. A project reporting 10 cm NVA can easily have metre-scale ground-surface error under dense canopy on steep terrain. Do not treat the reported accuracy as an error bar for your features.

5.7 What a flight plan leaves in the data

Summarizing Chapter 5 as a set of recoverable signatures — things you can measure in a tile that tell you about the mission:

Measurable in the tile	Tells you about
Distinct <code>PointSourceId</code> count and their spatial arrangement	Flight-line layout, coverage pattern, sidelap geometry
Scan-angle histogram (unimodal vs. bimodal vs. flat-topped)	Scanner type, FOV, whether the sample spans sidelap
Density map at 5–10 m resolution	Sidelap stripes, turbulence striping, atmospheric dropout, terrain-following AGL variation
First-return density vs. total point density	Returns per pulse, hence canopy structure <i>and</i> sensor return-extraction behaviour
<code>GpsTime</code> range and gaps	Number of passes, time separation between passes, sortie structure
Ground-return fraction vs. scan angle	Canopy penetration geometry; also a QC signal for ground classification
Fraction of points with <code>Withheld</code> set	Noise burden, and whether the project follows the modern flagging regime (§6.4)

All seven are cheap and none are standard practice. I would compute all seven per sample and store them in the provenance record. Together they reconstruct a surprising amount of the mission design without reading a single report.

Further reading and visual resources for Chapter 5

- [Lidar Base Specification: Collection Requirements](#) — the source for §5.4. Read the

ANPD/ANPS clauses word for word.

- [Lidar Base Specification Online: Table of Contents](#) — links to Tables 1–6 and, importantly, to the three collection-geometry diagrams described in §5.3. Those diagrams are the visual reference for this chapter.
- [NEON: Flight box design](#) and [Flight schedules and coverage](#) — a mission design fully documented in public, useful as a concrete counterexample to the contractor model.
- Baltsavias (1999), *Airborne laser scanning: basic relations and formulas* — the algebra of §5.1–5.2.

Chapter 6. USGS 3DEP: The Specification-Driven Model

6.1 What 3DEP is

The 3D Elevation Program is a partnership-funded acquisition program, not a survey organization. USGS publishes a specification, solicits partner contributions (state agencies, counties, federal partners, utilities), aggregates them into work units, and contracts acquisition to private geospatial firms through vehicles like the Geospatial Products and Services Contracts. The contractor flies, processes, and delivers; USGS and its quality-assurance partners verify against the specification; the data enters the national holdings and is released as public domain.

Three structural consequences follow, and they are the reasons 3DEP looks the way it does.

Consequence 1: coverage is driven by partner funding, not by scientific sampling design. Areas with motivated, funded partners — flood-prone regions, states with active geospatial programs, areas with utility or infrastructure interest — were flown early and often at higher quality levels. Areas without such partners were flown late, at minimum quality, or not at all until the national completion push. **[Case study]** A “representative sample of 3DEP” is therefore not a representative sample of the United States along any ecologically meaningful axis unless you explicitly stratify. The archive’s own spatial density of coverage encodes political economy, not ecology.

Consequence 2: the unit of homogeneity is the project, not the region. Everything a contractor does — sensor, altitude, season, software, classification approach — is constant within a work unit and can change discontinuously at its boundary. Project boundaries are therefore the natural blocking unit for cross-validation, and adjacent tiles across a project boundary can differ more than tiles a thousand kilometres apart within one project.

Consequence 3: the specification governs deliverables, and therefore methods vary maximally subject to the deliverable constraints. This is the theme of §6.3.

6.2 Quality levels

The [quality-level definitions](#) as currently published:

QL	Source	Vertical accuracy RMSEz	Nominal pulse spacing	Nominal pulse density	DEM cell size
QL0	Lidar	5 cm	at most 0.35 m	at least 8 pls/m ²	0.5 m
QL1	Lidar	10 cm	at most 0.35 m	at least 8 pls/m ²	0.5 m
QL2	Lidar	10 cm	at most 0.71 m	at least 2 pls/m ²	1 m
QL3	Lidar	20 cm	at most 1.41 m	at least 0.5 pls/m ²	2 m
QL4	Imagery (photogrammetric)	139 cm	—	—	5 m
QL5	IfSAR	185 cm	—	—	5 m

A quality level is thus a *pair* of requirements: density and vertical accuracy. QL0 and QL1 have identical density and differ only in accuracy — **QL0’s required NVA and VVA are twice as good as QL1’s**, with correspondingly stricter check-point survey requirements. QL1 as currently defined is, in the specification’s own words, a hybrid: QL2’s accuracy class with QL0’s pulse density.

The minimum acceptable quality level for 3DEP collections is QL2. Some QL3 data exists in the holdings from earlier eras and from certain regions (notably Alaska, much of which is IfSAR-derived QL5 rather than lidar at all).

The quality levels have been revised. QL1 and QL2 accuracy moved from 9.25 cm to 10.0 cm RMSEz; QL3 moved from 18.5 cm to 20.0 cm and its density from 0.7 to 0.5 pls/m². The pre-revision figures appear in older project metadata. If you parse quality-level metadata programmatically, expect both vocabularies.

[**Case study**] For a national pretraining corpus, the practical distribution is roughly: QL2 is the workhorse and covers the majority of CONUS; QL1 appears where partners paid for it, concentrated in urban, coastal, and infrastructure-heavy areas; QL0 is rare and project-specific; QL3 and QL5 dominate Alaska. That distribution is geographically structured, so **quality level is a confounder with land cover and region**, and belongs in your stratification and in your leave-one-out evaluation axes.

6.3 What the specification requires — and what it declines to require

The requirements side is extensive: density, accuracy, classification, format, tiling, datum, deliverables, reports, and proof-of-performance artifacts. But the *omissions* are more informative, because they mark the axes along which your corpus varies without constraint.

Removed or never specified:

- **Flight-line overlap and scan-angle limits.** These requirements were removed from the specification, with data producers instead cautioned that gaps and density shortfalls would be scrutinized. So 3DEP does not tell contractors how much to overlap or how

wide to scan; it tells them the resulting sample must have no voids and must meet single-swath central-region density. **Sidelap and FOV therefore vary freely across your corpus**, and with them the occlusion geometry and swath-position density structure of every tile.

- **Sensor, altitude, speed, PRF.** Entirely at the contractor’s discretion, subject to meeting the deliverable constraints.
- **Waveform.** References to waveform collection and delivery were *removed* from the specification in the 2022 revision. 3DEP does not want, receive, or archive waveform data. Whatever full-waveform information existed at acquisition is gone, and there is no path to recovering it.
- **The discretization algorithm.** The specification requires only that collection be *capable of at least three returns per pulse* and that intensity be recorded for each. How many returns are actually extracted, by what algorithm, with what thresholds, is unconstrained (§3.4).
- **The ground-classification algorithm.** The specification defines the required classes and assesses the resulting DEM against check points on flat open ground (§5.6). It does not specify the algorithm, its parameters, or any assessment of ground-surface quality under canopy on slopes.

That last omission is the most consequential single fact in this chapter. **The variable with the largest influence on your ecological features — ground-surface quality under vegetation — is neither specified nor systematically assessed.** I develop the practical response in Chapter 10 (whether to trust vendor ground classification or re-derive it yourself), but the conclusion is worth stating early: for a benchmark, re-deriving ground with one consistent algorithm is defensible and probably correct, and it should at minimum be run as an ablation against vendor ground.

6.4 Classification, bit flags, and the withheld regime

The minimum classification scheme

The specification defines a minimum required classification (its Table 5) and permits additional classes per project. The core requirements to understand:

- **All points within the minimum scheme and not flagged withheld must be properly classified.**
- **No points may remain in Class 0** in the classified deliverable, *unless those points are flagged as withheld*.
- **Point classification must be consistent across the entire project.**
- **Class 20, Ignored Ground**, is used for vertical, near-vertical, or overhanging bare-earth surfaces where including them in the DEM would produce interpolation artifacts. So class 20 is *ground that is deliberately excluded from the terrain model* — a distinction that matters if you build a DTM from class 2 alone versus classes 2 and 20.
- **Model key points**, if computed, are identified via the key-point bit flag, optionally also via class 8.
- Water classification is only verified where associated with a breakline.

The withheld flag, and why it is the most important bit in your pipeline

Starting with LBS 2020 rev. A, use of the withheld bit flag is **required for all points that are not valid surface returns**, and — this is the crucial clause — [the withheld bit flag supersedes any classification applied to these points](#); provided it is applied appropriately, any classification, including class 0, is acceptable.

Read that as an instruction to your data loader: **you cannot determine point validity from the classification code. You must read the withheld flag.** A point may carry class 1 (unclassified), class 0, or class 7/18 (noise) and be withheld; a filter on classification alone will admit invalid returns into your training data. The failure mode is specific and nasty: withheld points are disproportionately high-altitude noise (birds, clouds, multipath, MPiA ghosts), so a pipeline ignoring the flag will occasionally see canopy-height maxima of hundreds of metres, and any model or metric sensitive to the upper tail of the height distribution — p99, maximum height, rumple — will be corrupted at exactly the samples where it matters.

USGS takes this seriously enough to require proof of performance: **Maximum Surface Height Rasters**, built from the highest non-withheld point per pixel, delivered as a metadata product. [Properly flagged data produces a raster of uniform appearance](#); [improper flagging produces visible non-uniformity](#). This is, incidentally, an excellent QC idea to steal: computing an MSHR over your own sampled tiles is a fast, visual, whole-corpus check for flagging failures and outlier contamination.

Practical filter policy. In PDAL terms, the correct default first stage on 3DEP data is to drop withheld points, which `filters.range` and `filters.expression` can express via the classification-flags dimensions. Do this before computing anything, including your provenance statistics. Then separately record *what fraction* you dropped, because it is a useful QC signal and because a project with zero withheld points is either pristine or pre-2020.

Correction to Part I, §2.4

Part I stated that 3DEP overlap points are variously delivered flagged, removed, or neither. That is accurate for the historical corpus, but I under-specified the current rule and should sharpen it, because it changes what you should expect in a modern tile:

- **The overlap bit flag shall not be used** for lidar point clouds intended for inclusion in 3DEP holdings.
- **Classification codes shall not be used in lieu of the overlap bit flag** to identify overlap points — a clarification added in the 2022 revision, closing the older practice of marking overage with class 12.
- **Tiled delivery is without overlap**, using the project tiling scheme.

The resolution of the apparent contradiction is that overage points are *removed during tiling* rather than flagged, so in a current-specification tiled deliverable there is nothing to flag. The full sidelap redundancy exists in the swath deliverables, not the tiles.

For your corpus this means: **modern 3DEP tiles are single-coverage-per-point but multi-flight-line in composition**, so you will still see multiple `PointSourceId` values and

a bimodal scan-angle distribution near line seams, and you will still see density and geometry discontinuities at seams — but not the doubled density that raw sidelap would give. Older projects in the archive may show class 12, a set overlap flag, or doubled density. Detect empirically rather than assuming; the diagnostics in §5.7 will tell you which regime a tile is in.

This also revises the §2.4 recommendation: **[Case study]** rather than deciding whether to drop overlap points, record per-sample seam structure (distinct `PointSourceId` count, scan-angle bimodality, local density variance) and treat *that* as the covariate. And if you want genuine two-look sidelap geometry — for multi-view self-supervision, per §4.5 Strategy 3 — you need the swath deliverables, not the tiles. That is a meaningful data-acquisition decision to make early, since swath files are larger, less curated, and less conveniently indexed.

6.5 The accuracy regime, and the transition now in progress

Accuracy reporting is being overhauled, and your corpus will span the transition.

The older regime used FVA/SVA/CVA (fundamental, supplemental, consolidated vertical accuracy) and reported at 95% confidence. These were replaced by **NVA** (non-vegetated vertical accuracy) and **VVA** (vegetated vertical accuracy). Currently, USGS has adopted the second edition of the ASPRS Positional Accuracy Standards, which brings several changes worth knowing:

- **RMSE terminology renamed**, with RMSEV replacing RMSEz in the new vocabulary.
- **95% confidence level no longer used** in the accuracy tables.
- **VVA is reported but is no longer pass/fail** — a project can no longer fail solely on VVA. NVA must be met; VVA must be reported.
- **Survey error must now be included in RMSEV calculations** — incorporated by reference from the ASPRS standard rather than stated in the LBS itself.
- **Horizontal accuracy** reported as a stated RMSEH accuracy class.

The ML-relevant reading of this: **the accuracy figure that is actually enforced is the one measured on open, flat, non-vegetated ground, and the vegetated figure is now explicitly non-binding**. Combined with §5.6's siting constraints, this means the archive contains no systematic information about vertical accuracy under forest on slopes. If your benchmark reports canopy height RMSE of 1.5 m, you cannot decompose that into model error versus DTM error using published accuracy metadata. The only route to that decomposition is direct comparison against independent high-quality ground truth — surveyed plot ground elevations, or a co-located higher-quality collection.

Also required, and useful to you: **swath separation images** (interswath relative accuracy proof), and, since the 2022 revision, **delivery of survey point data in a GeoPackage**. The latter is genuinely valuable — it gives you the project's own check-point locations and surveyed elevations in a machine-readable format, which is a free, if sparse, independent vertical reference.

6.6 Deliverables

The [deliverables specification](#) defines what exists in the archive. Abbreviated, with the ML-relevant details:

Point cloud. - LAS specification version 1.4-R15, **Point Data Record Format 6, 7, 8, 9, or 10** — so all modern 3DEP data uses the newer formats with the 4-bit return-number fields, the separate classification-flag byte, and scan angle in finer units than the legacy rank (§12.2). - **Delivered as LAZ 1.4**, with “compatibility mode” compression explicitly prohibited. Compatibility mode is the mechanism that encodes 1.4 point formats inside a 1.2-style structure for legacy readers; prohibiting it means you get real 1.4 chunked LAZ, which is what makes dimension-selective streaming and COPC conversion clean. - **Withheld bit flag set as appropriate** (§6.4). - **WKT georeferencing in every file header**, correctly formatted. Note that LAS 1.4 with PDRF 6–10 requires WKT rather than the older GeoTIFF-style keys; if you encounter GeoTIFF keys, you are looking at older or non-compliant data. - **GPS times as Adjusted GPS Time, at precision sufficient for unique per-pulse timestamps.** This clause is more useful than it looks: it means you can reliably group points into pulses by timestamp, which is what makes pulse-level thinning (§4.5) and per-pulse profile analysis possible on 3DEP data. - **Intensity linearly scaled to 16 bits.** - **Tiled delivery without overlap** on the project tiling scheme; also **swath LAS** files as a separate deliverable. - Classification per Table 5 minimum.

Bare-earth DEM. Raster, cell size per quality level (Table 6), generated to the limits of the Defined Project Area, NODATA of -999999 declared in the GDAL_NODATA tag, same CRS as the point data, tiled without overlap, no edge artifacts between tiles. Note that all ground-classified points not flagged withheld are used in DEM creation — so the DEM and a class-2-only surface you build yourself should agree closely, and disagreement is a QC signal.

Breaklines where required, typically hydro-flattening breaklines for water bodies and streams.

Metadata and reports. Collection report, survey report, processing report, QC reports, tile index, swath separation images, Maximum Surface Height Rasters, survey points in GeoPackage. This is where Layers 2 and 3 of §4.1 are documented, project by project, in prose.

The Defined Project Area is the Area of Interest plus a **100 m buffer**. (Earlier specification versions used a separate “Buffered Project Area” concept; that language was eliminated and folded into the DPA.) Practically: expect a 100 m collar of data outside the nominal project boundary, and expect the tiles at the boundary to be partially populated.

6.7 Reading a project’s metadata

The discovery path, developed fully in Chapter 15 but sketched here because Chapter 6 is where you will first want it:

1. **WESM** (Work Unit Extent Spatial Metadata) is the authoritative index of 3DEP work units: geometry, project name, quality level, acquisition date range, and the LBS version in force. Given a point or polygon, WESM tells you which project covers it and with what nominal characteristics. This is the single most valuable metadata resource for your provenance schema and it is machine-readable.
2. **The National Map / Lidar Explorer** for interactive discovery and for locating the per-project metadata bundles.
3. **The project’s own reports**, which carry the sensor model, altitude, PRF, coverage pattern, and processing workflow. These are PDFs, they are inconsistent in structure,

and extracting them at national scale is genuinely tedious. Consider extracting them only for the projects that intersect your evaluation strata, where the metadata matters most, and relying on measured statistics (§5.7) elsewhere.

6.8 What all of this means for a pretraining corpus

Summarizing Chapter 6 as design guidance.

3DEP’s strengths as a pretraining corpus are real and they are the reason to use it: national extent, public domain, a uniform delivery format enforced by specification, machine-readable project-level metadata via WESM, cloud-native access (Chapters 13–14), and enough volume that data quantity will not be your binding constraint.

3DEP’s weaknesses are: coverage driven by funding rather than sampling design; unconstrained variation in sensor, mission, and processing; no waveform; a leaf-off bias in deciduous regions; ground-surface quality neither specified nor assessed under vegetation; a quality-level mixture that is geographically structured; and acquisition dates spanning well over a decade of sensor evolution.

The design implications, which I’ll carry into Chapter 24:

1. **Stratify explicitly** on ecoregion, quality level, acquisition year, and phenological state. Do not rely on the archive’s own spatial distribution for representativeness.
2. **Block cross-validation on project boundaries**, not on arbitrary spatial grids, because the project is the unit of homogeneity.
3. **Read the withheld flag**. This is a correctness requirement, not an optimization.
4. **Re-derive ground classification** with one consistent algorithm, and ablate against vendor ground. The variable with the largest influence on your features is the one the specification leaves unconstrained.
5. **Compute, don’t assume, density** (§5.4).
6. **Record LBS version in force**, from WESM, per sample. It determines what the flags and classes mean.
7. **Decide about swath files early**, because two-look sidelap geometry is only available there and it is the natural substrate for multi-view self-supervision.

Further reading for Chapter 6

- [Lidar Base Specification Online](#) — the current version. The sections most worth reading in full: Collection Requirements, Data Processing and Handling, Deliverables, and Appendix 1.
- [Revision History](#) and [Revision Status](#) — what changed, and what is about to. Your corpus spans versions; you need both.
- [Topographic Data Quality Levels](#) — the QL table and the QL0 discussion.
- [Appendix 1: Maximum Surface Height Rasters](#) — read this for the withheld-flag regime and steal the MSHR idea for your own QC.
- [Adopt updated accuracy standards](#) — the ASPRS Edition 2 transition.
- For historical comparison, the archived [LBS version 2.1 PDF](#) and the [2022 rev. A PDF](#) are useful for interpreting older projects.

Chapter 7. NEON AOP: The Research-Observatory Model

7.1 Design logic: colocation, not coverage

NEON’s Airborne Observation Platform exists to connect ground measurements to landscape-scale remote sensing. That single purpose explains every design choice, and it is the opposite of 3DEP’s purpose.

Where 3DEP asks “how do we map the elevation of the United States,” NEON asks “how do we measure the same ecosystem at the plot, airshed, and landscape scales simultaneously, at the same time, with instruments whose relationship to each other is understood.” The NEON design describes a deliberate scale ladder: fixed-wing aircraft censusing vegetation at landscape scales with high-resolution remote sensing at annual time steps, tower-based sensors capturing continuous fluxes over roughly half a square kilometre, and plot-based observations at the scale of a site — since *neither a census nor temporally continuous measurement is appropriate for understanding organisms and biogeochemistry at site scale.*

[Case study] This is why NEON is the right *evaluation* corpus and the wrong *training* corpus. Colocation is exactly what an evaluation needs: labels and lidar measuring the same ground at the same time with documented geometry. Forty-seven terrestrial sites is exactly what a training corpus cannot be.

7.2 Flight box and season

A *minimum flight box of 10 km by 10 km is flown over each terrestrial site, a minimum of 100 km² surveyed*, initially centred on the flux tower and then repositioned or expanded to encompass the tower airsheds, the distributed observational sampling plots, representative vegetation types, and relevant watershed boundaries. *A minimum of 2 km is maintained between the tower and the edge of the survey area*, so that land cover contributing to off-airshed airflow is characterized.

The *flight season runs roughly March to October, with collection targeted at peak greenness and synchronized with ground campaigns.*

Three contrasts with 3DEP worth naming explicitly:

1. **Leaf-on by design.** NEON flies at peak greenness. 3DEP prefers leaf-off in deciduous regions. This is the largest single sensing-regime difference between the two corpora, and it is a *confound in any 3DEP-train / NEON-evaluate design* unless handled. It is also partly fixable: 3DEP contains leaf-on collections, so you can restrict or stratify.
2. **Temporally coordinated with ground data.** 3DEP acquisition dates are uncorrelated with any field campaign.
3. **Repeat annual acquisition.** Each site is revisited, giving multi-year time series over identical footprints — which is a change-detection resource and, more subtly, a source of genuine multi-view pairs for self-supervision (§4.5, Strategy 3), with the important caveat that the “views” differ in vegetation state as well as in acquisition geometry.

7.3 Payload and lidar specifics

The AOP payload combines imaging spectroscopy, high-resolution RGB, and discrete-plus-waveform lidar on one platform, which is the source of NEON’s value for multimodal work.

Lidar acquisition parameters, from NEON’s [data collection documentation](#):

- Flown at approximately **1000 m AGL**
- Adjacent flight lines designed for **30% overlap**
- Full scan angle **at least 36°**
- Minimum **4 shots per m²** in non-overlap areas
- Up to **5 points discretized per shot**
- Flight lines typically **600 m wide by 5–20 km long**
- Waveform recorded at a **1 ns sampling interval**, with a full waveform potentially holding hundreds of samples where the discrete product identifies one to eight targets
- **Waveform retained** as a separate product alongside discrete returns

Compare to a QL2 3DEP collection: NEON’s 4 pulses/m² is roughly double the QL2 minimum and half the QL1 minimum, and NEON’s designed 30% overlap raises aggregate density further. So NEON sits between QL2 and QL1 in pulse density, with the important difference that NEON’s figure is a *minimum in non-overlap areas*, i.e. defined similarly to the USGS single-swath convention (§5.4).

The sensor history, which is the §4.2 worked example, bears repeating because it is the thing most likely to be overlooked in a NEON-based evaluation: **Optech Gemini (2013–2020), Riegl Q780 (from 2018), Optech Galaxy Prime (from 2021), Riegl VQ-780 II-S (beginning 2025)**. Oscillating-mirror and rotating-polygon mechanisms, different footprints, different return-count limits, different intensity conventions. A NEON site-year is a confounded variable and you must recover the payload from metadata to interpret cross-site or cross-year results.

7.4 The calibration and QA regime

This is where NEON’s protocol-driven model shows most clearly, and it is worth knowing because it is the standard against which 3DEP’s project-by-project variation should be judged. Per NEON’s discrete-return ATBD and collection documentation: sensors are **calibrated each winter between flight seasons; horizontal, vertical, and timing calibration-validation flights** are conducted at the start and end of each flight season; and **nominal runway flights are conducted per domain** to check for accuracy drift during the season.

The per-site **QA reports in the Metadata folder** of each data download carry the sensor used, acquisition parameters, processing parameters, and QA information. These are your provenance source and they are far easier to parse than 3DEP’s per-project PDFs.

7.5 The product ladder

NEON’s data-level convention (DP0 raw, DP1 quality-controlled at native resolution, DP2 derived, DP3 mosaicked) maps onto lidar as:

- **DP1.30003.001** — Discrete return lidar point cloud, delivered by flight line and also as tiles.

- Waveform lidar as a separate DP1 product.
- **DP3** — mosaicked, tiled raster products: DTM, DSM, canopy height model, slope/aspect, at 1 m.
- Ecosystem structure products derived from these.

Two cautions. First, **DP3 CHM is a processed product with its own algorithmic choices**, and using it as ground truth for a canopy-height task means evaluating your model against another model. That is legitimate as a consistency check and illegitimate as an accuracy claim. Second, the **flight-line versus tiled** distinction matters: flight-line files preserve the acquisition geometry (single look direction, coherent scan-angle progression, one `PointSourceId`), while tiles mosaic across lines and thus mix look directions. For studying acquisition-geometry effects, use flight lines; for area-based analysis, tiles.

7.6 The ground: TOS plot design

Since NEON’s value to you is its ground truth, the plot design deserves detail. From NEON’s spatial sampling design documentation:

Distributed base plots are 40 m by 40 m, with a 20 m by 20 m core sampling area surrounded by a 10 m “high-impact” band. The core supports colocated plant diversity, herbaceous clip-harvest, leaf area index, vegetation structure, and canopy foliar chemistry; the high-impact band carries soil biogeochemistry and microbial sampling. **Ground beetle traps are installed around the plot perimeter.** Nested subplots at 10 m and finer scales support the finer-grained protocols.

Tower plots at sites dominated by tall-stature vegetation are also **40 m by 40 m, composed of four 20 m by 20 m subplots**, of which **two are randomly selected for sampling**.

Crucially for remote sensing: distributed base plots are established in a **spatially representative manner across all NLCD vegetation types occupying at least 5% of the site’s area**, and NEON’s own documentation states that data from these plots are [an important resource for calibrating and validating remote sensing datasets collected by the AOP](#), with explicit design links between TOS protocols and AOP products. Plot-scale colocation across protocols is identifiable directly via the `plotID` variable.

[Case study] The support geometry is a gift. A 40 m by 40 m plot with a 20 m by 20 m core is close to the classical area-based-approach plot size, close to PureForest’s 50 m patches, and a natural fit for a multi-scale embedding scheme (30 m / 100 m / 300 m concentric pooling, per the Part-I-to-evaluation-protocol discussion). You can align your model’s spatial support to the label’s spatial support almost exactly, which is unusual and worth exploiting: it removes one of the largest sources of ambiguity in benchmark design.

Note also the *subplot* structure. Because vegetation structure is measured in a 20 m by 20 m subplot while tree crowns were often annotated over the full 40 m by 40 m plot in derived datasets, field data may not cover every tree in a plot footprint. NeonTreeEvaluation’s documentation makes this explicit: each evaluation plot overlaps a NEON 40 m by 40 m plot, but field crews survey a 20 m by 20 m subplot, so field data exist for most plots without covering every annotated tree. If you join annotations to field measurements, check the support carefully.

7.7 Known biases

NEON documents its own design constraints, which is more than most data sources do, and several of them matter for the biodiversity tasks in your evaluation set.

Road proximity, by protocol. NEON’s spatial sampling design states that **mosquito sampling occurs within 45 m of roads, and small mammal sampling occurs within 300 m of roads**, because of visit frequency requirements. This is the same accessibility bias I flagged for eBird, present in a supposedly rigorous protocol, and it is specific by taxon. Any small-mammal occupancy task built on NEON data inherits a 300 m road-proximity constraint, and road proximity is strongly visible in ALS structure. **Include a road-distance-only baseline in the ablation ladder for these tasks.** It is cheap and it will tell you whether your model has learned habitat or learned access.

Plot placement constraints. Plots must be separated by 150% of one edge (so 40 m plots are at least 60 m apart); plot centres must be more than 50 m from large paved roads and 50 m from buildings and non-NEON infrastructure; plot edges must be 10 m from two-track dirt roads. These rules exclude edge and near-infrastructure habitat systematically, which means NEON’s plot network under-samples exactly the disturbed, fragmented, edge-dominated conditions that dominate much of the actual US landscape.

Site selection. NEON sites were chosen to be representative of domains and to be operationally feasible over decades, which biases toward relatively intact, accessible, securely tenured land. The network is not a random sample of American ecosystems; it is a stratified sample of *studyable* American ecosystems.

Plot retirement. Sampling locations that become unsamplable due to disturbance are retired and given new IDs. This is honest data management and a subtle selection effect: the surviving long-term series is biased toward locations that did not experience severe disturbance.

[Case study] None of these disqualify NEON as an evaluation corpus. They do mean that “the model achieves X on NEON” is a statement about performance on intact, accessible, non-edge habitat, and that generalization to the fragmented working landscape is *not* tested by NEON. This is a substantive argument for the two-tier evaluation structure — a rigorous, biased, small-n tier plus a noisy, broadly-sampled, large-n tier — rather than for treating NEON as the gold standard tout court.

7.8 What NEON is and is not good for

Good for: paired label-and-lidar evaluation with known geometry and timing; multimodal work (lidar plus hyperspectral plus RGB on the same platform, same day); quantifying what discretization costs, since waveform is retained; cross-taxon evaluation via `plotID` joins; repeat-collection change detection; and measuring the sensor domain gap when combined with 3DEP coverage of the same footprints.

Not good for: national generalization claims; training a general-purpose representation; representing disturbed or fragmented landscapes; or serving as a single gold standard.

The specific opportunity I’d flag. NEON retains waveform *and* discrete returns *and* has co-located field measurements at documented plot support. That combination makes it

possible to answer a question nobody has answered systematically: **how much ecological information does discretization destroy?** Run your benchmark tasks on discrete returns and on waveform-derived profiles over the same plots, with the same model family, and report the difference. If the gap is large, it is an argument for waveform retention in future national programs — a policy-relevant result that falls out of infrastructure you are building anyway.

Further reading and visual resources for Chapter 7

- [NEON: Lidar data collection, Flight box design, Flight schedules and coverage](#) — the primary sources for §7.2–7.3, with flight-box diagrams.
- [NEON.DOC.001292, L0-to-L1 Discrete Return Lidar ATBD](#) — the discretization algorithm and calibration regime.
- [NEON.DOC.000913, TOS Science Design for Spatial Sampling](#) and [NEON.DOC.005108, Spatial and Temporal Sampling Strategy](#) — plot design with figures. The plot-layout diagrams in the latter are the visual reference for §7.6 and are worth printing.
- Meier et al. (2023), *Spatial and temporal sampling strategy connecting NEON Terrestrial Observation System protocols*, Ecosphere — the authoritative account of colocation across protocols, including the table of TOS products with explicit design links to AOP products. Read this before designing any NEON-based evaluation task.
- [NEON discrete point cloud tutorial \(Python\)](#) for hands-on access.

Chapter 8. Other Programs, and Other Platforms

8.1 Why you need them

Three reasons a 3DEP-plus-NEON project should still care about the rest of the world.

External validity. A model trained on 3DEP and evaluated only on US data has been tested for geographic generalization within one acquisition program. Evaluating on a European national collection tests generalization across programs, sensors, densities, and specifications simultaneously. It is the hardest available test and the most informative one.

Existing benchmarks live there. PureForest, FRACTAL, and several tree-species and semantic-segmentation benchmarks are built on European national ALS. If you want to compare your representation against published numbers rather than only against your own baselines, you need to run on their data.

Density headroom. Several European programs fly at densities several times 3DEP’s, which lets you ask a question 3DEP alone cannot: does a representation pretrained at 2–8 pulses/m² transfer *upward*? The augmentation logic of §4.5 says degradation is simulable and enrichment is not, which predicts that upward transfer is the harder direction. That prediction is testable and, to my knowledge, untested.

8.2 IGN Lidar HD (France)

The most ML-ready national program, and the one whose practices are most worth copying.

France’s Institut national de l’information géographique et forestière is executing a national ALS coverage at approximately **10 pulses/m²**, **roughly 40 points/m²** — several times

3DEP’s QL2 density and comparable to QL1’s pulse density with far more returns per pulse. Source data spans multiple sensor families (Leica, Riegl, Teledyne/Optech), so it is heterogeneous in the §4.1 sense despite the uniform target density.

What makes IGN worth studying is not the lidar; it is the data engineering built on top of it:

- **PureForest** — 135,569 patches of 50 m by 50 m, 13 semantic classes hierarchically grouping 18 tree species, with a reference train/validation/test split, monospecific patches, and paired aerial imagery. Note the honest caveat in its documentation that lidar and imagery may be separated by up to three years.
- **FRACTAL** — 100,000 dense point clouds, 7 semantic classes, 250 km², sampled from five French regions with rare classes and difficult landscapes explicitly oversampled, released with a dataset card, a trained baseline model, and the sampler.
- **pacasam** — the sampler used to build FRACTAL. This is the single most directly reusable piece of code for your Chapter 24 corpus construction.
- **Myria3D** — the training and inference library, whose receptive-field-based interpolation back to the full point cloud and overlapping-subtile inference are the stitching solutions you would otherwise have to rediscover.

[Case study] Even if you never train on French data, read PureForest’s dataset card and FRACTAL’s paper for release engineering, and read pacasam before writing your own sampler. The pattern — separate repos for sampling, training, and dataset, plus released weights and a versioned split — is what a benchmark release should look like.

8.3 A survey of other national programs

The table below is orientation, not specification. National programs revise their targets, and several are mid-campaign. **Verify current density and licensing against the national portal before building anything on these numbers.**

Program / country	Character	Notes for transfer evaluation
IGN Lidar HD (France)	High density, national, ML-ready	Best-documented; existing benchmarks; §8.2
AHN (Netherlands)	Long-running, multiple generations at increasing density	Multi-epoch coverage of the same ground; good for change and for multi-view pairs
swissSURFACE3D (Switzerland)	National, classified, high relief	Excellent stress test for terrain-driven failure modes
Denmark (DHM)	National, repeat coverage	Open access, well documented
Norway (Hoydedata)	National, project-based, variable density	Boreal forest; density varies widely by project
Sweden	National, forestry-oriented derived products	Notable for the derived forest-attribute products layered on the lidar
Finland (NLS)	National, and the source of multispectral ALS species work	Multispectral ALS is a distinct sensing regime worth knowing about
Spain (PNOA-LiDAR)	National, multiple coverage generations	Mediterranean systems poorly represented elsewhere

Program / country	Character	Notes for transfer evaluation
England / Wales / Scotland	National programmes at varying density	Heavily modified landscapes; good complement to NEON's intact-habitat bias
Canada (HRDEM)	Regional, expanding	Boreal, and a large-area complement to CONUS
New Zealand (LINZ)	National, open	Southern-hemisphere, distinctive vegetation

The pattern worth noticing: **most European national programs are higher density than 3DEP and most are leaf-off**. So European transfer tests density generalization and often shares 3DEP's phenological regime, while NEON transfer tests phenology and sensor while roughly matching density. They probe different axes; use both.

8.4 Other platforms, and the temptation to mix them

Your corpus is airborne. Several adjacent data types will be offered to you. A brief taxonomy of how far each is from ALS, because the temptation to pool is strong and usually wrong.

UAV / drone laser scanning (ULS). Tens to thousands of points per m², footprints of centimetres, flying heights of tens of metres, and frequently non-repetitive Risley scan patterns (§2.1). FOR-instance is the notable annotated benchmark. Distance from ALS: large. The point *statistics* are unlike any airborne collection, occlusion structure is different, and — critically — ULS resolves individual branches and stems that ALS cannot. Treat ULS as a **transfer target**, never as in-domain training data, and expect naive density normalization to fail because the difference is not merely density.

Terrestrial and mobile laser scanning (TLS/MLS). Bottom-up viewing geometry, so the occlusion pattern is *inverted* relative to ALS: TLS sees stems and understory well and canopy tops poorly. Extremely useful as reference truth for stem-level attributes, and nearly useless as training data for an ALS model. TLS-derived labels transferred to ALS point clouds is a legitimate and productive pattern (this is roughly what SegmentAnyTree and related tree-instance work does); TLS point clouds as ALS training data is not.

Spaceborne lidar. GEDI (full-waveform, ~25 m footprints, sampling rather than mapping, roughly 51.6° latitude limits) and ICESat-2 ATL08 (photon-counting, along-track segments). Neither is a point cloud in the ALS sense; both are footprint- or segment-level structural estimates. Their value to you is **national-to-global weak labels and cross-references** where no plot network reaches, and as a possible auxiliary pretraining target. Do not attempt to treat them as low-density ALS.

Photogrammetric point clouds (DAP / SfM). Derived from imagery rather than ranging. They have no return structure, no penetration below the visible surface, and therefore no understory and no reliable ground under closed canopy. They look superficially like a point cloud and behave nothing like one. QL4 in the USGS scheme is photogrammetric for a reason: 139 cm vertical accuracy. Exclude.

A useful framing. Rank these by *viewing geometry* rather than by density, because viewing geometry determines what is systematically missing: ALS (top-down, airborne range), ULS (top-down, near range), spaceborne (top-down, coarse footprint, sampled), TLS (bottom-up), MLS (side-on), DAP (top-down, no ranging). Two datasets with the same viewing geometry and different density are much more compatible than two with the same density and different geometry. Density you can simulate; geometry you cannot.

8.5 A transfer taxonomy for your benchmark

Pulling Part II together into the evaluation axes it implies. Each row is a distinct generalization question, and each should be a reported result rather than an assumption.

Axis	Held constant	Varied	Test data
Geographic	Program, sensor family, density regime	Ecoregion, species pool	Held-out 3DEP ecoregions
Project	Ecoregion	Contractor, sensor, processing	Leave-one-3DEP-project-out
Quality level	Ecoregion, era	Density, accuracy class	Train QL2, test QL1 and vice versa
Sensor (paired)	Footprint, labels, date-ish	Payload, mission, processing	NEON AOP vs. 3DEP over identical NEON footprints
Phenology	Region, sensor	Leaf-on vs. leaf-off	Matched leaf-on/leaf-off 3DEP pairs where available
Program	Broad biome analogue	Specification, sensor, density, language of metadata	PureForest / FRACTAL / other national data
Density direction	Program	Upward vs. downward density transfer	Train low, test high; train high, test low
Platform	Species pool	Viewing geometry and scale	FOR-instance (ULS) as an explicit out-of-domain probe

Eight axes is a lot to report, and reporting all eight is the point. A single “generalization” number for an ALS foundation model is not interpretable, because the axes have different magnitudes and different practical importance to different users. A forester in Oregon cares about the quality-level and sensor rows; someone applying your model in France cares about the program and density-direction rows; an ecologist modelling bird occupancy cares most about the geographic and phenology rows. Give them the table.

Further reading for Chapter 8

- [PureForest](#) and its [dataset card](#); [FRACTAL](#) and its [dataset card](#); [pacasam](#); [Myria3D](#).
- [FOR-instance](#) for the ULS-density annotated-tree benchmark, and [SegmentAnyTree](#) for the platform-agnostic modelling approach.
- [OpenForest](#) as a catalogue of forest datasets across platforms and scales — the fastest way to find out whether the dataset you need already exists.

- For national portals, go to the source rather than to secondary summaries; the density and licence terms in this chapter’s table change.

Bridge to Part III

Parts I and II covered the instrument and the institution. Part III covers the transformation between them and the file: georeferencing from trajectory and boresight, strip adjustment, ground classification, and then — in the chapter I consider the most important in the first half of this book — an audit of what has already been done to the LAZ you downloaded, which operations are irreversible, and what each of them costs you.

Chapter 9 develops the georeferencing chain and its error budget. Chapter 10 covers post-flight adjustment and classification. Chapter 11 is the audit, and it is where the provenance schema of §4.6 and §6.8 becomes a concrete specification rather than a recommendation.

Part III — From Photons to the File You Downloaded

Airborne Laser Scanning for Machine Learning in Ecology

Orientation to Part III

A discrete return, at the moment of detection, is three numbers in the scanner’s own frame: a range, a mirror angle, and a timestamp. Everything else — the fact that it has a latitude, a longitude, an orthometric height, a classification, and a place in a tile — is added afterward by a processing chain that runs for weeks and is documented in prose reports nobody reads.

Part III is about that chain. Three chapters:

- **Chapter 9** develops direct georeferencing: the transformation from range-and-angle to coordinates, the GNSS/inertial machinery that makes it possible, the error budget and why horizontal error dominates vertical, and the datum situation — including a transition currently in progress that will change every coordinate in the United States.
- **Chapter 10** covers post-flight adjustment and classification: strip adjustment, the ground-classification algorithm zoo and its failure modes, hydro-flattening, noise filtering, and the question of whether to trust any of it.
- **Chapter 11** is the audit. It is the chapter I consider most important in the first half of this book. It asks, operation by operation, what has already been done to your LAZ file, which of those operations are reversible, which are lossy but detectable, and which are silently inconsistent across the archive — and then turns that into a concrete diagnostic protocol and a provenance schema.

The through-line: **you are not training on measurements. You are training on the output of a pipeline.** Knowing which parts of that pipeline are consistent across your corpus, and which are not, is the difference between a benchmark that measures ecology and one that measures processing software.

Chapter 9. Georeferencing and Trajectory

9.1 The direct georeferencing equation

Airborne lidar uses *direct* georeferencing: the sensor's position and orientation are measured continuously, so each observation can be transformed to mapping coordinates without requiring ground features. The transformation, in the standard form:

$$\mathbf{p}^m = \mathbf{p}_{\text{GNSS}}^m + \mathbf{R}_b^m(t) [\mathbf{R}_s^b \mathbf{r}^s(t) + \mathbf{a}^b]$$

with:

- $\mathbf{p}^m$ — the point in mapping frame
- $\mathbf{p}_{\text{GNSS}}^m$ — the GNSS antenna phase-centre position at time t , from the processed trajectory
- $\mathbf{R}_b^m(t)$ — the rotation from body (IMU) frame to mapping frame, i.e. the aircraft attitude: roll, pitch, heading, time-varying
- $\mathbf{R}_s^b$ — the **boresight** rotation from scanner frame to body frame: three small, constant, *unknown* angles, determined by calibration
- $\mathbf{r}^s(t)$ — the range vector in the scanner frame, constructed from the measured range R and the mirror angle θ : for a single-axis scanner, $\mathbf{r}^s = R [0, \sin \theta, -\cos \theta]^\top$
- $\mathbf{a}^b$ — the **lever arm**: the constant offset vector from the IMU origin to the scanner origin, expressed in body frame, measured by terrestrial survey of the installed hardware

Read the structure of that expression rather than its details. There are three classes of quantity:

1. **Measured per pulse at high rate** — R, θ, t . These are the lidar observations proper. Their errors are small (centimetres, microradians).
2. **Measured continuously and interpolated** — position and attitude. These come from a separate instrument and carry their own, larger, temporally correlated errors.
3. **Constant and estimated** — boresight and lever arm. Errors here are *systematic*: they do not average out over a project, they produce coherent geometric distortion, and they are the target of calibration.

The dominance of category 3 systematics over category 1 random error is why lidar accuracy is a *calibration* problem rather than a *precision* problem, and why strip adjustment (§10.1) exists.

9.2 GNSS positioning

The trajectory's absolute position comes from GNSS, processed in one of several modes:

Differential / post-processed kinematic (PPK). Carrier-phase processing against one or more reference stations with known coordinates, usually from a CORS network or a project-specific base. Accuracy degrades with baseline length — roughly, error grows with distance from the nearest reference station as ionospheric and tropospheric decorrelation increase. Centimetre-level at short baselines; decimetre-level at 100+ km. **This is a spatially structured error source:** projects in areas with dense CORS coverage are systematically better positioned than projects in remote areas. In CONUS this correlates with population density, which correlates with land cover.

Precise point positioning (PPP). Single-receiver processing using precise satellite orbit and clock products, no local base needed. Increasingly competitive and attractive for remote areas where no base station exists. Convergence time and residual biases differ from PPK, so a PPP-processed project has a different error signature than a PPK one.

Network RTK / real-time. Rarely the final answer for mapping-grade lidar; used for navigation during flight, with post-processing for the delivered trajectory.

The processed output is conventionally a **Smoothed Best Estimate of Trajectory (SBET)** — a time series of position, velocity, and attitude at high rate (typically 200 Hz), produced by a tightly- or loosely-coupled GNSS/INS Kalman filter run forward and backward and smoothed. Applanix POSPac is the dominant commercial implementation; the SBET file format is a de facto standard.

The SBET is not delivered to you. It is an intermediate product held by the contractor. This matters more than it might seem: without the trajectory you cannot reconstruct scan geometry rigorously, cannot re-derive the incidence angle onto a sloped surface, and cannot recompute anything that depends on sensor position. The `ScanAngleRank` field and `PointSourceId` are your only proxies, which is another reason to preserve them (§2.3).

9.3 Inertial attitude

The IMU provides attitude between GNSS updates and at the high rate the scanner needs. Its quality is the single largest driver of horizontal accuracy.

Grades. Navigation-grade units use ring-laser or fibre-optic gyros with drift rates on the order of 0.001–0.01 degrees per hour; tactical-grade units are an order of magnitude worse. Airborne mapping systems use navigation-grade. Typical delivered attitude accuracy after GNSS/INS integration: roughly 0.003–0.005 degrees in roll and pitch, and 0.005–0.010 degrees in heading.

Heading is always the worst axis, by roughly a factor of two, because heading is weakly observable from GNSS-derived velocity when the aircraft flies straight and level. It becomes observable during turns and accelerations, which is why flight plans include manoeuvres and why the trajectory quality is best in the middle of a well-flown block.

Why the asymmetry matters geometrically. Roll and pitch errors project to horizontal error proportional to slant range. Heading error projects to horizontal error proportional to the *lateral offset from nadir* — which is zero at nadir and maximal at swath edge. So heading error produces a **scan-angle-dependent horizontal distortion**: the swath is slightly sheared. In sidelap, where two swaths meet at their edges, the shear appears as a horizontal misregistration between strips. This is one of the signatures strip adjustment removes.

9.4 Lever arm and boresight

Lever arm is a geometric measurement: the vector from the IMU reference point to the scanner reference point, surveyed on the installed system to centimetre accuracy. It changes whenever the payload is removed and reinstalled, which happens between seasons and between aircraft.

Boresight is the angular misalignment between the IMU axes and the scanner axes. It cannot

be measured mechanically to the required precision (microradians), so it is *estimated* from calibration flights. The classical geometry:

- **Opposing-direction line pairs over flat, hard ground** separate roll boresight (a roll error produces opposite lateral shifts in opposite-heading passes) and, with altitude variation, help with scale-like errors.
- **Cross-strips** — lines flown perpendicular to the main coverage — provide the strongest constraint on pitch and heading boresight.
- **Lines at two altitudes** separate angular from offset errors, since angular errors scale with range and offsets do not.
- **Gable roofs and other sloped planar surfaces** provide multi-directional normal vectors that constrain all three angles simultaneously; this is why urban calibration sites are prized.

Boresight is solved per installation, and residual boresight error is one of the systematic terms strip adjustment absorbs.

9.5 The error budget

A worked budget for the §5.2 mission: $h = 1200$ m, $\theta_{\max} = 20^\circ$, navigation-grade IMU, short-baseline PPK.

Source	Magnitude	Horizontal contribution	Vertical contribution
GNSS position	2–3 cm horiz., 4–6 cm vert.	3 cm	5 cm
Roll/pitch attitude	0.005°	10.5 cm (at nadir, $= h \cdot \Delta\theta$)	3.8 cm at 20° ($h \Delta\theta \tan \theta$); 0 at nadir
Heading attitude	0.008°	6.1 cm at 20° ($= h \tan \theta \cdot \Delta\psi$); 0 at nadir	negligible
Range measurement	1–2 cm	negligible at nadir	1.5 cm
Lever arm	1–2 cm	1.5 cm	1.5 cm
Residual boresight	0.002°	4 cm	1.5 cm
Angular encoder	0.001°	2 cm	negligible
Combined (RSS)		~13 cm	~7 cm

Three readings to take from this table.

First, horizontal error is roughly twice vertical error, and the reason is structural: horizontal error is dominated by attitude multiplied by range, while vertical error is dominated by GNSS height and range measurement. There is no attitude term in vertical error at nadir. This asymmetry is why lidar specifications are written in terms of vertical accuracy — it is the good axis — and why horizontal accuracy is reported loosely. For ML: **your horizontal registration between a field plot and a point cloud is worse than your vertical registration**, typically by a factor of two, and this compounds with plot GNSS error (§18).

Second, error grows with slant range, hence with flying height and with scan angle. A collection flown at 2400 m has roughly double the attitude-driven horizontal error of one flown

at 1200 m. Swath-edge points are worse than nadir points in position as well as in the ways established in Part I.

Third, this is the *relative* budget, before absolute calibration. After vertical adjustment to ground control, systematic vertical bias is largely removed and the reported NVA (§6.5) reflects the residual. So the delivered vertical accuracy can be better than the raw budget suggests, while the horizontal accuracy is essentially as computed, because horizontal control is rarely applied.

9.6 Time

Lidar timing conventions cause a specific, recurring class of bug.

GPS time is a continuous atomic timescale with no leap seconds, counting from the GPS epoch (1980-01-06). It runs ahead of UTC by an integer number of leap seconds (18 as of the last insertion, and no further insertions have occurred as of this writing — the possibility of a *negative* leap second is now under discussion, which would be a first).

GPS week seconds count from midnight Sunday of the current GPS week, resetting at 604800. Older LAS files sometimes carry week seconds, which is a disaster for any analysis crossing a week boundary and is why the practice was deprecated.

Adjusted Standard GPS Time — standard GPS seconds-of-week-continuous minus 10^9 — is what LAS 1.4 requires and what 3DEP mandates. The subtraction exists purely to keep the value inside a double-precision float without losing sub-microsecond resolution. The global encoding bit in the LAS header declares which convention a file uses. **Check it; do not assume.**

Two practical uses of the timestamp worth flagging:

1. **Pulse grouping.** 3DEP requires timestamps precise enough to be unique per pulse (§6.6). All returns from one pulse share a timestamp, so grouping by `GpsTime` recovers pulse structure — which is what makes pulse-level thinning (§4.5) and per-pulse vertical profile analysis possible.
2. **Pass structure.** The distribution of timestamps within a tile reveals how many passes contributed and how far apart in time they were. Large gaps mean sidelap from different sorties, possibly different days, with correspondingly larger relative registration error and real vegetation change.

9.7 Datums, geoids, and a transition in progress

The current situation

US lidar is conventionally delivered in **NAD83(2011)** horizontally and **NAVD88** vertically, with orthometric heights derived from ellipsoid heights via a geoid model (GEOID12B, GEOID18, depending on era). Projected coordinates are usually UTM or a State Plane zone.

Two facts about these datums matter for a continental-scale ML corpus, and they are not widely appreciated outside geodesy:

- **NAD83 is not geocentric.** NGS estimates it is misaligned with the Earth’s centre of mass by about 2.2 metres.
- **NAVD88 is biased and tilted.** NGS’s own characterization is that NAVD88 is biased by roughly 0.5 m and tilted by roughly 1.0 m coast-to-coast relative to current global geoid models.

Sit with the second one. **A metre of coast-to-coast tilt means that absolute orthometric height in your corpus carries a continental-scale systematic error field.** For most ecological purposes this is irrelevant, because ecology cares about height *above ground*, which is a difference of two heights in the same datum and is therefore immune. But it is highly relevant if you:

- feed absolute Z or elevation to the model as a feature (a common and tempting choice, since elevation is genuinely ecologically informative);
- use elevation as a stratification variable;
- compare DTMs across projects, especially across regions;
- or train a model that could learn “elevation offset” as a proxy for longitude.

[Case study] This is a real argument in the height-normalization debate of Chapter 22. Height above ground is not merely more transferable across terrain — it is also datum-immune, whereas absolute Z inherits a systematic continental field. If you want the model to have access to elevation as an ecological covariate, provide it explicitly as a separate, deliberately coarsened channel (say, DEM elevation smoothed to 1 km) rather than as raw per-point Z . That decouples “elevation as ecology” from “elevation as datum artifact plus longitude proxy.”

The transition

NGS is replacing the entire system. All three NAD83 frames and all vertical datums including NAVD88 are being replaced by four plate-fixed terrestrial reference frames — **NATRF2022** (North America), **PATRF2022** (Pacific), **CATRF2022** (Caribbean), **MATRF2022** (Mariana) — each mathematically defined relative to ITRF2020 and rotating at the average rate of its tectonic plate, plus a single geopotential datum, **NAPGD2022**, whose zero-orthometric-height surface is defined by **GEOID2022** (built on the GRAV-D airborne gravity campaign, with a static component representing the geoid at epoch 2020.00).

Status, as best I can determine: NGS is conducting a phased beta rollout, the current NSRS remains official in the meantime, and the Federal Geodetic Control Subcommittee vote to approve the modernized NSRS was expected in 2026, with a several-month transition afterward. Recent industry guidance suggests beta product phases through 2026 with official release possibly slipping to 2027. **This is exactly the kind of fact that changes; check geodesy.noaa.gov/datums/newdatums rather than trusting this paragraph.**

Because the transition corrects NAD83’s origin offset and NAVD88’s tilt, every coordinate in the official NSRS will change, in some areas by several decimetres or more.

[Case study] Three design implications, and I’d act on all three now:

1. **Record the CRS and vertical datum per sample, verbatim from the file header**

WKT, not as an inferred or normalized value. When the transition lands, you need to know exactly what each tile was in.

2. **Do not build absolute-elevation dependence into your benchmark’s task definitions.** A task defined on orthometric height will change value under NAPGD2022; a task defined on height above ground will not.
3. **Expect a mixed corpus indefinitely.** New collections will arrive in the modernized system while the archive stays in NAD83/NAVD88, and reprocessing 20 years of 3DEP is not going to happen quickly. Your pipeline should handle both, and your provenance schema should make the distinction queryable.

9.8 What survives into the file

Closing Chapter 9 with an inventory, because the gap between what was measured and what you receive is the theme of Part III.

Quantity	In the LAZ?	Notes
Point coordinates	Yes	Quantized by scale/offset (§12.1)
Range	No	Recoverable only approximately, if at all
Mirror/scan angle	Partially	ScanAngleRank (1° integer, formats 0–5) or finer (formats 6–10); not the true instrument angle
Scan direction flag	Yes	Useful for identifying bidirectional scanners
Edge-of-flightline flag	Yes	Rarely populated correctly; check before trusting
Timestamp	Yes	Convention declared in header; enables pulse grouping
Flight line identity	Yes, as PointSourceId	The most under-used field in ALS ML
Sensor position at pulse time	No	The trajectory is not delivered
Attitude	No	—
Boresight, lever arm	No	—
Waveform	No, for 3DEP	Removed from the specification entirely (§6.3)
Intensity	Yes, semantics unknown	§3.7
Return number / count	Yes	Capped by sensor and by point format

The two rows worth acting on: **PointSourceId** and **GpsTime** together let you reconstruct a great deal of the acquisition geometry that is nominally absent — pass count, pass timing, seam locations, per-line scan-angle progression. Almost no ALS-ML pipeline uses either. I would treat them as first-class inputs to the provenance computation in §11.6.

Further reading for Chapter 9

- Skaloud & Lichti (2006) and the broader boresight self-calibration literature, for §9.4. Search: *airborne lidar boresight self-calibration planar features*.

- Glennie (2007), *Rigorous 3D error analysis of kinematic scanning LIDAR systems*, Journal of Applied Geodesy — the definitive error-budget treatment; §9.5 is a simplification of what this paper does properly.
- [NGS New Datums](#) and the [New Datums FAQ](#) — authoritative and updated; the FAQ answers the questions you will actually have.
- *Brave New Coordinates*, xyHt — a readable practitioner’s summary of the transition, with the GEOID2022 undulation and uncertainty maps that are the best available visuals for §9.7.
- [ASPRS LAS 1.4-R15 specification](#) for the time conventions and global encoding bits.

Chapter 10. Post-Flight Adjustment and Classification

10.1 Strip and block adjustment

After direct georeferencing, adjacent strips disagree. The disagreement is dominated by the systematic terms of §9.1 category 3 — residual boresight, lever-arm error, range bias, scan-angle scale — plus slowly-varying trajectory error. Strip adjustment estimates corrections that minimize the disagreement.

The observation model. Correspondences are established not between points (there are no identical points in two strips) but between *surfaces*. Common formulations:

- **Planar patch matching.** Segment planar regions (roofs, roads, flat ground) in each strip; require that a point in strip A lie on the corresponding plane in strip B. This is the rigorous approach and it works well where planes exist.
- **ICP-style surface matching.** Minimize point-to-plane or point-to-surface distance between overlapping strips iteratively. More general, less interpretable, and prone to sliding along degenerate directions.
- **Tie-line and tie-point methods.** Manual or semi-automatic identification of correspondences; older practice.

The parameter model ranges from a simple 3D shift per strip (cheap, common, and geometrically wrong, since it cannot absorb angular error) to a rigorous per-strip correction of boresight angles plus time-dependent trajectory corrections. Which model was used determines what residual distortion remains. A shift-only adjustment leaves scan-angle-dependent residuals in place — meaning **the vertical agreement between strips will be good at nadir and worse at swath edge**, which is precisely the geometry that will confuse you when you find canopy-height disagreement patterns that follow flight lines.

What the specification requires as evidence. 3DEP requires interswath relative accuracy to be demonstrated and requires **swath separation images** as a deliverable — rasters of the elevation difference between overlapping swaths, colour-ramped so that residual misalignment is visible. Where these are archived, they are a direct, spatially explicit measurement of the relative registration quality of a project, and I know of no ML work that has used them. They would make an excellent per-tile quality covariate.

Also required: **smooth surface precision** (within-swath repeatability over flat hard surfaces), which is the diagnostic for range noise and high-frequency trajectory error.

10.2 Absolute vertical calibration

Once strips agree with each other, the block is shifted to agree with ground control. This is usually a single vertical offset per project, sometimes a tilted plane, occasionally a low-order surface.

Two consequences:

1. **The absolute vertical accuracy of a project is largely set by the quality and distribution of its control**, and the check-point regime of §5.6 assesses it on flat open ground.
2. **A single project-wide vertical shift cannot correct spatially varying error**. So residual vertical error within a project has spatial structure — typically correlated with distance from control and with GNSS baseline geometry. For a benchmark, this means **vertical error is spatially autocorrelated within a project**, which matters if you are evaluating a height-regression task with spatially blocked cross-validation: the errors are not independent within a block.

10.3 Ground classification: the algorithm zoo

Ground classification is the highest-leverage processing step for ecology (§4.4), because every height-above-ground feature depends on it. The specification does not prescribe an algorithm (§6.3). Here is what contractors actually use, in rough historical order.

Slope-based morphological filters. The founding idea: a point is non-ground if the slope from it to a nearby point exceeds a threshold that depends on the horizontal distance (Vosselman 2000). Cheap, simple, and biased on steep terrain, where real slope exceeds the threshold.

Progressive morphological filtering (Zhang et al. 2003) applies opening operations with a window that grows progressively, removing successively larger non-ground objects. Requires an elevation-difference threshold schedule tied to terrain slope.

Progressive TIN densification (Axelsson 2000). Start from a sparse set of confident low points; build a TIN; iteratively add points whose distance to the TIN facet is below a threshold; refine. This is the algorithm behind TerraScan and therefore behind a very large share of the world’s classified ALS, including much of 3DEP. Its behaviour is controlled by the initial seed selection, the iteration angle/distance thresholds, and the maximum building size parameter.

Hierarchical robust interpolation (Kraus & Pfeifer 1998). Fit a smooth surface with robust weighting, iteratively down-weighting points above the surface; run coarse-to-fine. This is SCOP++’s approach. Elegant, and it produces a surface rather than a point classification, which is a subtly different output.

Cloth simulation filtering (Zhang et al. 2016). Invert the cloud, drop a simulated cloth of specified rigidity onto it, and classify points close to the settled cloth as ground. Intuitive to tune (rigidity, cloth resolution, distance threshold) and available as `filters.csf` in PDAL.

Simple Morphological Filter (SMRF) (Pingel et al. 2013). A morphological approach with careful handling of the slope threshold and an explicit treatment of low outliers. Available as `filters.smrf` in PDAL, and the pragmatic default there.

Learned ground classification. Increasingly common in commercial software and in research. Circular for your purposes if the training labels came from one of the above.

PDAL-relevant summary: you have `filters.smrf`, `filters.csf`, and `filters.pmf` available, plus `filters.hag_nn` / `filters.hag_delaunay` / `filters.hag_dem` for computing height above ground once you have a ground set. See Chapter 14.

10.4 Ground classification failure modes

This section is the practical heart of the chapter. Each failure mode below produces a *specific, spatially structured* error in your height-above-ground features, and each is ecologically non-random.

Steep terrain. Slope-based and morphological methods have a threshold that trades off between removing low vegetation and removing real terrain. On steep ground the algorithm either keeps low vegetation (inflating the DTM, deflating canopy height) or removes convex terrain features (deflating the DTM, inflating canopy height). Both happen, in different places, in the same project. **Effect: canopy height error correlated with slope.** Since slope correlates with forest type, disturbance history, and species composition almost everywhere, this is a first-order confound.

Dense understory and closed canopy. Where few or no pulses reach the ground, the DTM is interpolated across gaps of tens of metres. The interpolated surface is smooth where the real terrain is not, and it is biased upward where the lowest returns are actually low vegetation rather than ground. **Effect: canopy-height bias that is worst in exactly the dense forests where biomass is highest**, i.e. the error is correlated with the target variable in biomass regression. This is the single most pernicious systematic in ALS forestry.

Low vegetation confusion. Grass, shrubs, crop residue, and coarse woody debris sit inside the algorithm's ground tolerance. Combined with the dead-time blind zone (§3.5), the near-ground stratum is doubly compromised: partly unmeasured, partly misclassified.

Buildings and large flat structures. Progressive TIN densification has a maximum-building-size parameter; structures larger than it are classified as ground. Large agricultural buildings, parking structures, and bridges are the usual casualties. Class 17 (bridge deck) exists to handle bridges explicitly.

Terraces, berms, road cuts, and breaklines. Sharp terrain discontinuities violate the smoothness assumptions of surface-fitting methods. Class 20 (Ignored Ground) exists precisely for vertical and overhanging bare-earth surfaces whose inclusion would create interpolation artifacts (§6.4) — which means **the delivered DTM deliberately omits some real ground**.

Water and wet ground. Absorption produces sparse or absent returns; class 9 and hydro-flattening (§10.5) handle it by fiat.

Snow and dense litter. The classified “ground” is the top of the snow or litter layer, not the soil surface.

[Case study] The practical conclusion: **re-derive ground classification yourself, with one algorithm and one parameter set, for the whole corpus, and ablate against vendor ground.** The argument is not that your SMRF will be better than a skilled contractor’s TerraScan tuning — it probably won’t be, per project. The argument is that it will be *consistent*, and consistency is what a benchmark needs. A model trained on vendor ground learns partly to invert the distribution of contractor parameter choices; a model trained on uniformly re-derived ground does not. Report both, because the difference between them is itself an interesting measurement: it tells you how much of your task performance is sensitive to terrain-model provenance.

There is a caveat to that recommendation. Vendor ground classification, whatever its inconsistency, was produced with human QC over the project, and re-derived ground will fail catastrophically in some places where the contractor caught the problem. Screen your re-derived DTM against the delivered DEM (§11.6) and flag large disagreements rather than silently accepting them.

10.5 Hydro-flattening

Water surfaces in raw lidar are noisy, sparse, and sometimes below the true surface (multipath through water). Terrain products require water to be flat and, for rivers, monotonically descending. The solution is **hydro-flattening**: breaklines are compiled around water bodies, water-classified points are set or the surface is forced flat within the breakline, and rivers are enforced to descend.

For ecology this is a fabrication with sharp edges. Consequences:

- Riparian zones — ecologically among the most important areas in any landscape — contain an artificial surface whose edges are compiled by a human operator to a minimum-size threshold (small ponds and narrow streams below the threshold are not flattened, so treatment is inconsistent by feature size).
- Class 9 (water) is only *verified* where associated with a breakline (§6.4), so water classification elsewhere is unchecked.
- Emergent and floating vegetation over water behaves unpredictably: sparse returns over an artificial flat surface.
- **Height above ground in wetlands is computed relative to a fabricated datum.**

Practical advice: carry a wetland/water-proximity flag in your provenance record and consider reporting wetland-heavy samples as a separate stratum. For tasks with an explicit hydrological or riparian component, treat hydro-flattened areas as a known-artifact zone.

10.6 Noise and outlier handling

Removing or flagging invalid returns is required (§6.4). Common methods:

- **Extended Local Minimum (ELM)** for low outliers — points far below their neighbourhood, typically from multipath or MPiA ambiguity. `filters.elm` in PDAL.

- **Statistical outlier removal** — drop points whose mean distance to k neighbours exceeds a multiple of the global standard deviation. `filters.outlier` with `method=statistical`.
- **Radius-based isolation removal** — drop points with too few neighbours within a radius. `filters.outlier` with `method=radius`.
- **Height gating** — reject returns above a plausible ceiling.

The variation here is in *aggressiveness*, and it is invisible in the file. An aggressive filter removes genuine sparse canopy-top returns along with noise; a permissive one leaves birds and cloud returns. Since canopy-top returns in sparse crowns are exactly the low-density, isolated points that isolation-based filters target, **aggressive noise filtering systematically truncates the upper canopy in sparse-crowned forests** — savanna, woodland, open conifer stands. This is a direct bias on maximum-height and upper-percentile metrics, concentrated in specific ecosystems.

Combined with §6.4, the practical policy is: honour the withheld flag, then apply your own consistent conservative filter, and record how many points each stage removed.

10.7 Above-ground classification

3DEP requires only a minimum classification scheme oriented toward terrain production — ground, water, noise, bridge deck, ignored ground, plus unclassified. **Vegetation classes (3/4/5, low/medium/high) and buildings (6) are optional and project-specific.** Many 3DEP projects deliver everything non-ground as class 1.

This is important and often misunderstood by people arriving from the semantic-segmentation literature: **3DEP is not a labelled semantic segmentation dataset.** You cannot assume vegetation classes exist, and where they do exist you cannot assume they mean the same thing across projects. If your benchmark includes a semantic segmentation task, the labels have to come from somewhere else — from a program that requires them (IGN Lidar HD, FRACTAL) or from your own annotation.

Where vegetation classes *are* delivered, the height thresholds separating low/medium/high vegetation are project-specific and unrecorded, so class 3-vs-4-vs-5 boundaries are not comparable.

10.8 Intensity normalization

Whatever the vendor did to intensity is done (§3.7). The specification requires linear scaling to 16 bits and forbids image-style stretches, which constrains the transform without identifying it.

If you decide to use intensity, the defensible options are:

1. **Per-project rank or quantile normalization.** Removes any monotone reparameterization, including unknown range normalizations and log scalings. Does not remove within-project AGC variation or incidence-angle modulation.
2. **Range normalization**, if you can estimate range from `ScanAngleRank` and a nominal flying height. Correct for beam-filling targets, wrong for canopy (§1.2), and it may double-apply if the vendor already did it.

3. **Incidence-angle correction** using terrain normals plus scan angle. Removes the cos term for ground; meaningless for canopy.
4. **Learned normalization** — a per-project affine transform fit as part of training. Elegant; fails at inference on unseen projects.

My recommendation stands from §3.7: exclude by default, ablate explicitly, and if you use it, use per-project quantile normalization and report leave-one-project-out results.

10.9 Tiling

The final step. Points are partitioned into a project tiling scheme, delivered without overlap (§6.4). Consequences:

- **Tile boundaries are arbitrary** relative to landscape features, and any per-tile computation (density, statistics, normalization) has edge effects.
- **Tiling schemes differ between projects** — 1 km, 1500 ft, 5000 ft, State Plane vs. UTM grids. Do not build a pipeline that assumes a tile size.
- **Any neighbourhood computation near a tile edge is truncated** unless you read the neighbouring tile. Ground classification, HAG, and geometric features all need a buffer. Cloud-native formats (COPC, EPT) largely dissolve this problem because you query by bounding box rather than by tile, which is one of the strongest practical arguments for using them (Chapter 13).

10.10 Trust or re-derive: a decision table

Consolidating Chapters 9 and 10 into a recommendation per processing stage.

Stage	Accept vendor output?	Rationale
Georeferencing / geometry	Accept. You have no alternative.	Trajectory not delivered; no route to reprocessing
Strip adjustment	Accept, but measure.	Use swath separation images and seam diagnostics as quality covariates
Absolute vertical calibration	Accept.	Control not available to you; note residual spatial structure
Withheld flagging	Honour it, always.	Required since LBS 2020 rev. A; supersedes classification (§6.4)
Noise / outlier removal	Re-derive, conservatively.	Vendor aggressiveness varies and biases sparse canopy tops
Ground classification	Re-derive, and ablate.	Highest-leverage variable; unspecified and unassessed under canopy
Height above ground	Always re-derive.	Depends on ground; must be consistent with your ground choice
Hydro-flattening	Accept, flag, and stratify.	Cannot be undone; treat as known-artifact zone
Vegetation / building classes	Do not rely on.	Optional and inconsistent across 3DEP

Stage	Accept vendor output?	Rationale
Intensity	Exclude by default; ablate.	Semantics unknown and project-correlated
Tiling	Ignore; use bbox queries.	COPC/EPT make tile boundaries irrelevant

Further reading for Chapter 10

- Axelsson (2000), *DEM generation from laser scanner data using adaptive TIN models*, on progressive TIN densification — the algorithm behind much of your corpus.
- Kraus & Pfeifer (1998), on hierarchical robust interpolation.
- Zhang et al. (2016), *An easy-to-use airborne LiDAR data filtering method based on cloth simulation*, Remote Sensing — the CSF paper; readable and with good figures.
- Pingel et al. (2013), *An improved simple morphological filter for the terrain classification of airborne LIDAR data*, ISPRS Journal — the SMRF paper, and the reference for PDAL’s implementation.
- Sithole & Vosselman (2004), *Experimental comparison of filter algorithms for bare-Earth extraction*, ISPRS Journal — the classic comparative benchmark of ground filters, and the best single source for the failure modes in §10.4. Its figures showing where each filter fails are worth studying carefully.
- [PDAL filter documentation](#) for the concrete implementations.

Chapter 11. What Has Already Been Done to Your File

This chapter is an audit. The question it answers is: given a LAZ file from 3DEP, what transformations have been applied, which can be undone, which can be detected, and which are silently inconsistent across the archive?

I consider this the most important chapter in the first half of the book, because it is the chapter that determines whether your benchmark’s negative results are interpretable.

11.1 A taxonomy of transformations

Every operation in Parts I–III falls into one of four categories, and the category determines what you can do about it.

Category A — Reversible. Rare. Compression is the main example.

Category B — Irreversible and total. The information is gone and no amount of cleverness recovers it. You can only decide whether your task depends on it.

Category C — Lossy but detectable. Information was degraded, but you can measure the degradation from the delivered data and either correct it or condition on it.

Category D — Silently inconsistent. The operation was applied differently across projects, and the delivered file does not record which variant was used. **This is the dangerous category**, because it produces variation that looks like signal.

The purpose of an audit is to move as much as possible from D to C.

11.2 Category B: irreversible and total

Lost	When	Consequence
Full waveform	At discretization; and 3DEP removed it from the specification entirely (§6.3)	No echo width, no sub-dead-time structure, no calibratable amplitude, no continuous vertical energy profile
Sub-resolution-cell structure	Dead time and pulse width (§3.5)	Near-ground stratum permanently under-sampled; magnitude sensor-dependent
AGC gain state	At intensity recording, if not carried through	Intensity uninterpretable in absolute terms
Returns beyond the sensor or format cap	At discretization (§3.6)	Profile interior truncated; return-ratio metrics unreliable
Overage/sidelap points	At tiling, under current specification (§6.4)	No two-look geometry in tiles; must use swath files
Trajectory (SBET)	Never delivered	Cannot recompute rigorous scan geometry or incidence angles
Range and mirror angle as observed	At georeferencing	Only the coarse <code>ScanAngleRank</code> proxy remains
Points removed by noise filtering	At QC	Sparse canopy tops may be gone; §10.6
Real ground under hydro-flattening	At hydro-flattening	Riparian and wetland HAG is relative to a fabricated surface

How to use this table. Not as a lament, but as a filter on task design. If a proposed benchmark task depends on something in this list, either the task is impossible on 3DEP or it requires a different data source. Concretely: **understory-dependent tasks are limited by rows 2 and 8; any task requiring calibrated radiometry is limited by row 3; echo-width-based roughness measures are impossible; and multi-view self-supervision requires swath files, not tiles.**

11.3 Category C: lossy but detectable

Degradation	How to detect it	What to do
Coordinate quantization	Read <code>scale</code> and <code>offset</code> from the header; check whether coordinate values fall on the implied grid	Usually 0.01 m or 0.001 m — negligible for ecology, but check, because some projects use coarser scales
Point density realization	Compute first-return density per unit area (§5.4)	Store as a covariate; use for stratification and for thinning-augmentation targets
Returns per pulse	Total points / first returns	Store; it partly captures the energy/footprint regime
Return-count cap	Max of <code>NumberOfReturns</code> over the sample	Store; needed for return-truncation augmentation

Degradation	How to detect it	What to do
Withheld burden	Fraction with the Withheld flag set	QC signal; zero may mean pristine <i>or</i> pre-2020
Seam structure	Count of distinct PointSourceId ; scan-angle histogram modality; local density variance	Store as a covariate; identifies sidelap regions
Scan-angle range actually used	Percentiles of ScanAngleRank	Reveals FOV and swath position; a proxy for occlusion geometry
Pass timing	GpsTime histogram and gaps	Reveals multi-sortie composition and change risk
Vertical outlier contamination	Upper tail of <i>Z</i> after withheld filtering; MSHR (§6.4)	Screen and flag; do not let it reach the model
Ground-classification quality	Ground-return fraction by slope class and scan angle; agreement between your re-derived DTM and the delivered DEM	Store disagreement magnitude as a quality covariate
Vertical datum	Header WKT	Store verbatim; do not normalize away (§9.7)
Intensity distribution shape	Histogram; look for clipping at 0 and 65535, for suspiciously uniform distributions (stretch), for multimodality (AGC states)	Diagnoses whether the vendor complied with linear-16-bit; informs whether to use intensity at all

This table *is* the audit protocol. Every row is computable from the delivered data, most in a single pass, and together they characterize a project’s position in the heterogeneity space of §4.1 without reading a single PDF.

11.4 Category D: silently inconsistent

The dangerous category. For each item, the variation exists, the file does not record it, and the effect is systematic rather than random.

Discretization algorithm and thresholds (§3.4). Two vendors on the same waveform produce different point counts at different elevations. Partially detectable via returns-per-pulse statistics, which is why that covariate matters.

Ground classification algorithm and parameters (§10.3–10.4). The highest-leverage inconsistency. Partially detectable by comparing your own re-derivation against the delivered classification: the *pattern* of disagreement (concentrated on slopes? in dense canopy? at breaklines?) is diagnostic of the algorithm family.

Noise-filter aggressiveness (§10.6). Detectable only indirectly, via the sparse-canopy-top statistics, and confounded with real ecology. This is the hardest one to pin down; I would treat it as an acknowledged unmeasured confound.

Intensity convention (§3.7, §10.8). Partially detectable from distribution shape.

Strip-adjustment parameter model (§10.1). Detectable where swath separation images are archived; otherwise inferable from scan-angle-dependent vertical residuals at seams.

Overlap handling (§6.4). Detectable: class 12 present, overlap flag set, or doubled density all leave signatures. This one moves cleanly from D to C.

Vegetation-class height thresholds (§10.7). Not detectable, and a reason not to rely on those classes.

Phenological state. Not recorded in the file, but *fully recoverable* from the acquisition date via WESM plus an ecoregion-appropriate phenology model — which converts a major confound from D to C for the cost of a spatial join. This is one of the highest-value cheap moves available to you.

11.5 The extra layer: cloud copies and derived distributions

One transformation category that Parts I–III have not covered: the conversions applied by whoever hosts the copy you actually read.

The USGS EPT resources on AWS are served in **EPSG:3857** (Web Mercator), which means a reprojection from the project’s native CRS was applied at build time. Web Mercator is a poor choice for area and distance computation, and any geometric feature computed in it is distorted in a latitude-dependent way. If you read from EPT, **reproject to a locally appropriate CRS before computing anything geometric**, and be aware that you are reprojecting a reprojection.

Microsoft’s Planetary Computer COPC copy of 3DEP reprojects into UTM zones, and its own documentation notes that the **vertical datum was not adjusted during reprojection**, and that vertical datum descriptions may not match actual values for older (pre-2010) data. That is a documented, acknowledged inconsistency in the metadata of a widely used distribution. Combined with §9.7’s point about NAVD88’s tilt, it means: if you read PC’s COPC copy, you have a horizontal reprojection applied, a vertical datum that may be mislabelled, and a continental-scale systematic in the vertical datum itself. Do not build absolute-elevation dependence on top of that stack.

Add these to the provenance schema explicitly. **The distribution you read from is part of the processing chain**, and “3DEP” is not a sufficient description of your data source. 3DEP via PC COPC, reprojected to UTM 13N, vertical datum as labelled but unverified is.

[Case study] Given the reliability question flagged earlier about PC’s 3DEP collection availability, plus the reprojection layer, plus the Preview branding, my recommendation is to treat the AWS holdings as the primary source — with the note that `s3://usgs-lidar-public` EPT is Web Mercator and `s3://usgs-lidar` requester-pays LAZ is native-CRS full-density. For a benchmark you intend others to reproduce, native-CRS source data with your own documented reprojection is worth the extra engineering over a convenient pre-reprojected copy.

11.6 The audit protocol

Concretely: a single pass over each sampled tile that computes everything in §11.3 and stores it. My proposed implementation shape, in PDAL/Python terms:

1. **Header read only** (no points): CRS WKT, vertical datum, scale/offset, point format, LAS version, creation date, generating software (the `SoftwareId` field in the header is

- often filled in and is a direct clue to the processing stack — TerraScan, LP360, GeoCue, and so on), extent, point count by return number, class histogram if present in EVLRs.
2. **One streaming pass over points**, after dropping withheld: accumulate density on a 10 m grid; first-return count; max `NumberOfReturns`; `ScanAngleRank` histogram; `PointSourceId` set and per-ID counts; `GpsTime` min/max/histogram; Z percentiles including extreme tails; intensity histogram; class histogram.
 3. **Ground re-derivation** (`filters.smrf` or `filters.csf`) and HAG computation, plus comparison of the derived ground surface against the delivered DEM raster where available: mean, RMS, and 95th-percentile absolute difference, stratified by slope class.
 4. **MSHR** at 1 m or 2 m from the highest non-withheld point, retained as a small raster for visual QC. Borrow the specification's own idea (§6.4).
 5. **Spatial joins**: WESM for project ID, quality level, acquisition dates, LBS version; ecoregion; land cover; a phenology assignment from date plus ecoregion.
 6. **Write one provenance record** per sample.

Cost: roughly one extra pass over data you are already reading, plus the ground re-derivation which you want anyway. This is not an expensive protocol; it is an unusual one.

11.7 The provenance schema

The final form of §4.6 and §6.8, as something you could implement tomorrow.

```
{
  "sample_id": "3dep_co_2019_n40w105_t0421_s0007",
  "geometry_wkt": "POLYGON(...)",
  "source": {
    "program": "USGS_3DEP",
    "distribution": "aws_s3_usgs-lidar_requester_pays",
    "native_crs_wkt": "...",
    "vertical_datum": "NAVD88(GEOID18)",
    "vertical_datum_verified": true,
    "reprojections_applied": ["EPSG:6342"],
    "access_date": "2026-08-10"
  },
  "project": {
    "wesm_work_unit": "C0_CentralFrontRange_2019_D19",
    "quality_level": "QL1",
    "lbs_version": "2018 rev. A",
    "acquisition_start": "2019-09-14",
    "acquisition_end": "2019-10-30",
    "sensor_model": "Riegl VQ-1560i",
    "wavelength_nm": 1064,
    "scanner_mechanism": "rotating_polygon",
    "nominal_agl_m": 1700,
    "prf_khz": 800,
    "max_returns_per_pulse": 8,
```

```
    "coverage_pattern": "single_with_30pct_sidelap",
    "metadata_completeness": "report_parsed"
  },
  "file": {
    "las_version": "1.4",
    "point_data_record_format": 6,
    "generating_software": "TerraScan 020.005",
    "scale_xyz": [0.01, 0.01, 0.01]
  },
  "realized": {
    "pulse_density_per_m2": 8.4,
    "point_density_per_m2": 19.7,
    "returns_per_pulse_mean": 2.35,
    "max_return_number_observed": 8,
    "scan_angle_abs_p50": 9.0,
    "scan_angle_abs_p95": 19.0,
    "scan_angle_modality": "bimodal",
    "point_source_id_count": 3,
    "gps_time_span_hours": 26.4,
    "withheld_fraction": 0.0021,
    "class_histogram": {"1": 0.71, "2": 0.27, "7": 0.001, "9": 0.02},
    "density_cv_10m": 0.31,
    "z_p999_m": 2841.2,
    "intensity_distribution": "linear_16bit_plausible"
  },
  "processing": {
    "withheld_honoured": true,
    "noise_filter": "pdal_filters.outlier_statistical_k8_m3",
    "ground_source": "rederived_smrf",
    "ground_params": {"scalar": 1.25, "slope": 0.15, "window": 18, "threshold": 0.5},
    "hag_method": "filters.hag_nn",
    "dtm_vs_delivered_dem_rmse_m": 0.14,
    "dtm_vs_delivered_dem_p95_m": 0.38,
    "dtm_disagreement_by_slope_class": {"0-5": 0.06, "5-15": 0.11, "15-30": 0.29, ">30": 0.71},
    "intensity_retained": false
  },
  "context": {
    "epa_ecoregion_l3": "21_Southern_Rockies",
    "nlcd_dominant": "42_Evergreen_Forest",
    "nlcd_fractions": {"42": 0.68, "52": 0.19, "71": 0.09, "31": 0.04},
    "phenology_state": "leaf_off_conifer_dominant",
    "slope_p50_deg": 17.4,
    "slope_p95_deg": 33.1,
    "elevation_mean_m": 2610,
```

```

    "water_fraction": 0.001,
    "hydroflattened_fraction": 0.0,
    "distance_to_road_m_p50": 480
  },
  "splits": {
    "pretrain_eligible": true,
    "eval_holdout": null,
    "spatial_block_id": "SR_blk_0143",
    "neon_site_buffer_exclusion": false
  }
}

```

Notes on the design:

- **metadata_completeness** records how much you actually recovered — `report_parsed`, `wesm_only`, or `measured_only`. Without this field, missing sensor metadata is indistinguishable from a sensor you failed to look up, and you cannot honestly report how much of your corpus has full provenance.
- **vertical_datum_verified** exists because of §11.5.
- **dtm_disagreement_by_slope_class** is the field that will let you answer the question “is this model’s canopy-height error model error or terrain error?” — which you cannot answer from published accuracy metadata (§6.5).
- **distance_to_road_m_p50** is there for the shortcut-detection baseline in the biodiversity tasks (§7.7).
- **splits** lives in the provenance record rather than in a separate manifest so that a sample can never be used in a way inconsistent with its own eligibility. In particular `neon_site_buffer_exclusion` enforces the leakage rule for the paired evaluation design.

11.8 What the audit buys you

Three concrete returns, to justify the engineering.

Interpretable negative results. When a task underperforms, you can slice by quality level, sensor, density, slope, DTM disagreement, and phenology, and see where the failure lives. Without the schema, “the model does poorly on biomass in the Southeast” is a sentence with a dozen possible explanations and no way to choose.

Legitimate stratified reporting. The eight-axis transfer table of §8.5 requires exactly these fields. You cannot report leave-one-sensor-out if you do not know the sensor.

Defensible augmentation. The degradation augmentation of §4.5 needs realistic parameter ranges. Those ranges come from the empirical joint distribution of `pulse_density`, `returns_per_pulse`, `max_return_number`, and `scan_angle_abs_p95` across your corpus — i.e. from the audit. Augmenting toward a distribution you have measured is defensible; augmenting toward a guessed one is not.

[Case study] And one specific payoff for the paired 3DEP/NEON design. To estimate the sensor domain gap, you need the two clouds over the same footprint to be *identical in every respect except acquisition*. The provenance schema is what lets you verify that: same geometry,

same re-derived ground algorithm, same noise filter, same HAG method, same labels, different `project.sensor_model` and `realized.pulse_density`. Without that verification the “sensor gap” you measure could be a processing gap, and the headline result of that design collapses.

Further reading for Chapter 11

- [Lidar Base Specification: Data Processing and Handling Requirements and Deliverables](#) — the authority for what was supposed to happen to your file.
- [Appendix 1: Maximum Surface Height Rasters](#) — the QC idea in §11.6 step 4.
- [PDAL filters reference](#) for the implementations in §11.6.
- [pacasam](#) — for how another team structured the sampling-plus-metadata problem at national scale. Their schema choices are a useful check on mine.
- On dataset documentation as a practice: Gebru et al., *Datasheets for Datasets*, and Pushkarna et al., *Data Cards*. The provenance schema in §11.7 is a per-sample version of the same idea; the dataset-level version belongs in Chapter 26.

Bridge to Part IV

Parts I–III took a photon to a file and audited what happened along the way. Part IV is about the file itself, and about getting at it efficiently: the internals of LAS and LAZ, the cloud-native reorganizations (COPC and EPT) that make continental-scale querying tractable, PDAL as the data layer, and the discovery and catalog machinery that connects a coordinate to a work unit and its metadata.

Chapter 12 covers LAS/LAZ internals, including the fields Part III has repeatedly told you to preserve. Chapter 13 covers COPC and EPT. Chapter 14 is the PDAL chapter — the longest practical chapter in the book. Chapter 15 covers discovery, catalogs, and the provenance-capture engineering that turns §11.7 from a schema into a database.

Part IV — Format, Access, and Provenance Capture

Airborne Laser Scanning for Machine Learning in Ecology

Orientation to Part IV

Part III ended with a schema and a claim: that a per-sample provenance record is the difference between an interpretable benchmark and an uninterpretable one. Part IV is about the machinery that makes the schema fillable and the corpus buildable.

Four chapters:

- **Chapter 12** covers LAS and LAZ internals. This is a format chapter, but it is not a reference-manual chapter: I cover the fields that matter for ecology and for the audit protocol, the bit-level details that cause real bugs, and the LAS 1.5 transition that is now in progress.

- **Chapter 13** covers the cloud-native reorganizations — COPC and EPT — which are the reason continental-scale ALS querying is tractable at all. The key idea is that both are octrees, and understanding the octree explains both the capabilities and the traps.
- **Chapter 14** is the PDAL chapter: the longest practical chapter in the book, and the one to keep open while writing code.
- **Chapter 15** covers discovery and provenance capture: WESM, the National Map, the AWS holdings, NEON's API, and the engineering that turns §11.7's schema into a database you can query.

A recurring theme: **the cheap thing and the correct thing diverge more often here than anywhere else in the book**. Reading a pre-reprojected cloud copy is cheap. Trusting **resolution** to mean what it says is cheap. Skipping the header parse is cheap. Each of these costs you something specific, and this Part is largely about naming those costs.

Chapter 12. LAS and LAZ Internals

12.1 The container

A LAS file has four parts, in order: **public header block**, **variable length records (VLRs)**, **point data records**, and **extended variable length records (EVLRs)**. All data is little-endian. The header is fixed-layout; everything else is described by offsets and counts in the header.

The public header block carries: file signature (LASF), version, system identifier, generating software, creation day/year, header size, offset to point data, VLR count, point data record format and length, point counts (legacy 32-bit and, in 1.4, 64-bit), point counts by return number, scale factors and offsets for X/Y/Z, the bounding box, and — in 1.4 — offsets to the waveform data packet record and to the first EVLR.

Three header fields deserve immediate attention for the audit protocol:

Generating software. A free-text field, usually populated with the producing package and version: TerraScan 020.005, LP360, GeoCue, PDAL. This is a *direct* clue to the processing stack (§11.4) and it costs nothing to read. Across a national corpus, the distribution of this field is essentially a map of contractor tooling.

Scale and offset. LAS stores coordinates as 32-bit signed integers; the real-world value is $\text{scale} \times \text{value} + \text{offset}$. Typical scale is 0.01 or 0.001 m. Two failure modes: a coarse scale (0.1 m, occasionally seen) quantizes coordinates enough to matter for fine geometric features; and the 32-bit integer range combined with a small scale bounds the representable extent, which is why offsets are usually set near the tile origin. **Read them; do not assume.**

Point counts by return number. In 1.4 this is a 15-element array. It gives you the returns-per-pulse distribution for the whole file *from the header alone*, without touching a single point. For a cheap first-pass corpus characterization — density regime, return-count cap, canopy structure proxy — the header is often enough.

VLRs are records of at most 64 KB, each identified by a user ID and record ID: CRS information, the LASzip compression descriptor, Extra Bytes definitions, classification lookups, and application-specific payloads (including COPC's, per Chapter 13). **EVLRs** are the same idea with a 64-bit length field, stored after the points, which is where waveform packets and large payloads go.

12.2 Point data record formats

This is the table you will consult repeatedly.

PDRF	Base size	Adds	Return bits	Notes
0	20 B	—	3+3	Legacy core
1	28 B	GPS time	3+3	Legacy
2	26 B	RGB	3+3	Legacy
3	34 B	GPS time + RGB	3+3	Legacy
4	57 B	GPS time + wave packets	3+3	Legacy waveform
5	63 B	GPS time + RGB + wave packets	3+3	Legacy waveform
6	30 B	GPS time, always	4+4	The modern baseline; 3DEP default
7	36 B	+ RGB	4+4	
8	38 B	+ RGB + NIR	4+4	Enables per-point NDVI
9	59 B	+ wave packets	4+4	
10	67 B	+ RGB + NIR + wave packets	4+4	

The 0–5 versus 6–10 divide is the one that matters, and 3DEP requires 6–10 (§6.6). Five differences:

1. **Return number and number of returns widen from 3 to 4 bits** — a cap of 7 becomes a cap of 15. Sensors capable of 8+ returns per pulse *cannot be represented* in formats 0–5 without truncation (§3.6).
2. **Classification widens from 5 bits to a full byte** — 32 classes become 256. In formats 0–5, classification and three flag bits shared one byte; in 6–10 they are separate fields.
3. **A dedicated classification-flags byte** holds Synthetic, Key-point, Withheld, and **Overlap** — the last of which does not exist in formats 0–5, which is why older data used class 12 for overage (§6.4).
4. **Scan angle changes from ScanAngleRank** (signed byte, 1° resolution, ± 90) **to a signed 16-bit field at 0.006° resolution**. This is a large precision gain and it matters if you use scan angle as a covariate (§2.3). PDAL exposes both **ScanAngleRank** and **ScanAngle** depending on the reader and format; check which you are getting.
5. **A ScannerChannel field** (2 bits) identifies which channel of a multi-channel sensor fired the pulse. For dual- and quad-channel systems (§1.6), this is how you separate channels,

and it is essential if you want to reason about per-channel geometry.

[Case study] Points 1 and 4 are the ones with direct ML consequences. If your corpus mixes formats — and a decade-spanning 3DEP sample will — then the return-count cap and the scan-angle resolution are *format-dependent*, not just sensor-dependent. Record `point_data_record_format` in the provenance schema (it is already in §11.7’s `file` block) and check whether your return-count statistics show a suspicious pile-up at 7.

12.3 The classification byte and the flag byte

A recurring source of bugs, so explicitly:

In PDRF 0–5, one byte holds classification in bits 0–4 (values 0–31) and Synthetic, Key-point, Withheld in bits 5–7. Reading the raw byte as “classification” yields values above 31 when flags are set. Every mature library handles this; hand-rolled parsers frequently do not.

In PDRF 6–10, classification is a full byte (0–255) and the flags live in a separate byte: bit 0 Synthetic, bit 1 Key-point, bit 2 Withheld, bit 3 Overlap, bits 4–5 Scanner Channel, bit 6 Scan Direction, bit 7 Edge of Flight Line.

The four flags, with their practical status in a 3DEP corpus:

- **Synthetic** — the point was not measured but derived (e.g. from a breakline). Rare in 3DEP point deliverables but present in some hydro-flattened products. If present, these are fabricated points and should not be treated as observations.
- **Key-point** — a model key point, flagged as required by the specification if computed (§6.4). A thinned subset that adequately represents the surface; occasionally useful, more often ignorable.
- **Withheld** — **the critical one** (§6.4). Not a valid surface return. Supersedes classification entirely. Filter on it first, always.
- **Overlap** — shall not be used in 3DEP deliverables (§6.4), because overage is removed at tiling instead. Its presence in a file tells you the file predates or diverges from current practice, which is itself informative.

Standard classes worth knowing: 0 never classified, 1 unclassified, 2 ground, 3/4/5 low/medium/high vegetation, 6 building, 7 low noise, 9 water, 10 rail, 11 road surface, 12 (reserved; historically overlap), 13–16 wires and transmission tower, 17 bridge deck, 18 high noise, 19 overhead structure, 20 ignored ground, 21 snow, 22 temporal exclusion. Classes 19–22 were added in LAS 1.4 R14 and exist only for PDRF 6–10.

Two of these deserve comment for ecology. **Class 20 (ignored ground)** is real ground deliberately excluded from the terrain model (§6.4, §10.4) — so a DTM built from class 2 alone differs from the delivered DEM in exactly the places where terrain is steep or overhanging. **Class 22 (temporal exclusion)** exists for points that were valid at acquisition but represent transient conditions; it is rarely used and worth checking for.

And the caution from §10.7 bears repeating: **vegetation classes 3/4/5 are optional in 3DEP and their height thresholds are project-specific**. Do not build a task on them.

12.4 Fields worth preserving

Part III repeatedly said “preserve this.” Consolidated:

Field	Why
Classification	Ground for HAG; noise screening; not much else in 3DEP
ClassFlags / Withheld	Correctness (§6.4)
ReturnNumber, NumberOfReturns	Pulse structure; return-ratio features; augmentation targets
ScanAngleRank / ScanAngle	Occlusion geometry, path length, footprint proxy (§2.3)
PointSourceId	Flight line identity — seam structure, multi-view pairing (§9.8)
GpsTime	Pulse grouping; pass timing; change risk (§9.6)
ScannerChannel	Multi-channel sensor geometry
EdgeOfFlightLine	Nominally useful; frequently unpopulated — verify before trusting
Intensity	Retain in storage, exclude from model by default (§3.7)
NIR (PDRF 8/10)	Per-point NDVI where available; rare in 3DEP

The two most under-used are **PointSourceId** and **GpsTime**. Together they reconstruct acquisition geometry that is nominally absent from the file, and I know of no published ALS-ML pipeline that uses either as a feature or a covariate.

12.5 LAZ

LAZ is LAS with the point block compressed. The header and VLRs stay readable; a VLR describes the compression; the points are encoded with an arithmetic coder using predictive models specialized per field.

Two properties matter architecturally:

Chunking. Points are compressed in independent chunks (default 50,000 points), with a chunk table giving offsets. Any chunk can be decompressed without decompressing its predecessors. This is what makes random access possible, and it is the foundation COPC builds on.

Layered encoding for PDRF 6–10. For the modern point formats, LAZ subdivides each chunk into *layers*, each holding a subset of fields. A reader wanting only X, Y, Z can decompress only those layers and skip intensity, GPS time, and RGB entirely. **This is dimension-selective access**, and for a training pipeline that ignores intensity and RGB it is a substantial I/O saving that comes for free if your reader supports it.

Compatibility mode encodes 1.4 point formats inside a 1.2-style structure so legacy readers can open the file, moving the extra fields into Extra Bytes. 3DEP explicitly prohibits it (§6.6), which is good news: you get real 1.4 chunked LAZ, with the layered encoding and clean COPC conversion.

Implementations: LASzip (original C++), laz-perf (C++ with a JavaScript build), laz-rs (Rust). The JavaScript builds are why browser-based viewers like viewer.copc.io work at all.

12.6 Extra Bytes

The Extra Bytes VLR defines additional per-point dimensions with declared name, data type, scale, offset, and no-data value. Vendors use it for amplitude, reflectance, deviation, echo width, pulse width, AGC gain, and other sensor-specific quantities.

This is where the interesting sensor information hides when it is present at all. Riegl systems, for instance, can carry calibrated amplitude and reflectance in dB plus a pulse-shape deviation measure — which is a partial recovery of the radiometric information §3.7 said was lost, and echo width is a direct measure of within-footprint roughness (§3.3). If a project delivered these, your intensity problem is materially better.

In 3DEP, expect them to be absent, since the specification does not require them and tiling workflows often strip them. But *check*, because when present they are valuable and free. Reading the Extra Bytes VLR is a header-only operation.

12.7 LAS 1.5

LAS 1.5 was approved in August 2025. The [specification is now published in HTML at asprslas.org](https://asprslas.org), and the changes relevant to you are:

- **Legacy Point Data Record Formats 0–5 are removed.** LAS 1.5 supports only PDRF 6–10. The legacy formats are gone from the forward specification, though existing files remain readable under 1.4.
- **Max/Min GPS Time fields added to the header block.** A small change with a real benefit: the temporal extent of a file is now header-readable, so the pass-timing diagnostic of §9.6 becomes free.
- **An “Offset GPS Time” definition added**, formalizing the epoch-offset convention.
- **CRS support expanded to all published WKT versions, with GeoTIFF CRS encoding removed entirely.**
- **The Extra Bytes VLR must be unique** (a clarification).
- **The Classification Lookup VLR is deprecated.**
- Backward compatibility with 64-bit LAS 1.4 files, and header compatibility with 1.1–1.4.

Also worth knowing: **work on LAS 2.0 has been suspended indefinitely.** The LAS Working Group’s stated strategy is to route changes into revisions, domain profiles, and the wiki rather than breaking compatibility with new major versions. So LAS 1.4/1.5 is the stable substrate for the foreseeable future, and you are not building on sand.

Domain Profiles are the extension mechanism: a derivative of the base specification that *adds* point classes and attributes without removing or altering existing ones. The first was the **Topo-Bathy Lidar Domain Profile**, which adds classification values for bathymetric point, water surface, derived water surface, submerged object, IHO S-57 object, and bottom-not-found depth, plus Extra Bytes for pseudo-reflectance, uncertainty, water column depth, and figure of merit.

[**Case study**] There is an opportunity here worth flagging. A **vegetation/forestry domain profile** — standardizing per-point attributes like normalized height above ground, echo width, calibrated reflectance, and a consistent vegetation stratification — does not exist and would be a natural home for exactly the derived fields your pipeline computes. If your benchmark’s preprocessing converges on a stable set of per-point derived dimensions, proposing them as a domain profile is a low-cost, high-leverage contribution to the field’s infrastructure. The mechanism is public: the [ASPRS LAS GitHub repository](#) is the official collaborative editing environment, and issues there are how changes get proposed.

Further reading for Chapter 12

- [ASPRS LAS 1.5 specification \(HTML\)](#) — the current specification, browsable. The [LAS 1.5 change list](#) is the fastest orientation.
- [LAS 1.4-R15 PDF](#) — the version most of your corpus was written against.
- [ASPRS LAS GitHub](#) and its [wiki](#) — issues, rationale, and interpretation guidance. Reading the issue history is the best way to understand *why* fields are the way they are.
- [GeoCue: What is the LAS format?](#) — the clearest version-history summary, including domain profiles.
- [LAZ 1.4 R1 specification](#) for the chunking and layered encoding of §12.5.
- [LAS: What’s on the Horizon](#), LIDAR Magazine — the LAS Working Group’s own account of its stewardship philosophy, including why LAS 2.0 was abandoned.

Chapter 13. Cloud-Native Organizations: COPC and EPT

13.1 The problem both solve

A 1 km² QL2 tile is perhaps 100 MB compressed. CONUS at QL2 is a few hundred terabytes. Any workflow that requires downloading whole files to extract a 50 m patch is dead on arrival at this scale — and 50 m patches are exactly what your evaluation tasks need (§7.6).

What you want is: given a bounding box and a resolution, fetch approximately the points inside it and nothing else, over HTTP, without a server. That is the Cloud-Optimized GeoTIFF idea applied to point clouds, and both COPC and EPT implement it.

Both are **octrees**. Understanding the octree explains everything else, including the traps.

13.2 The octree

Recursively subdivide the cloud’s bounding cube into eight octants; subdivide each octant; and so on. Each node is identified by a **VoxelKey** (d, x, y, z): depth and integer position within that depth’s grid.

The crucial design decision — shared by both formats and inherited from Entwine — is that **points are distributed across depths rather than stored only at the leaves**. The root node holds a coarse subsample of the entire cloud. Depth 1 holds eight nodes, each a slightly finer subsample of its octant. And so on, until at the deepest level the remaining points are stored in place.

Two consequences follow, and they are the whole value proposition:

Level-of-detail is free. Reading only depths 0 through k gives you a spatially uniform subsample of the cloud at a resolution determined by k . No separate overview pyramid is needed; the structure *is* the pyramid.

Spatial subsetting is free. Reading only nodes whose bounding boxes intersect your query region gives you the points in that region, because the octree key encodes position.

Combined: **a bounding box plus a depth limit is a single indexed query returning approximately the right points.** That is the operation your data loader needs.

13.3 COPC

A COPC file is a LAZ 1.4 file that stores point data organized in a clustered octree, containing a VLR that describes the octree organization of data stored in LAZ 1.4 chunks. It is a *single file*, valid LAZ, readable end-to-end by any conventional LAZ reader that ignores the COPC VLRs.

The mechanism, per the [specification](#):

- **The COPC info VLR** sits immediately after the header at offset 375, and carries the octree centre coordinates, root node spacing, GPS time bounds, and the offset and size of the root hierarchy page.
- **Hierarchy pages** list entries, each of which refers to a child hierarchy page, an octree node data chunk, or an empty node. Each entry gives the VoxelKey, the file offset, the byte size, and the point count. So a client reads the header, reads the root hierarchy page, walks to the pages it needs, and then issues HTTP range requests for exactly the chunks it wants.
- **Only PDRF 6, 7, or 8 are permitted.** No waveform formats, no legacy formats.
- **Index overhead is small** — a few percent over plain LAZ.

The property I want to emphasize, because it is unusual and underappreciated: **COPC is not a new format requiring new software.** Clients that do not read the COPC VLRs can still read all the LAZ content without impact. And it is a valid LAS 1.4 deliverable, so unlike proprietary octree formats it can satisfy the Lidar Base Specification. A single artifact serves both archival delivery and cloud-native access. That is a rare and genuinely good piece of format design.

Validation and inspection: validate.copc.io for structural checking, viewer.copc.io for browsing a file in the browser (including remote URLs via a query parameter). Both are worth using early and often.

13.4 EPT

Entwine Point Tile is the same octree exposed as a **directory** rather than a file: an `ept.json` metadata document, a hierarchy of JSON files describing the tree, and per-node binary data files (LAZ, binary, or zstandard).

The differences that matter in practice:

	COPC	EPT
Storage	One file	Directory of many files
Metadata	VLRs inside the file	<code>ept.json</code> plus hierarchy JSONs
Access	HTTP range requests	One request per node
Source provenance	Not supported	<code>ept-sources.json</code> maps points to source files
Scale	Natural fit for a tile or a project	Natural fit for a whole campaign or an unbounded mosaic
Deliverable	Valid LAS 1.4	Not a LAS deliverable

COPC currently provides no support for `ept-sources.json`. That single row is more consequential for you than it looks. EPT’s source tracking is what makes the `origin` option work — the ability to select points that came from one specific source file within a mosaicked resource. In a corpus where source-file identity is a provenance field, that is a capability worth having.

The USGS EPT holdings on AWS are organized by USGS project name and served in **EPSG:3857** (Web Mercator) — the reprojection issue of §11.5. This is the single biggest practical drawback of the EPT route for your purposes, and I return to it in §13.7.

13.5 The resolution trap

The most important operational detail in this chapter.

Both readers accept a **resolution** parameter, and its documented semantics are: a point resolution limit expressed as a grid cell edge length, in resource coordinate system units — and the resulting resolution may not be exactly this value, since the minimum possible resolution that is at least as precise as the requested resolution will be selected.

Unpack that. Octree depths have fixed spacings, each roughly half the previous. A request for 1.0 m resolution does not return points at 1.0 m spacing; it returns **every point in every node down to the first depth whose spacing is at least as fine as 1.0 m**. If that depth’s spacing is 0.7 m, you get 0.7 m data — roughly twice the point count you may have budgeted for.

Three practical consequences:

1. **resolution is a coarse dial, not a fine one.** Sweeping it in small increments produces a step function.
2. **Point counts are not predictable from resolution alone.** If your loader assumes a point budget, verify empirically per resource.
3. **Across resources with different octree structures, the same resolution yields different effective resolutions.** For a corpus spanning many projects, this means “read everything at 1 m” does *not* produce a uniformly-sampled corpus. If you need uniform density, **read at full resolution and downsample yourself** with `filters.sample` or `filters.voxeldownsize`, which give you exact control. The extra I/O is real but the consistency is worth it, and for a benchmark that others will reproduce it is close to mandatory.

[**Case study**] This interacts directly with §5.4’s density point. You have two sources of density non-uniformity — the acquisition (uncontrollable) and the read (controllable). Do not let the second compound the first. Read full-resolution, record realized density, then normalize deliberately.

13.6 A second trap: reader bounds versus crop

A subtlety that has bitten people: `readers.copc`’s `bounds` option and `filters.crop`’s `bounds` option can produce **different point sets** from the same file, because the reader’s bounds are used for octree node pruning and the results are not always identical to a strict per-point spatial test. [PDAL issue #4230](#) documents a case where the reader returned 435 points and the crop filter 1,682 from the same query region.

Behaviour may have changed since that report; the operative advice is defensive rather than version-specific:

Use reader bounds or polygon as a prefetch optimization, and follow with an explicit filters.crop for the exact geometry you want. The reader restricts what is fetched over the network; the filter guarantees what is returned. The double specification costs almost nothing and makes your extraction geometry exact and reader-independent.

This matters especially for evaluation tasks, where the sample geometry must correspond precisely to a field plot’s footprint. An off-by-a-node sample is a silent label-misalignment bug.

13.7 Which to use

Recapping §11.5’s conclusion with the format detail now in place.

The AWS holdings are the durable option: `s3://usgs-lidar-public` for EPT resources (Web Mercator) and `s3://usgs-lidar requester-pays` for full-density LAZ in native CRS.

Planetary Computer’s COPC copy is convenient — STAC-searchable, COPC-native, well-documented with [example notebooks](#) — but it reprojects to UTM without adjusting vertical datum, carries possibly-mismatched vertical datum descriptions for pre-2010 data, has open reports of unreachable URLs, and is branded Preview.

My recommendation for a reproducible benchmark: native-CRS source LAZ from the requester-pays bucket, converted to COPC yourself with `writers.copc` or `untwine`, stored alongside your own provenance records. You pay conversion compute once and gain: native CRS with a documented reprojection of your choosing, verified vertical datum, no dependency on a third party’s Preview service, and a corpus artifact you can publish and version.

Use EPT when you need `origin`-based source tracking, or when you want to query a whole project as one mosaic rather than tile by tile. **Use COPC** for everything else, and especially for anything you intend to redistribute, since it is a single valid LAS 1.4 file.

Further reading for Chapter 13

- [copc.io](#) — the specification. Short and readable; read it properly rather than relying on summaries.
- [entwine.io/entwine-point-tile.html](#) — the EPT specification.

- [Cloud-Native Geospatial Formats Guide: COPC](#) — the best conceptual introduction, with the COG analogy developed properly.
- *Cloud Native Geospatial Lidar with the Cloud Optimized Point Cloud*, LIDAR Magazine — the design rationale from the people who built it, including the LBS-compatibility argument.
- [viewer.copc.io](#) and [validate.copc.io](#) — use both.
- [AWS Open Data registry: USGS 3DEP](#) for the bucket structure.

Chapter 14. PDAL as the Data Layer

14.1 The model

PDAL is a pipeline library: **readers** produce points, **filters** transform them, **writers** consume them. A pipeline is a JSON array of stages; stages connect implicitly in sequence or explicitly via **tag** and **inputs**.

```
[
  { "type": "readers.copc", "filename": "https://.../tile.copc.laz",
    "polygon": "POLYGON(...)) / EPSG:32613" },
  { "type": "filters.expression", "expression": "Withheld == 0" },
  { "type": "filters.smrf" },
  { "type": "filters.hag_nn" },
  { "type": "writers.las", "filename": "out.laz", "extra_dims": "HeightAboveGround=float" }
]
```

Streaming versus standard mode. Some stages process points one at a time in bounded memory (streamable); others need the whole set (ground classification, statistics, anything global). A pipeline is streamable only if every stage is. `pdal pipeline --stream` forces streaming and errors if it cannot. For corpus-scale work this distinction determines whether you can process a 1 km² tile in 2 GB or need 30 GB.

The Python bindings let you build pipelines programmatically and get numpy arrays back:

```
import pdal
pipeline = (
    pdal.Reader.copc(url, polygon=wkt, requests=3)
    | pdal.Filter.expression(expression="Withheld == 0")
    | pdal.Filter.smrf()
    | pdal.Filter.hag_nn()
)
n = pipeline.execute()
arr = pipeline.arrays[0] # structured numpy array
meta = pipeline.metadata
```

The pipe operator composes stages; `execute()` returns the point count; `arrays` gives structured numpy arrays; `metadata` returns a nested dict containing every stage's reported metadata — **which is where much of your provenance data comes from for free.**

Two performance notes: `pipeline.arrays` materializes everything in memory, so for large reads either restrict the region or use `execute_streaming()` with a chunk size. And the numpy round-trip copies, so avoid gratuitous pipeline splits.

14.2 Readers

readers.las for local LAS/LAZ. Options worth knowing: `extra_dims` to surface Extra Bytes (§12.6), `nosrs` to ignore an embedded CRS, `use_eb_vlr_names`, and `ignore_vlr`.

readers.copc for COPC, local or remote. Key options:

- **bounds** — (`[minx,maxx],[miny,maxy],[minz,maxz]`), used for octree pruning (see §13.6).
- **polygon** — a clipping polygon as WKT, specifiable more than once (treated as a multi-polygon), optionally followed by a slash and a spatial reference specification. This both pre-filters node selection and clips as read.
- **ogr** — a JSON object describing an OGR query to fetch polygons, treated exactly like the **polygon** option. This is the one to know: **you can drive extraction directly from a vector dataset of plot footprints** — a GeoPackage of NEON plots, a shapefile of eBird checklist buffers — without writing a loop. It accepts `datasource`, `drivers`, `openoptions`, `layer`, `sql`, and a `geometry` filter.
- **resolution** — the depth limit (§13.5).
- **requests / threads** — the number of worker threads, defaulting to 15, with the documentation’s own caveat that more is not necessarily better, since a fast connection can fetch faster than the data can be processed, leading to memory consumption and slower performance.

readers.ept for EPT, with the same **bounds**, **polygon**, and **resolution** semantics plus:

- **origin** — restrict to points from one source file within the resource. The provenance capability COPC lacks (§13.4).
- **threads** — default 15.

On concurrency, a practical warning. The default of 15 concurrent requests is per reader. A pipeline merging 10 tiles, run across 16 worker processes, is 2,400 simultaneous HTTP requests. You will be throttled, and worse, you may not realize it — throttling manifests as mysterious slowness rather than an error. **Set requests explicitly to something small (2–4) whenever you are fanning out across workers**, and let process-level parallelism do the scaling.

readers.stac drives reads from a STAC search: **bounds** prunes features, an OGR query can select a polygon to prune features by, and **reader_args** forwards per-reader options to the underlying reader, e.g.:

```
--readers.stac.reader_args '{"type": "readers.copc", "filename": {"headers": {...}}}'
```

This is elegant for catalog-driven workflows, at the cost of coupling your pipeline to a STAC endpoint’s availability.

14.3 Filters: the ones that matter

Organized by role in your pipeline.

Validity and selection

- `filters.expression` — the modern general filter, taking expressions like `Withheld == 0 && Classification != 7 && NumberOfReturns > 0`. Prefer this over the older `filters.range` for anything non-trivial.
- `filters.range` — the legacy dimension-range syntax; still common in examples.
- `filters.crop` — exact spatial cropping by `bounds`, `polygon`, or `distance` from a point. **Always use this to define your final sample geometry** (§13.6).
- `filters.head` / `filters.tail` — first or last N points; useful for quick tests.

Reprojection and CRS

- `filters.reprojection` — `in_srs/out_srs`. Note the vertical datum caveat: reprojection does not automatically transform vertical datums unless the CRS definitions are compound and the transformation grids are available. Verify rather than assume (§9.7, §11.5).

Ground and height

- `filters.smrf` — Simple Morphological Filter; the pragmatic default. Parameters: `scalar`, `slope`, `threshold`, `window`, `cell`.
- `filters.csf` — Cloth Simulation Filter. Parameters: `resolution`, `rigidness`, `threshold`, `smooth`.
- `filters.pmf` — Progressive Morphological Filter.
- `filters.hag_nn` — height above ground from nearest ground neighbours. Fast, reasonable.
- `filters.hag_delaunay` — HAG from a Delaunay triangulation of ground points. More accurate, especially with sparse ground; more expensive.
- `filters.hag_dem` — HAG from an external raster DEM. **The one to use if you want HAG relative to the delivered 3DEP DEM** rather than to your own re-derived ground — which makes it the tool for the §10.10 ablation.

Noise

- `filters.outlier` — `statistical` (mean distance to `mean_k` neighbours beyond multiplier standard deviations) or `radius` methods.
- `filters.elm` — Extended Local Minimum, for low outliers specifically.

Both *classify* rather than remove by default (assigning class 7), so follow with an expression filter if you want them gone.

Density normalization

- `filters.sample` — Poisson-disk sampling at a specified `radius`. Produces a spatially uniform subsample that preserves the point distribution's character better than voxel methods. **This is the right tool for cross-sensor density normalization** (§4.5).

- `filters.voxel_downsize` — one point per voxel (`center` or `first` mode). Faster, more regular, less natural.
- `filters.decimation` — every Nth point. **Do not use this for density simulation:** it ignores spatial structure and, worse, it breaks pulse grouping (§4.5).
- `filters.relaxation_dart_throw` — an alternative Poisson-disk implementation.

Geometric features

- `filters.covariance_features` — computes linearity, planarity, scattering, verticality, omnivariance, anisotropy, eigenentropy, and curvature from the local covariance eigendecomposition, over a `knn` or `radius` neighbourhood. This is the classic handcrafted feature set (§16), computed correctly and fast.
- `filters.eigenvalues`, `filters.normal` — the underlying quantities.
- `filters.optimal_neighborhood` — selects a per-point neighbourhood size by minimizing eigenentropy, then feeds `filters.covariance_features`. Worth knowing about: it addresses the scale-selection problem that makes fixed-k geometric features fragile under variable density — which is exactly your situation.

Partitioning

- `filters.chipper` — partitions into spatially coherent blocks of approximately `capacity` points. The natural tool for producing training crops.
- `filters.splitter` — partitions on a fixed-size grid with optional `buffer`. **Use this rather than chipper when you need fixed *area* rather than fixed point count** — which, per the block-size argument, is usually what an ecological task wants.
- `filters.divider` — splits into a fixed number of partitions.

Assignment and computation

- `filters.assign` — set dimension values conditionally; useful for injecting provenance flags into the point record.
- `filters.ferry` — copy a dimension to a new name; needed when you want to preserve an original before overwriting.
- `filters.python` — run arbitrary numpy code in-pipeline. The escape hatch. Convenient, and a serialization risk for reproducibility — prefer explicit stages where possible.

Statistics

- `filters.stats` — per-dimension count, min, max, mean, stddev, and optional `enumerate` for categorical dimensions. **This one stage populates a large fraction of §11.7's *realized block***, and it is streamable.

14.4 Writers

- `writers.las` / `writers.copc` — point output. `extra_dims` writes computed dimensions like HeightAboveGround into Extra Bytes; `forward` copies header fields from the input, which is how you preserve provenance through a pipeline.
- `writers.gdal` — rasterize to GeoTIFF with `resolution`, `output_type` (min, max, mean, idw, count, stddev), `window_size`, and `dimension`. This is how you produce CHM, DTM,

density rasters, and the MSHR of §11.6 step 4 — a single stage with `dimension: Z`, `output_type: max`, on non-withheld points.

- `writers.text / writers.numpy` — for debugging and interchange.

14.5 A worked extraction pipeline

Extracting one evaluation sample around a field plot, with provenance capture, in the form I would actually use:

```
import pdal, json

def extract_sample(copc_url, plot_wkt_utm, epsg, buffer_m=100):
    """Extract a plot-centred sample with a buffer for edge-free neighbourhoods."""
    # Buffered polygon for reading; exact polygon for the final crop.
    read_poly = f"{{buffered(plot_wkt_utm, buffer_m)}} / EPSG:{{epsg}}"
    exact_poly = f"{{plot_wkt_utm}} / EPSG:{{epsg}}"

    p = (
        # 1. Prefetch: octree pruning to the buffered region only.
        pdal.Reader.copc(copc_url, polygon=read_poly, requests=3)
        # 2. Validity: withheld first, always (§6.4).
        | pdal.Filter.expression(expression="Withheld == 0")
        # 3. Conservative outlier screen; classifies to 7 rather than removing.
        | pdal.Filter.outlier(method="statistical", mean_k=8, multiplier=3.0)
        | pdal.Filter.expression(expression="Classification != 7")
        # 4. Ground re-derivation, consistently across the corpus (§10.10).
        | pdal.Filter.smrf(scalar=1.25, slope=0.15, threshold=0.5, window=18)
        # 5. HAG on the BUFFERED cloud, so ground interpolation has context.
        | pdal.Filter.hag_delaunay()
        # 6. Now crop to the exact plot footprint (§13.6).
        | pdal.Filter.crop(polygon=exact_poly)
        # 7. Provenance statistics on exactly the delivered points.
        | pdal.Filter.stats(
            dimensions="Z,Intensity,ScanAngleRank,NumberOfReturns,HeightAboveGround",
            enumerate="Classification,PointSourceId,ReturnNumber")
    )
    n = p.execute()
    return p.arrays[0], json.loads(p.metadata)["metadata"]
```

Five design points, each of which corresponds to something from Parts I–III:

1. **Withheld filtering is the first substantive stage.** Not negotiable (§6.4).
2. **Ground classification and HAG run on the buffered cloud, and the crop to exact geometry comes after.** This is the single most common correctness bug in ALS extraction pipelines: ground classification on a 50 m patch of dense forest may find no ground at all, whereas a 250 m buffered patch usually will. **The buffer must exceed the scale at which ground interpolation operates**, which in closed canopy can be

tens of metres. 100 m is a reasonable default; on flat open terrain less will do; in dense forest on complex terrain, more.

3. **The reader gets the buffered polygon and the crop filter gets the exact one.** Prefetch versus guarantee (§13.6).
4. **The outlier filter classifies rather than removes, and a separate expression removes.** Explicit, inspectable, and it lets you count what was removed.
5. **`filters.stats` runs last**, so the statistics describe exactly the points that reach your model — which is what the provenance record should reflect.

14.6 Where PDAL ends

PDAL is an ingest and preprocessing layer. It is **not** a training-time data loader, and attempting to use it as one is a mistake worth naming explicitly.

Reasons:

- Per-sample pipeline construction and execution has overhead measured in tens of milliseconds — orders of magnitude too slow to feed a GPU.
- Network reads at training time couple throughput to a remote service’s availability and mood.
- Nondeterministic network behaviour makes exact reproducibility impossible.
- Ground classification per epoch is wasted compute; it should happen once.

The correct pattern is offline materialization: PDAL extracts, cleans, classifies, normalizes, and writes; a fast sharded format serves training. I develop the storage side in Chapter 15 and the corpus construction in Chapter 24, but the principle belongs here because it determines how you write your PDAL code: **write it as a batch job that emits durable artifacts, not as a library your Dataset class calls.**

The one legitimate exception is exploratory work, where the convenience of streaming a fresh region beats the cost. Keep that path separate from the training path so it cannot accidentally become the training path.

14.7 Alternatives worth benchmarking

lasR / pylasr — a C++ core with C++, R, and Python APIs, reading and writing LAS, LAZ, and `.copc.laz` from local and remote storage, with an explicit pipeline model and multicore file-collection processing. It is the successor to lidR’s engine and is materially faster for some workloads. **If your corpus materialization turns out to be CPU-bound, benchmark it against PDAL before scaling up.** The benchmark costs a day; the difference could be weeks of compute.

laspy — pure Python, excellent for header and VLR inspection and surgery that PDAL does not expose cleanly. The right tool for §11.6 step 1.

copc-lib — direct octree node access when you want to control *which depth* you read rather than asking for a **resolution** (§13.5).

untwine — builds COPC from LAS/LAZ; the tool for the §13.7 recommendation of converting source LAZ yourself.

Entwine — builds EPT resources from raw tiles.

WhiteboxTools — a large library of lidar and terrain operations, some with no PDAL equivalent.

CloudCompare, Potree, viewer.copc.io — visual inspection. Do not skip this. **A large fraction of ALS-ML bugs are visible in five seconds and invisible in a loss curve.** Colour by `PointSourceId`, by `ScanAngleRank`, by `Classification`, by `HeightAboveGround`, and look at a dozen tiles from each project before trusting any of them.

Further reading for Chapter 14

- [PDAL documentation](#) — the [stages reference](#) is what you will keep open.
- [PDAL workshop](#) — the best hands-on path, with exercises.
- [OpenTopography OT_3DEP_Workflows](#) — working 3DEP-specific notebooks; the closest thing to a reference implementation of what Chapter 14 describes.
- [Planetary Computer 3DEP COPC example](#) — the STAC-sign-read pattern, with the useful detail that intensity and RGB must be divided by 256 for 8-bit output.
- [lasR documentation](#) for the §14.7 alternative.

Chapter 15. Discovery, Catalogs, and Provenance Capture

15.1 WESM: the index that makes provenance possible

Work unit Extent Spatial Metadata is the single most valuable metadata resource in this book, and it is not widely used outside the 3DEP practitioner community.

WESM is a geopackage (with a CSV counterpart) published by work unit, containing information about which Lidar Point Cloud and source DEM work units have been processed and made available to the public, including lidar quality level, data acquisition dates, and links to project-level metadata, rewritten almost daily as more projects are processed and published.

Concretely, it gives you:

- **Work unit and project geometries** — so a spatial join from any coordinate returns the covering project.
- **Quality level** — the §6.2 stratification axis.
- **Acquisition date ranges** — from which phenological state is derivable (§11.4), converting a major confound from silently-inconsistent to detectable.
- **Links to source data locations** for both point clouds and DEMs.
- **Links to contractor and project metadata** for each work unit — the path to the sensor and mission information of §11.7’s **project** block.
- **A status field by product type**, indicating whether that product type meets 3DEP requirements, meets the requirements with variances, or does not meet 3DEP requirements, with a temporary “expected to meet” placeholder for products awaiting full project validation, plus a reason-code text field explaining why a publication unit was placed in a given category.

That status-and-reason field deserves special attention. It is a per-work-unit quality assessment written by USGS, in machine-readable form, including *why* a dataset failed or required a variance. As a training-corpus filter, and as a covariate in your stratified reporting, it is close to ideal — and I am not aware of any ALS-ML work that uses it.

Access:

<https://rockyweb.usgs.gov/vdelivery/Datasets/Staged/Elevation/metadata/WESM.gpkg>
<https://rockyweb.usgs.gov/vdelivery/Datasets/Staged/Elevation/metadata/WESM.csv>

Since a new WESM is written almost daily, **snapshot it and version the snapshot** with your corpus. A benchmark whose stratification depends on a file that changes daily is not reproducible unless you pin it.

15.2 RockyWeb and the project directory structure

Beneath WESM, the actual data and reports live on RockyWeb in a project-oriented hierarchy. Work unit folders under `LPC/Projects/` are named with state abbreviations and years (`CA_NoCAL_3DEP_Supp_Funding_2018_D18` and similar), and a TL element in the folder name indicates the files fall in a tribal land area.

Within a work unit, the `metadata/.../spatial_metadata/USGS/` folders hold the **tile index shapefiles** — which is how you determine which LAZ files you need without downloading everything. The same tree holds project and survey reports, USGS-determined vertical accuracy values, hydro breaklines, and control point shapefiles.

[Case study] The tile index is the practical entry point for a sampling workflow: WESM tells you which projects are eligible under your stratification, the tile index tells you which files cover your selected footprints, and only then do you fetch data. Fetching driven by the index rather than by brute-force enumeration is the difference between a manageable job and a multi-week download.

The reports are where §11.7's **project** block gets filled in beyond what WESM provides — sensor model, altitude, PRF, coverage pattern. They are PDFs with inconsistent structure. My recommendation stands from §6.7: **parse them for the projects intersecting your evaluation strata, where provenance matters most, and rely on measured statistics (§5.7) elsewhere**, recording the difference in `metadata_completeness`.

15.3 The other discovery paths

The National Map / Lidar Explorer — interactive discovery, useful for orientation and for spot-checking what WESM tells you.

AWS Open Data — the [registry entry](#) documents `s3://usgs-lidar-public` (EPT, Web Mercator, no cost) and `s3://usgs-lidar` (full-density LAZ, native CRS, requester-pays). Boundary files listing available resources are published alongside.

OpenTopography — a curated portal with its own API, some datasets not easily found elsewhere, and the [3DEP workflow notebooks](#).

Planetary Computer STAC — `pystac-client` search over the `3dep-lidar-copc` collection, with `planetary-computer` for token signing. Convenient; see §13.7 for the caveats.

NEON — the [data portal](#) and API, plus the `neonutilities` package (Python and R). NEON’s product IDs are stable and its metadata bundles include the QA reports of §7.4, which are far easier to parse than 3DEP’s project reports. For the evaluation side, this is where DP1.30003.001 (discrete point cloud), the DP3 rasters, and the TOS organismal and vegetation products all come from, joinable on `plotID` and `siteID`.

15.4 Turning the schema into a database

§11.7 gave a per-sample JSON record. At corpus scale that becomes a table, and the choice of storage matters more than it might seem.

My recommendation: GeoParquet, queried with DuckDB.

The argument: your manifest is a few million rows with geometry, dozens of columns, and heavy analytical querying (“give me all QL2 samples in evergreen forest in ecoregions 21 and 6 with pulse density above 4 and DTM disagreement below 0.3, excluding NEON buffers”). That is a columnar-analytics workload with a spatial predicate. GeoParquet plus DuckDB’s spatial extension handles it in-process, with no server, and the files are portable artifacts you can publish alongside the dataset. A relational database is heavier than you need; a pile of JSON files is unqueryable at this scale.

Practical structure:

- **projects.parquet** — one row per 3DEP work unit, from WESM plus parsed reports. A few thousand rows.
- **samples.parquet** — one row per training or evaluation sample, with a foreign key to project, the full `realized/processing/context/splits` blocks flattened into columns, and a geometry column.
- **labels.parquet** — one row per label instance, joined to samples by geometry and by acceptable temporal offset.
- **Partition samples.parquet by ecoregion or by a spatial block** so that stratified reads touch only relevant files.

Three design rules I would enforce:

1. **The manifest is the source of truth for splits.** Not a directory layout, not a filename convention, not a separate text file. If `splits.eval_holdout` says a sample is in the test set, no code path can put it in training.
2. **Every derived artifact records the manifest version it was built from.** When you regenerate the corpus, the old models remain interpretable.
3. **The manifest is versioned and published.** It is a first-class dataset artifact, arguably more valuable than the point clouds — anyone can re-download 3DEP, but nobody else has computed the provenance.

15.5 Storage for training

The other half of materialization. Requirements: sequential reads (random access to millions of small files kills throughput), reasonable compression, shard-level shuffling, and a format that survives being copied to whatever compute you end up on.

WebDataset — tar shards of samples, streamed sequentially, with shuffling at shard and buffer level. Mature, simple, works from local disk or object storage. The pragmatic default.

litdata — a newer alternative with similar goals and better random-access support.

Zarr — the right answer if you go voxel-grid rather than point-based, since it is chunked N-dimensional array storage with excellent partial-read support.

Parquet for points — viable, and it pairs naturally with the manifest. Column-selective reads are a real benefit when you store more dimensions than you train on.

Whatever you choose: **store the per-sample provenance record inside the shard alongside the points**, not only in the central manifest. Denormalization here is worth it — it means a shard is self-describing, survives being copied around, and cannot drift out of sync with the manifest.

15.6 The capture pipeline, end to end

Assembling Part IV into an ordered procedure:

1. **Snapshot WESM.** Version it. Record the snapshot date.
2. **Build the project table.** Spatial join WESM against ecoregions and land cover; parse project reports for the subset where you need full provenance; record `meta-data_completeness` honestly.
3. **Stratify and select.** Sample work units by ecoregion $\times$ QL $\times$ acquisition year $\times$ phenological state, honouring the WESM status field. (Chapter 24 develops this properly; pacasam is the reference implementation to study.)
4. **Resolve tiles.** Use per-project tile indexes to identify the specific files covering selected footprints.
5. **Extract.** PDAL pipelines per §14.5: buffered read, withheld filter, outlier screen, consistent ground, HAG, exact crop, stats.
6. **Compute provenance.** From the pipeline metadata, the header parse, and the spatial joins. One record per sample.
7. **Validate.** MSHR rasters for visual QC; distribution checks on density, returns-per-pulse, scan angle, and DTM disagreement; flag outliers for inspection rather than silently accepting.
8. **Write.** Sharded storage plus manifest, with provenance denormalized into both.
9. **Publish the manifest** as a versioned artifact.

Step 7 is the one most likely to be skipped and most likely to save you. **Look at the data.** Render a few hundred MSHRs to a contact sheet and scan it; the failures — flagging errors, ghost surfaces, dropout patches, seam artifacts — are visually obvious and statistically subtle.

15.7 What Part IV bought you

Closing the first half of the book with an accounting.

You can now, for any coordinate in the United States: identify which acquisition project covers it and with what specification, sensor, and date; assess whether USGS considered that project compliant and why; fetch exactly the points in an arbitrary polygon over HTTP without downloading a tile; apply a consistent, documented cleaning and ground-classification procedure; measure what you actually got as against what the specification promised; and record all of it in a queryable, publishable artifact attached to every sample.

That capability is the precondition for everything in the second half. Part V asks what features and tasks to compute on these samples; Part VI asks what architectures to run on them; Part VII asks how to assemble it into a benchmark. All of it presumes you know what your data is — which, as Parts I–IV have argued at some length, is not the same as having downloaded it.

Further reading for Chapter 15

- [3DEP Spatial Metadata](#) — the WESM documentation, including the status and reason-code fields.
- [Elevation Specification Resources](#) — includes the **Lidar Error Dictionary** and the **Spatial Metadata Data Dictionary**. The error dictionary standardizes the language used to describe errors found during USGS validation, which makes the WESM reason codes interpretable; read it before parsing that field.
- [About 3DEP Products & Services](#) for the product taxonomy.
- *Good ol' Rockyweb* — an informal but genuinely useful guide to navigating the RockyWeb directory structure, including the tile index and the TL naming convention.
- [AWS Open Data: USGS 3DEP](#).
- [pacasam](#) — again, because it is the reference implementation of steps 3 and 9.
- [DuckDB spatial extension](#) and the [GeoParquet specification](#) for §15.4.

Bridge to Part V

The first half of this book was about knowing what your data is. The second half is about what to do with it.

Part V turns to analysis. Chapter 16 develops the engineered-feature vocabulary — height distribution, cover, vertical arrangement, surface complexity, geometric neighbourhood features — organized by what they measure and traced to where they came from, because these are the baselines your deep models must beat and the intuitions your architectures should encode. Chapter 17 covers forest-structure tasks and their evaluation. Chapters 18 and 19 turn to the biodiversity evaluation sources and the protocol — probing, splitting, the ablation ladder, and multi-scale spatial support — which is where the benchmark's credibility will ultimately be decided.

Part V — Features, Tasks, and Evaluation

Airborne Laser Scanning for Machine Learning in Ecology

Orientation to Part V

The first half of this book was about knowing what the data is. This Part is about deciding what to ask of it.

Four chapters, and the order is deliberate:

- **Chapter 16** develops the engineered-feature vocabulary. Not as history, but because these features are simultaneously the baseline your models must beat, the encoding of thirty years of ecological intuition about what lidar measures, and — read carefully — a specification of the invariances a good learned representation should have.
- **Chapter 17** covers forest-structure tasks: area-based inventory, canopy height, tree detection, species classification, segmentation. These are the tasks with established literature, established metrics, and established baselines, which makes them the place where your model can be judged against something.
- **Chapter 18** covers the biodiversity evaluation sources — the ones in your proposal plus the ones I would add — with an honest account of what each can and cannot support.
- **Chapter 19** is the protocol chapter: probing, pooling, splitting, and the ablation ladder. **This is where the benchmark’s credibility is decided**, and it is the chapter I would write first if I were writing them out of order.

A framing worth stating up front, because it shapes everything in Chapters 18–19. Reading your task list together, the benchmark you are describing is not a forest-structure benchmark with biodiversity extras. It is a **representation-quality benchmark**: the question is whether a frozen ALS embedding, dropped into a shallow probe or used as a covariate in an ecological model, carries information about biodiversity that cheaper data sources do not. That is a different object from PureForest or FRACTAL, and a more interesting one. It also means the protocol matters as much as the data, because frozen-embedding probing has well-known failure modes — it undersells transferable models, it is exquisitely sensitive to how point-level features are pooled into one vector, and a two-layer MLP has ample capacity to launder spatial autocorrelation into apparent skill.

Chapter 16. The Engineered-Feature Vocabulary

16.1 Why this chapter is not optional

There is a temptation, arriving from deep learning, to treat handcrafted lidar metrics as the thing being replaced and therefore not worth learning. Three reasons to resist it.

They are the baseline. In area-based forest inventory, a random forest on thirty height and cover metrics is not a weak baseline; it is the operational state of the art in national forest inventory programs across several countries. A foundation model that does not beat it has not earned its compute.

They encode the physics. Every feature in this chapter exists because someone established that it relates to a forest attribute given the sensing model of Part I. Height percentiles are

robust to density because they are order statistics. Cover metrics are computed as return ratios because that is what gap probability means for a discrete-return sensor. When you design an architecture, you are implicitly deciding which of these relationships it can express.

They are diagnostic. When a learned model fails, comparing its errors against a metric-based model tells you whether the information was absent from the data or absent from the representation. That is the single most useful debugging signal available to you.

I organize by *what is measured* rather than by software package, and flag throughout which features are fragile under the heterogeneity of Part I.

16.2 Height distribution metrics

The workhorses. Computed on height above ground, over some spatial support, usually excluding returns below a threshold (2 m conventionally, per §3.5's blind zone).

Percentiles $h_{10}, h_{25}, h_{50}, h_{75}, h_{90}, h_{95}, h_{99}$ — the single most-used family. h_{95} and h_{99} are canopy-height proxies more robust than the maximum; h_{25} and h_{50} describe where the bulk of the material sits.

Moments — mean, standard deviation, coefficient of variation, skewness, kurtosis. Skewness distinguishes a top-heavy closed canopy (negative) from an open canopy with scattered emergents (positive).

Maximum height — direct, interpretable, and **the most fragile metric in this chapter**. It is an extreme order statistic, so it depends on pulse density (more shots means a better chance of hitting the apex), on noise filtering (§10.6 truncates sparse canopy tops), and on withheld-flag handling (§6.4 — an unfiltered bird gives you a 300 m tree). Use h_{99} or h_{95} unless you specifically need the maximum, and if you need it, check the outlier pipeline first.

L-moments — linear combinations of order statistics; more robust to outliers than conventional moments and less commonly used than they should be.

Robustness ranking, from thinning experiments in the literature: median and mid-percentiles are most robust to density variation; upper percentiles degrade next; maximum degrades first. Broadly, area-based height metrics hold up surprisingly well down to roughly 1–2 pulses/m², which is why QL2 supports operational forest inventory at all.

16.3 Cover, density, and gap metrics

Canopy cover — the fraction of returns above a height threshold. The subtlety is *which* returns:

- **First-return cover**: first returns above threshold / all first returns. This is the closest analogue to a vertical-projection cover fraction, because each pulse contributes once.
- **All-return cover**: all returns above threshold / all returns. Contaminated by return-count variation, hence by sensor and footprint (§1.5, §3.6).

Use first-return cover. It is the version that means the same thing across sensors, and the difference is not small in dense canopy.

Return-proportion metrics — proportions of first, intermediate, last, single, and multiple returns. These are ecologically suggestive (many intermediate returns implies a deep multi-layered canopy) and **among the least transferable features in this chapter**, because they are direct functions of the return-count cap, footprint size, and discretization thresholds (§3.6, §4.4). If you use them, expect them to carry sensor identity.

Gap fraction and the Beer–Lambert inversion. The physical model treats the canopy as a turbid medium: the probability a pulse penetrates to depth z without interception is

$$P_{\text{gap}}(z) = \exp \left(-k \int_z^h \text{LAD}(z') dz' \right)$$

with k an extinction coefficient depending on leaf angle distribution and view angle. Inverting this gives a **leaf area density profile** from the vertical distribution of returns — the MacArthur–Horn approach, adapted to discrete-return lidar. Implementations: `leafR`, `canopyLazR`, and the profile functions in `PyForestScan`.

Three cautions on LAD inversion, all following from Part I. Discrete returns are a *thresholded count* approximation to intercepted energy, not a measurement of it, so the inversion inherits every discretization inconsistency. The return-count cap truncates the profile interior, biasing deep-canopy LAD downward. And k is usually assumed constant when it depends on leaf angle distribution, i.e. on species — so LAD inversion is species-dependent in a way that is rarely acknowledged. Treat derived LAI as a structured proxy rather than a measurement.

Vertical Complexity Index and **foliage height diversity** — Shannon entropy over height-bin return proportions. FHD is a classic in the bird-habitat literature specifically (MacArthur & MacArthur 1961 predates lidar by decades but the concept transfers directly), which makes it a natural feature for your avian tasks and a good sanity check: if a learned embedding cannot beat FHD for bird occupancy, something is wrong.

16.4 Vertical arrangement

Height-bin proportions — the fraction of returns in each of a set of strata (0–2, 2–5, 5–10, 10–20, 20+ m, or relative bins as fractions of canopy height). A simple, interpretable vertical profile descriptor, and the natural low-dimensional summary to compare a learned representation against.

Canopy relief ratio — $(\bar{h} - h_{\min}) / (h_{\max} - h_{\min})$. Where the mean sits within the height range; a shape descriptor invariant to absolute height.

Rumple index — the ratio of canopy surface area (from a triangulated CHM) to ground area. A roughness measure sensitive to crown-scale structure. Note that it is computed on a *raster* CHM, so it inherits the CHM’s resolution and interpolation choices, and it is sensitive to pit-filling.

Vertical Complexity Index, entropy of the height distribution, number of detected strata from profile peak-finding.

KDE-based metrics — fit a kernel density estimate to the height distribution and extract the number, position, and prominence of modes. Implemented in `lidRmetrics`. These are a more principled way to answer “how many canopy layers are there” than fixed binning, and they are underused.

16.5 Neighbourhood geometric features

Distinct from the above: these are computed *per point* from a local neighbourhood, not per area. They are the classic input to point-wise semantic segmentation and the thing point-based networks are implicitly meant to replace.

From the eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3$ of the local covariance matrix, normalized so $\sum \lambda_i = 1$:

Feature	Definition	High when
Linearity	$(\lambda_1 - \lambda_2)/\lambda_1$	Points lie on a line — branches, wires, poles
Planarity	$(\lambda_2 - \lambda_3)/\lambda_1$	Points lie on a plane — ground, roofs, water
Sphericity / scattering	λ_3/λ_1	Isotropic — foliage, noise
Omnivariance	$(\lambda_1 \lambda_2 \lambda_3)^{1/3}$	Volumetric spread
Anisotropy	$(\lambda_1 - \lambda_3)/\lambda_1$	Directional structure
Eigenentropy	$-\sum \lambda_i \ln \lambda_i$	Maximal disorder
Surface variation	$\lambda_3/\sum \lambda_i$	Local curvature
Verticality	$1 - n_z $ from the normal	Vertical surfaces — stems, walls

All available via `filters.covariancefeatures` (§14.3).

The scale problem, and why it matters more in ALS than elsewhere. These features depend entirely on the neighbourhood definition — fixed k , fixed radius, or adaptive. Fixed- k neighbourhoods have *physically different sizes* in dense versus sparse regions, which under ALS’s variable density (§2.3, §4.4) means the same feature means different things in different parts of one tile. Fixed-radius neighbourhoods have constant physical scale but variable point count, which changes the estimator’s variance.

`filters.optimalneighborhood` selects a per-point scale by minimizing eigenentropy, which is the principled answer. **I would make fixed- k versus fixed-radius versus optimal-scale an explicit ablation in your feature baseline**, because the result tells you something real about how much of a network’s job is scale selection — which connects directly to the hierarchical-sampling discussion of Chapter 21.

16.6 Intensity and multi-return metrics

Mean, standard deviation, percentiles, and skewness of intensity, computed overall or by return type or height stratum.

Everything in §3.7 and §10.8 applies: exclude by default, ablate explicitly, quantile-normalize per project if used, and never in a task where labels are project-correlated. I mention them here only because they appear in every standard metric set and you will need to decide about them deliberately rather than by inheriting a default.

Where Extra Bytes carry calibrated reflectance or echo width (§12.6), the situation is materially better and those features are worth using — but they will be present for a minority of projects, which creates a missing-data pattern correlated with vendor.

16.7 Terrain and context features

Slope, aspect, curvature (profile, plan, mean), topographic position index, topographic wetness index, heat load index, and elevation — all derived from the DTM rather than the point cloud.

Two notes. First, these are *strong* predictors for most ecological tasks — often stronger than canopy structure — which makes them essential in the ablation ladder (§19.5) as the thing ALS structure must add to. Second, per §9.7, elevation carries a continental datum artifact; provide it coarsened and deliberately, not as raw Z .

16.8 What the vocabulary implies for architecture

Reading Chapter 16 as a specification rather than a list, four properties a good learned ALS representation should have:

1. **Permutation invariance** — every feature here is a function of the point *set*. This is table stakes, and every architecture in Chapter 20 satisfies it.
2. **Density robustness** — order statistics and first-return ratios are the features that survive density variation. A representation that is not density-robust will underperform them out of domain, which is exactly what leave-one-sensor-out will reveal.
3. **Multi-scale sensitivity** — the vocabulary spans per-point neighbourhoods (centimetres to metres) through plot-level distributions (tens of metres). A single-scale representation cannot span it.
4. **Gravity alignment and ground relativity** — nearly everything is computed on height above ground. A representation that treats z as just another coordinate is discarding the single most informative structural axis. But see §22 for the flip side: making HAG too available invites shortcut learning.

[Case study] A concrete recommendation for the benchmark. Compute the full metric set once, for every sample, and store it in the manifest alongside the provenance record. It costs one pass, it gives you the baseline for free, it lets you check learned embeddings against interpretable quantities (e.g. how much variance in the embedding is explained by h_{95} and cover?), and it lets you report *which* engineered features a learned representation fails to recover — which is a far more informative negative result than a single deficit in R^2 .

Further reading for Chapter 16

- [lidR book, Chapter 9: Derived metrics](#) — the best written treatment of the ABA workflow anywhere, and the reference implementation of `stdmetrics`.
- [lidRmetrics](#) — the extended set, with citations to the originating papers for each metric. Use this as your bibliography for §16.2–16.4.
- [PyForestScan](#) for the Python route; [leafR](#) and [canopyLazR](#) for LAD profiles specifically.
- Weinmann et al. (2015) on optimal neighbourhood selection and geometric feature relevance — the source for §16.5’s scale argument.

- MacArthur & MacArthur (1961) on foliage height diversity, for the historical root of §16.3’s connection to bird habitat.

Chapter 17. Forest-Structure Tasks

17.1 Area-based inventory

The task. Predict a stand attribute — aboveground biomass, basal area, stem volume, Lorey’s mean height, stem density — from lidar metrics over a plot-sized support.

The paradigm. The area-based approach computes metrics over a support congruent with a conventional field plot, typically 400–900 m², fits a model from metrics to field-measured attributes, and applies it to a wall-to-wall grid. It is the basis of enhanced forest inventory in Nordic countries and increasingly elsewhere.

Metrics. RMSE and relative RMSE (as a percentage of the mean), bias, R^2 . Report all three: a model can have excellent R^2 and severe bias, and for carbon accounting the bias is what matters.

Why this is the hardest task for a foundation model to win. The relationship between height metrics and biomass is close to deterministic given species composition, because biomass allometry is fundamentally a function of height and diameter, and lidar sees height. There is not much headroom above a well-specified parametric or random-forest model. Where a learned representation *should* help is in the residual — species composition, stand structure, and site effects that the metrics do not capture. **Design the comparison so that headroom is visible:** report performance stratified by forest type and by structural complexity, not just pooled.

The label is not ground truth. Field “biomass” is itself a model output: diameter and height measurements passed through allometric equations with their own error, which is often 10–20% at plot scale and is *systematically* structured by species and size class. Your RMSE has an allometric floor. Say so.

17.2 Canopy height

The task. Predict canopy height — as a raster, or per-plot as a scalar.

The trap. A CHM is a lidar product. Training a model to predict a CHM from the point cloud that produced it is a self-consistency exercise, not an accuracy assessment. It is a legitimate pretext or auxiliary task; it is not an evaluation.

Honest versions of the task: predict *field-measured* tree height at mapped stems; predict canopy height from a *different* sensor’s cloud over the same footprint (which is your sensor-gap design); or predict height from a degraded input (simulated sparse acquisition) with the full-density-derived CHM as target, which measures density robustness directly and is a genuinely useful capability.

The DTM coupling. Canopy height error decomposes into surface error and terrain error, and §10.4 established that terrain error is worst under dense canopy on slopes — i.e. correlated

with the target. The `dtm_disagreement_by_slope_class` field of §11.7 exists to let you make this decomposition; use it.

17.3 Individual tree detection and crown delineation

The task. Locate individual trees, and optionally delineate crowns, from the point cloud or CHM.

Classical methods: local maxima filtering on a smoothed CHM with a variable window; watershed segmentation; point-cloud clustering approaches (Li et al. 2012, Dalponte & Coomes 2016); graph and region-growing methods.

Metrics: detection recall, precision, F1 at a matching tolerance; and for delineation, IoU or crown-area agreement. **Report by size class.** Overall F1 is dominated by dominant trees; suppressed and understory trees are where every method fails and where the ecological information often is.

Benchmarks. [NeonTreeEvaluation](#) — 30,975 annotations across 22 NEON sites with co-registered RGB, lidar, and hyperspectral. Note its own caveat that point-cloud annotations were made by draping 2D boxes over the cloud and excluding sub-2 m non-ground points, so they are less thoroughly cleaned than the RGB boxes. [FOR-instance](#) for ULS-density annotated trees — a transfer target, not in-domain ALS (§8.4).

The density constraint. Individual tree detection degrades with density far earlier than area-based metrics do. At QL2’s 2 pulses/m², reliable detection is limited to dominant trees in open-to-moderate canopy. **[Case study]** This is a genuine limit on what your 3DEP-trained model can be asked to do, and it should be stated in the task definition rather than discovered in the results.

17.4 Species classification

The task. Predict tree species or species group, per tree or per patch.

The reference ALS benchmark is [PureForest](#): 135,569 patches of 50×50 m, 13 semantic classes hierarchically grouping 18 species, with a reference split and paired aerial imagery. Two design decisions to understand: patches are **monospecific** and carry **one label each**, which makes the task tractable and also makes it easier than the real problem of mixed stands. And lidar and imagery may be separated by up to three years.

The most relevant empirical result for your architecture choices comes from the Finnish multispectral ALS species work: point-based deep learning, specifically a point transformer, beat both shallow random forests and image-based deep learning on high-density multispectral clouds — 87.9% overall accuracy and 74.5% macro at 1,065 training segments, rising to 92.0% and 85.1% at 5,000, versus 83.2% and 61.3% for random forest. **The macro-average gap is the story.** Rare species are where everything falls apart, and the difference between overall and macro accuracy is the difference between a model that has learned species and one that has learned base rates.

Leaf-off is a distinct problem. Per §5.5, a leaf-off collection over mixed forest means the deciduous fraction is measured without foliage. Species discrimination on leaf-off data is partly

the task of detecting *which trees still have leaves*, which is a real signal but a different one. Stratify results by phenological state.

17.5 Semantic and instance segmentation

Semantic segmentation — per-point class labels. The dominant task in the point-cloud literature, and the one 3DEP cannot support: vegetation classes are optional and inconsistent (§10.7, §12.3). Labels must come from a program that requires them (IGN Lidar HD, FRACTAL) or from your own annotation.

Instance segmentation — per-tree instances. [SegmentAnyTree](#) and ForAINet are the platform-agnostic references.

[Case study] My recommendation: use FRACTAL and PureForest as *external* semantic benchmarks to compare your representation against published numbers, and do not construct a 3DEP semantic segmentation task. Annotating 3DEP at the scale needed would be a project in itself, and the resulting labels would inherit whichever ground and vegetation classification you chose, making the task partly circular.

17.6 Change and structural products

Repeat-collection change detection. NEON’s annual revisits and 3DEP’s regions of repeat coverage support growth, mortality, disturbance, and harvest detection. Cautions: co-registration error between epochs sets a floor on detectable change; wind-driven crown motion adds irreducible noise (§5.5); and the two epochs may differ in sensor, which means an apparent change signal may be a sensor signal. **Repeat coverage by a *different* sensor is a confound for change detection and simultaneously a gift for domain-invariance training (§4.5, Strategy 3).** Same data, two uses, opposite signs.

Structural diversity as dense weak labels. [NEON-SD](#) provides 30 m structural-diversity metrics aligned to the Landsat grid for 211 site-years across 45 NEON sites, 2013–2022. As a pretext or auxiliary target this is attractive: dense, aligned to a standard grid, and derived from the same AOP data. As an *evaluation* target it is circular, since it is computed from lidar.

17.7 What forest tasks are for, in your benchmark

They are the **calibration layer**. They have established baselines, so they tell you whether your representation is competent before you ask it harder questions. If your model cannot match a random forest on ABA biomass, the biodiversity results are uninterpretable — you will not know whether ALS lacks the signal or your model lacks the capacity.

Run them first, report them plainly, and treat a failure here as a blocker rather than a footnote.

Chapter 18. Evaluation Data: Ground-Truth Sources

18.1 A two-tier framing

The sources available to you differ by two orders of magnitude in both rigour and sample size, and the difference is systematic: rigorous protocols are expensive, so they are small; large

datasets are cheap, so they are noisy. Rather than pretending this away, make it the structure.

Tier 1 — gold standard, small n . Known surveyed coordinates, documented protocol, modelled detectability, contemporaneous lidar. NEON’s organismal and vegetation products; NEON vegetation structure; FIA with true coordinates if obtainable.

Tier 2 — noisy, large n , broad coverage. Citizen science and opportunistically aggregated data with coordinate uncertainty comparable to or larger than the ecological support, uncontrolled effort, and taxonomic and spatial bias. eBird, iNaturalist, GBIF-derived tree occurrences, Snapshot USA.

Why both. Tier 1 answers *does the representation carry signal at all?* Tier 2 answers *does it survive contact with the data ecologists actually have?* Disagreement between tiers is informative rather than embarrassing — it localizes the failure to either the representation or the label quality.

Four properties determine what a source can support, and I give them for each source below: **coordinate uncertainty** relative to the ecological support; **temporal alignment** with available lidar; **detection structure** (does the data support presence-absence, and is effort recorded?); and **CONUS $\cap$ 3DEP survivability** after filtering.

18.2 eBird — avian occurrence

What it is. Checklist-structured bird observations at enormous scale, available via GBIF and via the eBird Basic Dataset. Complete checklists — where the observer reports all species detected and identified — support inferred absence.

A correction to the proposal’s framing. “Fit occupancy models on complete checklists” is not quite right as stated. Occupancy models require replicated visits to defined sites; eBird provides effort covariates but not site replication unless you construct it by aggregating checklists into spatio-temporal cells. The two defensible options are **encounter-rate modelling with effort covariates**, or **explicit construction of replicate visits per cell** for a genuine occupancy formulation. Pick one and name it. The [eBird Best Practices](#) guide (Strimas-Mackey et al. 2023, version 2.0) covers both, and its central empirical finding is directly relevant: the largest gains in model performance came from using complete checklists and from including covariates describing variation in effort and detectability per checklist.

The shortcut risk, which is severe. eBird’s spatial bias toward roads, trails, parking areas, and accessible edges correlates strongly with ALS-visible structure. A model can score well by learning “near a road” rather than “suitable habitat.” Two mitigations, both of which I would treat as required:

1. **Restrict the ALS-matched subset to stationary counts of short duration.** For a stationary count the reported coordinate approximately *is* the survey location. Traveling counts report distance but not path geometry, so at 50–100 m patch scale you do not know where the observations happened. This filter is costly in sample size and non-negotiable for spatial precision.
2. **Include a road-distance-only baseline** in the ablation ladder, so accessibility-driven skill is visible.

Standardized prediction. The best-practices approach makes predictions for a *standard checklist* — for example a 1 km, 1 hour traveling count at the species’ peak detection time — which holds effort constant across the prediction surface. Adopt this convention so that your ALS-with and ALS-without comparisons differ only in habitat covariates.

Verdict. Strong Tier 2 source. Large n , national coverage, genuine absence information, mature methodology. Coordinate precision is the binding constraint, and the road bias is the thing most likely to produce a spurious positive result.

18.3 iNaturalist — insect occurrence

Three compounding problems, which I would state plainly in the task description rather than downplay.

Presence-only. Occupancy is not identifiable without absences. What you can fit is a Poisson point-process or MaxEnt-style model with a target-group background — a legitimate and well-understood approach, but it is not an occupancy model and the task description should not call it one.

Coordinate accuracy. Phone GPS under closed canopy is routinely accurate to 10–100 m — the same order as your patch size. For a taxon whose habitat association operates at metre scale, this is close to fatal.

Obscuring. iNaturalist obscures coordinates for sensitive and threatened taxa, and downstream datasets preserve the obscuring.

Family- or genus-level aggregation helps with taxonomic noise but not with spatial noise, and insects in particular have habitat associations at scales finer than the coordinate error.

Verdict. Keep, but reframe. This is a **deliberate stress test**: how much does ALS representation quality survive realistic citizen-science noise? That is a genuinely useful question and an honest framing. Pair it with a rigorous counterpart — NEON ground beetles (§18.7) — so you can distinguish “ALS does not predict insects” from “this data cannot detect whether ALS predicts insects.”

18.4 Snapshot USA and camera traps — mammal occurrence

This is your strongest task, and stronger than the proposal describes. The Haucke et al. work has moved on; the current version (early 2026) is titled *Seeing Above and Below the Canopy: Modeling and Interpreting Species Occupancy with Multimodal Habitat Representations*. The design fits single-species single-season occupancy models on large-scale Wildlife Insights data and evaluates on publicly available Snapshot USA data, finding that 12 of 16 focal species were modelled most accurately using multi-modal deep features, and that multi-modal features substantially enhance predictive power compared to traditional environmental variables alone.

Why this is the best fit in your set:

- **A published baseline with a strict protocol exists**, so you are adding a modality to an established comparison rather than inventing a task.

- **Their modalities are satellite imagery plus ground-level camera-trap imagery.** ALS is the obvious missing third, and slotting it in is a clean, interpretable ablation.
- **They ship `Biolith`,** a Python package for Bayesian ecological modelling — your launch-point rather than something to rebuild.
- **They explicitly guard against shortcut learning** (camera hardware metadata leaking region identity), which is the same class of problem as your sensor identity leaking geography. Their mitigations are worth reading as prior art for §19.

Caveats. Camera trap detection zones are ~10 m while home ranges are kilometres, so the support mismatch is extreme and multi-scale pooling (§19.4) is essential rather than optional. Camera placement is also non-random — often on trails and travel corridors, which is the road-proximity problem again in a different guise.

18.5 Tree identification — a correction

The proposal’s **GlobalGeoTree** description is **inaccurate and needs fixing before it reaches a reviewer**. **GlobalGeoTree** contains **no lidar**. It is 6.3 million geolocated tree occurrences spanning 275 families, 2,734 genera and 21,001 species, each paired with **Sentinel-2 image time series** and 27 auxiliary environmental variables (bioclimatic, geographic, soil), filtered to GBIF points inside the JRC 2020 forest mask with 2020 as the target year, and partitioned into **GlobalGeoTree-6M** for pretraining and **GlobalGeoTree-10kEval** for zero- and few-shot benchmarking. The “aerial lidar, 13 semantic categories” description belongs to **PureForest** — 13 classes grouping 18 species — so the two have crossed somewhere in drafting.

Two clean repairs, and I would do both:

1. **Use `PureForest` as the ALS tree-ID task.** It is actually ALS, it has a reference split, and it lets you compare against published numbers (§17.4).
2. **Use `GlobalGeoTree`’s GBIF occurrences as point labels intersected with `3DEP`.** Nobody has done this, and it would be a genuine contribution: a CONUS species-ID task at real ALS resolution. But filter hard on **`coordinateUncertaintyInMeters`** — GBIF tree occurrences mix herbarium records, forestry plots, and phone-GPS observations with uncertainty from 1 m to kilometres, and **GlobalGeoTree** deliberately preserves iNaturalist’s coordinate obscuring for sensitive and threatened species. Expect to discard most CONUS records. Run the survivability count before committing.

18.6 `sPlotOpen` — vegetation functional traits

What it is. 95,104 vegetation plots from 105 databases across 114 countries, with plot-level metadata (size, biome, elevation, slope, aspect, vegetation type, naturalness, layer cover) plus **community-weighted means and variances of 18 plant functional traits** from the TRY database. Organized as three partially overlapping environmentally-balanced resampled datasets of about 50,000 plots each, intended as replicates.

Four filters, and the survivability problem. Your proposed subset is: within CONUS $\cap$ 3DEP, high positional accuracy, 10–50 m diameter. Add a fourth that the proposal omits: **temporal overlap**. `sPlotOpen` spans **1888–2015**. A large fraction of plots predate any lidar acquisition entirely, and 3DEP began in earnest around 2012. The intersection of “recorded after 2012” with “CONUS” with “precise coordinates” with “10–50 m” is going to be small.

Do this count in week one. If it yields fewer than a few hundred plots, the functional-trait task is a case study, not a benchmark task, and you need a fallback — NEON plant presence and percent cover (DP1.10058.001) is the natural one.

A second caution. Plot size ranges from 0.01 to 40,000 m² across the full dataset, so your 10–50 m filter is doing heavy lifting, and even within it a 10 m plot and a 50 m plot are different supports for the same nominal quantity. Include plot size as a covariate.

Verdict. Keep, but verify feasibility immediately and prepare the NEON fallback. The task itself is attractive — functional traits are mechanistically linked to canopy structure, so this is one of the places ALS *should* outperform optical data.

18.7 NEON’s organismal suite — the rigorous tier you are missing

This is the addition I would most like you to take from this chapter.

NEON runs bird, mammal, and beetle sampling **at known surveyed plot coordinates with contemporaneous AOP lidar over the same footprint**, which simultaneously fixes the coordinate-uncertainty and temporal-mismatch problems that undermine eBird and iNaturalist:

- **Breeding landbird point counts (DP1.10003.001)** — adapted from the IMBCR spatially-balanced protocol, recording all individuals during a 6-minute count with distances from observer to bird and detection method. That is exactly the metadata needed to model detectability properly, which eBird approximates with effort covariates.
- **Small mammal box trapping (DP1.10072.001)** — the controlled analogue of the Snapshot USA task.
- **Ground beetles from pitfall traps (DP1.10022.001)** — sampled across 10 distributed plots per site. Your rigorous insect task, with the effort and coordinates iNaturalist lacks.
- **Plant presence and percent cover (DP1.10058.001)** — the sPlotOpen fallback.
- **Woody plant vegetation structure (DP1.10098.001)** — mapped stems with DBH, height, crown dimensions, species, and health status.

And crucially, **plot-scale colocation across protocols is identifiable directly via plotID**, so cross-taxon joins are cheap. The plot geometry (§7.6) — 40 m × 40 m with a 20 m × 20 m core — sits conveniently close to both the classical ABA support and PureForest’s 50 m patches.

The known biases apply (§7.7) and must be carried into the task descriptions: small mammal sampling occurs within 300 m of roads; mosquito sampling within 45 m; plots are sited away from infrastructure and edges; and site selection favours intact, accessible, securely tenured land. **NEON is a rigorous sample of studyable ecosystems, not a random sample of American ones.**

18.8 Other sources worth a paragraph

FIA. The obvious national reference, and nearly unusable as published: most plot coordinates are randomly relocated within half a mile, the remainder up to a mile, plus location swapping

between private-land plots in up to 20% of cases. Plot-level ALS matching against fuzzed coordinates is meaningless. True coordinates require a Memorandum of Cooperation — TreeMap did exactly this — and that has long lead time. **If you want FIA, start the conversation in month one, not month nine.** Note also a live complication: FIA’s spatial data services have reported being unable to process confidential-information requests due to federal workforce reductions. Verify current status before planning around it.

Breeding Bird Survey. Long time series, national, but 40 km routes — a support mismatch of three orders of magnitude relative to your patches. Usable only at aggregated scale.

NABat. Bats are structure-sensitive taxa with a plausible mechanistic link to ALS-visible clutter and canopy geometry. An underexplored and well-matched fit.

GEDI L2A/L4A and ICESat-2 ATL08. Not ground truth, but national-to-global structural cross-references and plausible auxiliary pretraining targets, especially where no plot network reaches. Cross-sensor and coarse; treat as weak labels.

LANDFIRE EVT, NLCD. The land-cover rung of the ablation ladder.

TreeMap. Imputed FIA attributes at 30 m — with the caveat that it is *imputed* from LANDFIRE covariates, so a model can score well by reproducing those covariates rather than by measuring forest.

Open Forest Observatory, and the Tolan et al. / Meta-WRI global canopy-height maps — the latter as a comparison target, not ground truth.

18.9 The source summary table

Source	Tier	Coord. uncertainty	Temporal fit	Detection structure	CONUS $\cap$ 3DEP survivability
NEON birds (DP1.10003)	1	Surveyed, <5 m	Contemporaneous AOP	Distance + method recorded	~45 sites, high
NEON small mammals	1	Surveyed	Contemporaneous	Capture- recapture	~45 sites; ≤ 300 m from roads
NEON beetles	1	Surveyed	Contemporaneous	Effort- standardized traps	10 plots/site
NEON veg. structure	1	Mapped stems	Contemporaneous	Census	High
FIA	1 (if MoC)	Fuzzed unless MoC	Variable	Census	National, gated
Snapshot USA	2	Camera location known	Variable	Detection histories	Good; trail bias
eBird	2	10 m–km (stationary vs. traveling)	Filterable by date	Complete checklists	Very high; road bias
iNaturalist	2	10–100 m + obscuring	Filterable	Presence-only	High n , low precision

Source	Tier	Coord. uncertainty	Temporal fit	Detection structure	CONUS $\cap$ 3DEP survivability
GBIF trees / GlobalGeoTree sPlotOpen	2 2	1 m–km, heterogeneous Variable	2020 target year 1888–2015	Presence-only Presence- absence in plot	Unknown; count first Verify — likely low
PureForest	External	Patch-level	≤ 3 yr lidar–imagery offset	Monospecific patches	France, not CONUS

18.10 Task construction principles

Applying across all sources:

Match support to support. The ALS extraction geometry should correspond to the label’s ecological support, not to a convenient tile size. A 6-minute bird point count has an effective radius of tens to a couple of hundred metres; a camera trap detection zone is ~ 10 m but the animal’s home range is kilometres; a beetle pitfall array spans a 40 m plot perimeter. **These differ by orders of magnitude and no single extraction extent serves them all** — which is the argument for §19.4.

Model detectability where the data supports it, and say so where it does not. NEON birds give you distance and method; eBird gives you effort covariates; iNaturalist gives you nothing. A benchmark that treats all three as equivalent presence-absence data is making a statistical error, not a simplification.

State the temporal offset policy per task, and report sensitivity to it. A strict ± 1 year window kills most matches; ± 5 years admits growth and disturbance into the “ground truth.”

Aggregate taxonomically where the labels demand it, and report at multiple levels. Species-level results with 3 detections are noise; family-level results may be too coarse to be ecologically interesting. Show both.

Chapter 19. Evaluation Protocol

This is the chapter that decides whether the benchmark is believed.

19.1 What is being measured

Be explicit, because it changes the design. You are not asking “can a model predict birds from lidar” — everything predicts birds a little. You are asking:

Does an ALS-derived representation carry ecological information beyond what is available from cheaper, more widely available data?

That framing makes the ablation ladder (§19.5) the core of the protocol rather than an appendix, and it makes rung 3 — an optical remote-sensing baseline — non-negotiable.

19.2 Frozen probing versus fine-tuning

Linear probe — a single linear layer on frozen features. Measures linear separability, is nearly hyperparameter-free, and is the cleanest comparison across representations.

Shallow MLP probe — one or two hidden layers, as in your proposal. More capacity, more sensitive to hyperparameters, and — the concern worth naming — **enough capacity to launder spatial autocorrelation into apparent skill**. With coordinates implicitly recoverable from a rich embedding (and they often are, since an ALS embedding encodes terrain, land cover, and acquisition characteristics that are all spatially structured), a two-layer MLP can memorize spatial position and interpolate the response. This is precisely what rung 1 of the ablation ladder detects.

Full fine-tuning — measures transferable capacity, costs far more compute, and confounds representation quality with optimization.

Report all three. They answer different questions and they disagree in informative ways: a representation that is strong under fine-tuning and weak under linear probe is entangled but useful; the reverse is well-organized but shallow.

Fix the probe protocol across all representations. Same optimizer, same schedule, same regularization search, same seed count. Otherwise you are comparing tuning budgets.

19.3 Pooling: the underrated design decision

An encoder produces per-point features. A probe needs one vector per sample. How you get from one to the other is a *modelling choice*, not a detail, and it can dominate the result.

Options, roughly in order of increasing expressiveness:

- **Mean pooling** — simple, and it discards the vertical structure that is the entire point of lidar.
- **Max pooling** — the PointNet default; sensitive to outliers, which under §10.6 means sensitive to noise-filter aggressiveness.
- **Mean + max concatenation** — cheap and usually better than either.
- **Height-stratified pooling** — pool separately within height-above-ground bands and concatenate. **This is the option I would default to**, because it preserves vertical arrangement, mirrors the engineered-feature vocabulary (§16.4), and is trivially interpretable.
- **Attention pooling** — a learned query attends over points. More expressive; adds parameters to the “probe,” which muddies the frozen-representation claim. If used, report it separately.

[Case study] I would make pooling an explicit reported ablation on at least one task. If mean-pooling a PTV3 encoder loses to height-stratified pooling by a large margin, that is a finding about the representation — it says the vertical information is present but not in the mean — and it is exactly the kind of result that makes a benchmark paper useful rather than merely evaluative.

19.4 Multi-scale extents

§18.10 established that your tasks span three orders of magnitude of ecological support. One extraction extent cannot serve them.

The design: emit **multi-scale pooled embeddings** — the same encoder applied over concentric extents (say 30 m, 100 m, 300 m, 1 km around each sample centre), each pooled to a vector, and let each task’s probe select or weight scales.

Three benefits:

- 1. **Each task gets the support it needs** without a separate model.
- 2. **The scale weights are a result**, not just a mechanism: *which structural scale predicts which taxon* is an ecological finding, and a more interesting one than the headline accuracy.
- 3. **It makes the block-size chapter load-bearing** (Chapter 23) rather than an appendix.

Implementation notes: the concentric crops share a centre, so the read is one buffered fetch with multiple crops (§14.5). Compute cost scales with area, so the 1 km extent dominates — consider a coarser resolution at larger extents, which is also ecologically sensible since fine structure matters less at landscape scale.

19.5 The ablation ladder

Every task gets the same five rungs. This is the spine of the protocol.

Rung	Predictors	Answers
1	Coordinates only (or spatial smoothing of the response)	The leakage detector. If rung 1 approaches rung 5, your split is not spatially independent and nothing else in the table means anything. The traditional SDM baseline.
2	Climate + terrain (WorldClim/DAYMET + 3DEP DEM derivatives)	
3	+ optical satellite embedding (Sentinel-2, or an existing RS foundation model)	The rung you must beat. Without it, “ALS embeddings improve prediction” is unfalsifiable — reviewers will assume the gain is generic remote-sensing information. Your contribution. Diagnostic of what the representation encodes in isolation.
4	+ ALS embedding	
5	ALS embedding only	

Add, for the sources where it matters, a **rung 2b: road distance and accessibility co-variates only** — the shortcut detector for eBird, Snapshot USA, and NEON small mammals (§18.2, §18.4, §18.7).

And add a **rung 4b: engineered ALS metrics instead of the learned embedding** (§16). This is the comparison that tells you whether the representation learned anything the vocabulary does not already encode. If rung 4b matches rung 4, you have built an expensive metric calculator — a legitimate and publishable negative result, and one worth knowing early.

19.6 Splitting

Random splits are meaningless here, and the field has moved decisively on this point. Ploton et al. (2020) reproduced a standard large-scale biomass mapping workflow on 11.8 million trees in central Africa: a standard non-spatial validation suggested the model predicted more than half the biomass variation, while spatial validation accounting for autocorrelation revealed quasi-null predictive power. The reported R^2 in that case study ranged from 0.14 to 0.53 depending purely on evaluation strategy.

There is a genuine counter-argument worth knowing rather than ignoring: Wadoux et al. argue that spatial cross-validation is not the right way to evaluate *map accuracy*, and that design-based inference from probability sampling gives unbiased accuracy estimates without adjusting for autocorrelation. The resolution is that the two camps are answering different questions — map accuracy over a population versus model transferability to new locations — and **your benchmark is about transferability**, so spatial splitting is the right instrument. But cite both, because a reviewer from the other camp will otherwise think you are unaware.

The splits I would require, reported side by side:

Split	Held out	Question
Random	—	Reported only as the inflation reference
Spatial block	Contiguous blocks with a buffer	Local transferability
Leave-one-ecoregion-out	An entire ecoregion	Ecological transferability
Leave-one-project-out	A 3DEP work unit	Sensor + processing + geography jointly
Leave-one-sensor-out	All projects with a sensor model	Acquisition transferability
Leave-one-QL-out	A quality level	Density transferability
Paired sensor (§4.3)	Same footprint, different acquisition	The sensor gap, isolated

Buffer sizing. The buffer between train and test blocks must exceed the model’s receptive field, not merely the autocorrelation range. A PTv3 with 1 km context sees a kilometre; a 100 m buffer is decorative. **[Case study]** State the buffer as a function of receptive field, and recompute it if you change architecture.

Reporting the random-versus-spatial gap is itself a contribution. Most ALS-ML papers report one number. Reporting both, with the inflation factor, gives the field a calibration point.

19.7 The paired sensor design

Reprising §4.3 with the protocol now in place, because this is the design element I consider most novel in your project.

3DEP covers the NEON sites. That gives you:

Track	Point cloud	Labels	Measures
A	3DEP over NEON footprints	NEON field plots	Ecological transfer, sensor near training distribution
B	NEON AOP, same footprints	Same labels	Full transfer: geography + sensor + density
A – B	—	—	The acquisition domain gap, isolated

Track A is your headline generalization number; Track B is the honest out-of-domain number; the difference is a clean measurement essentially nobody has published for ALS. It directly answers the question every downstream user will ask: *my county was flown with a different sensor at a different density — will this work?*

Three requirements to make it valid, all of which the provenance schema (§11.7) exists to enforce:

1. **Identical processing.** Same re-derived ground algorithm and parameters, same noise filter, same HAG method, same extraction geometry. Otherwise you measure a processing gap and call it a sensor gap.
2. **Leakage exclusion at the right granularity.** Exclude *all* 3DEP collections over NEON footprints from pretraining — not just the tiles used, since many areas have repeat coverage from different years — with a buffer exceeding the receptive field. If you also want leave-one-ecoregion-out, exclusion must happen at ecoregion granularity, which costs real training area. `splits.neon_site_buffer_exclusion` is the field that enforces this.
3. **Phenology control.** NEON flies at peak greenness; 3DEP prefers leaf-off in deciduous regions (§7.2). Restrict Track A to leaf-on 3DEP collections, or stratify and report both. Otherwise the “sensor gap” contains a phenology gap.

19.8 Metrics and reporting

Per task: the natural metric (RMSE/bias/ R^2 for regression; AUC, PR-AUC, and calibration for occurrence; overall *and macro* accuracy for classification).

Always report macro alongside overall for anything multiclass. §17.4’s Finnish result — 92.0% overall against 85.1% macro, versus a random forest’s 83.2% and 61.3% — shows how much of the story lives in that gap.

Calibration, not just discrimination. For occupancy and occurrence tasks, a well-discriminating but poorly-calibrated model is not usable for the ecological inference these data exist to support. Report reliability diagrams or expected calibration error.

Label-efficiency curves. Performance at 1%, 10%, and 100% of labels. This is where pretrained representations earn their keep, and where a headline number at 100% hides the actual benefit.

Compute cost per result. Reporting FLOPs or GPU-hours alongside accuracy lets readers judge whether a 1-point gain is worth a $40\times$ cost. Given the SPT result (212k parameters, 79.6% mIoU on DALES, trainable on a single GPU in hours), the efficiency frontier is a real and under-reported axis.

Uncertainty. Multiple seeds, and confidence intervals on the reported differences. Many ALS-ML comparisons report single runs with differences smaller than seed variance.

Stratified tables, not pooled numbers. Break every headline result by quality level, sensor, phenology, ecoregion, and slope class. This is what the provenance schema is for, and it is what makes negative results interpretable (§11.8).

19.9 Reporting negative results honestly

Some of your tasks will show that ALS adds little over optical plus terrain. That is a finding, not a failure, and how you present it determines whether the benchmark is trusted.

Three practices:

1. **Pre-register the ablation ladder and the splits** before running the biodiversity tasks. Not formally, necessarily, but in the repository with a timestamp. It removes the suspicion that rungs were added or dropped after seeing results.
2. **Report the tasks where ALS does not help as prominently as the ones where it does.** A benchmark that only surfaces wins is a marketing document.
3. **Distinguish “no signal” from “no measurable signal.”** §3.5’s near-ground blind zone means understory-dependent tasks have a sensing floor; §18.3’s coordinate error means iNaturalist has a measurement floor. Where a floor exists, name it, so a null result is attributed correctly.

[**Case study**] A specific prediction I would write down now, so it can be checked: I expect ALS to add most for taxa with mechanistic dependence on vertical structure measured at overstory scale — birds (FHD is a classic predictor for a reason), bats, arboreal mammals — and least for ground-dwelling taxa whose habitat is in the stratum the sensor barely measures. If that pattern does not appear, it is evidence that something in the representation or protocol is wrong, and the diagnostic is worth more than the result.

19.10 The protocol as a checklist

For each task, before running anything:

- ☐ Ecological support identified, and extraction geometry matched to it
- ☐ Detection structure named (presence-absence with effort / presence-only / census)
- ☐ Coordinate uncertainty stated and compared to support
- ☐ Temporal offset policy fixed, sensitivity planned
- ☐ All five ablation rungs specified, plus 2b (accessibility) and 4b (engineered metrics)
- ☐ Splits specified, buffer sized against receptive field
- ☐ Pooling strategy fixed and identical across representations
- ☐ Scales specified for multi-scale embedding
- ☐ Probe protocol fixed (linear + MLP + fine-tune, same budget for all)
- ☐ Metrics include macro, calibration, label-efficiency, and compute

- ☐ Stratification variables available in the manifest
- ☐ Known sensing or measurement floors documented in the task description

Twelve items, most of which are decisions rather than work. Making them before seeing results is the difference between a benchmark and a collection of numbers.

Further reading for Chapters 18–19

- [eBird Best Practices v2.0](#) (Strimas-Mackey et al. 2023) and Johnston et al. (2021), *Analytical Guidelines to Increase the Value of Community Science Data*, Diversity and Distributions — the methodological foundation for §18.2.
- Ploton et al. (2020), *Spatial validation reveals poor predictive performance of large-scale ecological mapping models*, Nature Communications, with [code on figshare](#); and Wadoux et al., *Spatial cross-validation is not the right way to evaluate map accuracy*, Ecological Modelling, for the counter-argument. Read both before fixing your splits.
- Roberts et al. (2017), *Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure*, Ecography — the practical guide, and the source of most blocking schemes in use.
- [Sabatini et al. \(2021\)](#), [sPlotOpen](#), Global Ecology and Biogeography, and the [data description site](#).
- [Biolith](#) for the occupancy modelling in §18.4.
- [NEON data portal](#) and Meier et al. (2023), [Ecosphere](#), for the §18.7 products and their design links to AOP.
- [OpenForest](#) for anything not covered here.

Bridge to Part VI

Part V specified what to measure and how to judge it. Part VI turns to the models.

Chapter 20 asks what makes ALS different from every other 3D benchmark — 2.5D acquisition geometry, systematic vertical occlusion, unbounded horizontal extent, anisotropic density, meaningful absolute z — and traces each property to an architectural consequence. Chapter 21 develops the hierarchical scaling mechanisms in detail: PointNet++’s sampling-grouping-pooling, the point transformer line, and PTV3’s replacement of neighbourhood *search* with neighbourhood *ordering* via space-filling curves, together with the question of what a curve defined on a cube means for a tile that is $1000 \times 1000 \times 50$ m. Chapter 22 covers self-supervised pretraining and the geometric shortcut that makes it fail in 3D specifically. Chapter 23 covers spatial extent, crop size, and inference stitching — the chapter that the multi-scale design of §19.4 has been quietly depending on.

Part VI — Deep Learning on ALS

Airborne Laser Scanning for Machine Learning in Ecology

Orientation to Part VI

The point-cloud deep learning literature was built on indoor scans and autonomous-driving data. Almost every architectural decision you will inherit was made against ScanNet, S3DIS, nuScenes, or SemanticKITTI. None of those datasets shares ALS’s acquisition geometry, extent, density structure, or semantics.

This Part is about which of those inherited decisions transfer, which need modification, and which are actively wrong for airborne data. Four chapters:

- **Chapter 20** establishes what makes ALS different, property by property, tracing each to an architectural consequence. It is the framing chapter and the one to read first.
- **Chapter 21** develops the hierarchical scaling mechanisms in detail — PointNet++’s set abstraction, the point transformer line, PTv3’s serialization, and superpoint partitioning. This is the longest chapter in the book because the mechanisms are where the ALS-specific questions actually live.
- **Chapter 22** covers self-supervised pretraining, the geometric shortcut that makes 3D SSL fail in ways image SSL does not, and why that failure mode is worse in ALS than anywhere else.
- **Chapter 23** covers spatial extent, crop size, and inference stitching — the chapter the multi-scale evaluation design of §19.4 has been depending on.

A warning about the literature. The frontier here moves fast and the ALS-specific literature lags it badly. A March 2026 ALS segmentation benchmark notes that the rapid emergence of transformer-based architectures has outpaced benchmark development, with recent models such as Superpoint Transformer and Point Transformer V3 largely unevaluated on aerial data. That gap is your opportunity, and it is also a reason to check publication dates on everything in this Part.

Chapter 20. What Makes ALS Different

20.1 The comparison

	ShapeNet / ModelNet	ScanNet / S3DIS	KITTI / nuScenes	ALS
Acquisition	Synthetic / complete	Multi-view RGB-D	Single spinning sensor	Single downward pass 2.5D, systematically occluded downward Unbounded horizontally, thin vertically
Completeness	Complete surfaces	Mostly complete rooms	Radial, self-occluded	
Extent	Object, normalized	Room to building	~100 m radius	

	ShapeNet / ModelNet	ScanNet / S3DIS	KITTI / nuScenes	ALS
Density	Uniform by construction	Roughly uniform	$1/r^2$ radial falloff	Anisotropic, swath-structured, non-stationary
Orientation	Canonical, often arbitrary z	Gravity-aligned	Gravity-aligned	Gravity-aligned with meaningful absolute z
Coordinates	Normalized to unit sphere	Room-local	Ego-centric	Georeferenced, globally meaningful
Semantics	Rigid objects	Rigid furniture, planes	Vehicles, roads, dynamics	Self-similar fractal vegetation
Colour	Sometimes	Registered RGB	Registered images	Rarely; intensity is not colour
Labels	Object class	Dense per-point	Boxes + per-point	Sparse, plot-level, indirect

Nine rows, and each generates a design consequence. I take them in order of how much they should change your architecture.

20.2 Property 1: 2.5D acquisition and systematic vertical occlusion

The property. Every point was measured along a nearly-vertical line of sight from above. There is no view from below, from the side, or from inside. Occlusion is not random missingness — it is a deterministic function of what is above a given location.

Why this is different from KITTI. A spinning automotive lidar also produces occlusion, but it is *radial* around a moving sensor, and over a sequence the ego-motion fills in many occluded regions from different angles. ALS gives you one look (or two in sidelap, §2.4), from one direction, once.

Consequences:

- **The lower canopy and understory are not sparsely sampled; they are conditionally sampled,** with the condition being canopy density above. So the point density at height z is a *measurement of the canopy above z* , not just a sampling artifact. A model that treats missing points as missing data discards this. A model that can condition on “how many pulses reached this depth” has access to real structural information.
- **Completion is ill-posed in a specific direction.** Generative or reconstruction-based objectives that ask the model to fill in occluded regions are asking it to hallucinate the interior of a crown. That is not necessarily useless as a pretext task, but the reconstruction loss on deeply occluded regions is measuring the model’s prior, not its perception.

- **Symmetry-based augmentation is partly invalid.** Rotating a cloud about a horizontal axis produces a configuration no sensor could have produced. Rotation about the vertical axis is fine (and should be used); flips in x and y are fine; anything that tilts the cloud breaks the acquisition model. **This rules out the full $SO(3)$ augmentation that object-centric literature uses freely.**

20.3 Property 2: unbounded horizontal extent, thin vertical extent

The property. A tile is 1000×1000 m and perhaps 50 m of vegetation on top of arbitrary terrain relief. The aspect ratio is 20:1 or worse in the vegetation dimension, and the whole surface is a graph over the horizontal plane.

Consequences:

- **Isotropic voxels waste capacity.** A 0.5 m cubic voxel over a 1 km tile is $2000 \times 2000 \times 100$ — and the vertical dimension is where the information is. Anisotropic voxelization (finer in z than in x, y) is worth trying and is almost never done, because the inherited defaults come from rooms and street scenes where the aspect ratio is near 1.
- **Space-filling curves are defined on cubes.** This is the concrete problem for PTV3 on ALS, developed in §21.6.
- **The 2.5D structure means 2D methods are not obviously wrong.** A CHM plus a 2D CNN discards the vertical detail but keeps the horizontal structure at full resolution, in a mature and extremely efficient architecture family. **This is why the rasterized baseline is not a strawman**, and why I insist on it in §20.7.
- **Batching is awkward.** Samples have variable point counts over fixed areas, or fixed counts over variable areas. Neither batches cleanly. This is a real engineering cost and it drives the block-size decisions of Chapter 23.

20.4 Property 3: anisotropic, non-stationary density

The property. Established at length in Parts I–II: scan-mechanism density piling at swath edges, sidelap stripes, turbulence striping, atmospheric dropout, terrain-following AGL variation, and canopy-dependent return counts. Density varies *within* a tile with structure that has no landscape correlate.

Consequences:

- **Fixed- k neighbourhoods have variable physical scale**, so any k -NN-based operator computes features at different physical scales in different parts of one sample (§16.5). Ball queries with fixed radius have constant scale and variable point count, which is the better trade for ALS — and this is exactly the argument PointNet++ made for ball query over k -NN, for exactly this reason (§21.3).
- **Voxel-based methods are more robust to this** than point-based ones, because a voxel has fixed physical size by construction. This is an underrated argument for sparse convolution on ALS.
- **Density should be an input, not just a nuisance.** Per-point local density, or per-sample pulse density as a global conditioning vector, lets the model calibrate rather than confound (§4.5, Strategy 2).

- **Farthest point sampling interacts badly with density variation.** FPS is density-agnostic by design, which sounds good, but it means the sampled set is *uniformly spread* while the original points were not — so the ratio of sampled to original points varies spatially, and the information content per sampled point varies with it.

20.5 Property 4: gravity alignment and meaningful absolute z

The property. ALS is always gravity-aligned, and z has two meanings: absolute elevation (ecologically informative, datum-contaminated per §9.7) and height above ground (the workhorse, §16).

Consequences:

- **Vertical and horizontal axes should not be treated symmetrically.** Most architectures do, inheriting from object-centric work where orientation is arbitrary. Anisotropic voxels, separate treatment of z in positional encoding, and height-stratified pooling (§19.3) all follow.
- **HAG is enormously predictive**, which is both the opportunity and the trap. It is the trap because a model given HAG can predict most vegetation classes from z alone, learning nothing about structure. This is the geometric shortcut in its ALS-specific form (§22.3).
- **Absolute z carries a continental datum artifact** and is a longitude proxy (§9.7). Provide elevation coarsened and deliberately; do not hand it to the model as raw per-point z .

20.6 Property 5: self-similar semantics and sparse indirect labels

The property. Indoor and driving datasets are dominated by objects with characteristic scales and rigid shapes — a chair is chair-sized, a car is car-shaped. Vegetation is approximately scale-free: a branch looks like a smaller branch, a crown like a cluster of crowns. And your labels are not per-point; they are plot-level scalars measured by a field crew (§18).

Consequences:

- **Scale is not a nuisance to be normalized away; it is the signal.** In object recognition, normalizing an object to a unit sphere is standard and harmless. In ALS, absolute metric scale is the information — a 30 m tree and a 3 m shrub differ *only* in scale. **Never scale-normalize an ALS crop.**
- **Multi-scale processing is not optional**, because the phenomena span two orders of magnitude within one sample.
- **The label structure pushes toward encoder-plus-pooling rather than encoder-decoder.** You mostly need one vector per plot, not per-point predictions. This has a specific architectural consequence, and it is the same one Sonata arrived at from a different direction (§22.4): if you do not need a per-point decoder, not having one avoids reintroducing local geometric shortcuts.
- **Weak and noisy supervision is the norm**, so label-efficiency and robustness to label noise matter more than in benchmarks with dense clean annotations.

20.7 Representation families

Rasterized 2.5D. CHM, DTM, DSM, density, and height-percentile bands stacked as a multi-channel image; 2D CNN or ViT. Discards within-pixel vertical detail; keeps everything else at full resolution with a mature, efficient, well-understood architecture family and access to ImageNet-scale pretraining.

This is the baseline everyone under-reports and you must include. For area-based tasks over plot supports, a multi-band raster CNN is often within noise of a point-based model at a fraction of the cost. If your point-based model does not beat it, that is the headline result, and it is better to discover it yourself.

Voxel / sparse convolutional. Submanifold sparse convolution (spconv, torchsparse) on a voxel grid. Fixed physical scale per voxel (robust to density variation), efficient on sparse data, mature. The voxel size is the single most consequential hyperparameter, and anisotropic voxels are worth trying.

Pillar encodings — voxelize in x, y only, encode the full vertical column as a feature vector. Borrowed from automotive BEV detection, and *unusually well matched to ALS's 2.5D geometry*, since a pillar is exactly “everything above this ground location.” This is why BEV-MAE is a sensible pretraining choice for ALS (§22.5).

Point-based. PointNet $\rightarrow$ PointNet++ $\rightarrow$ KPConv $\rightarrow$ point transformers. Operates on points directly, no discretization loss, and the family where the ALS-specific questions of Chapter 21 arise.

Superpoint / graph-based. Partition into geometrically homogeneous superpoints, then reason over the (much smaller) superpoint graph. Extremely efficient; a fundamentally different answer to hierarchical scaling (§21.7).

Hybrid and range-image. Range images exploit the sensor's native acquisition geometry, which for a spinning automotive lidar is a natural 2D grid. For a scanning ALS system there *is* an analogous native parameterization (scan angle $\times$ along-track position per flight line), and it is essentially unexplored. **[Case study]** Worth flagging as a genuinely novel direction: a per-flight-line range-image representation would make scan geometry explicit rather than implicit, would naturally handle the anisotropy of §20.4, and would make the occlusion structure of §20.2 a channel rather than a confound. It would require flight-line-organized data (§6.4's swath deliverables) rather than tiles.

20.8 The design summary

ALS property	Architectural response
2.5D, downward occlusion	Restrict augmentation to z -axis rotation and horizontal flips; treat penetration depth as signal
Unbounded horizontal, thin vertical	Anisotropic voxels; keep the raster baseline honest; expect batching friction
Non-stationary density	Prefer fixed-radius over fixed- k ; condition on density; degradation augmentation (§4.5)

ALS property	Architectural response
Gravity-aligned, meaningful z	Asymmetric axis treatment; height-stratified pooling; guard against the HAG shortcut
Self-similar semantics	Never scale-normalize; multi-scale mandatory
Sparse plot-level labels	Encoder + pooling over encoder-decoder; prioritize label efficiency
Intensity semantics unreliable	Exclude by default; ablate (§3.7)

Chapter 21. Hierarchical Scaling in Detail

Every architecture in this space solves the same problem: a point cloud has too many points for global pairwise operations, so you need a hierarchy that grows the receptive field while shrinking the point count. The mechanisms differ, and the differences matter for ALS specifically.

21.1 The problem

Global attention over N points costs $O(N^2)$. A 1 km² QL2 tile has around 10^7 points. This is not a constant-factor problem.

Three families of solution:

1. **Sample and group** — reduce the point set, define local neighbourhoods, aggregate. PointNet++, KPConv, point transformers V1/V2.
2. **Discretize** — impose a grid, exploit its regularity. Sparse convolution, grid pooling.
3. **Order** — impose a 1D ordering on the points and operate on contiguous windows of it. PTV3.

And a fourth that is really a preprocessing move: **partition** — group points into superpoints and reason over the partition graph. SPT.

21.2 PointNet: the foundation

PointNet processes each point independently through a shared MLP, then aggregates with a symmetric function (max):

$$f(\{x_1, \dots, x_N\}) = \gamma \left(\max_{i=1..N} h(x_i) \right)$$

Max-pooling over per-point embeddings gives permutation invariance — the essential requirement. The universality argument is that this form can approximate any continuous set function given sufficient width.

The limitation that motivated everything after: **no local structure**. Each point sees only itself and the global max. There is no notion of a neighbourhood, hence no hierarchy, hence no multi-scale reasoning. For ALS, where the semantics are multi-scale (§20.6), this is disqualifying.

21.3 PointNet++: set abstraction

The fix: apply PointNet hierarchically over nested neighbourhoods. A **set abstraction (SA) layer** is three operations.

Sampling

Choose $M < N$ centroids by **farthest point sampling**: start from a random point, then repeatedly add the point maximally distant from the already-selected set. Result: a subset spread roughly uniformly over the cloud’s support.

FPS costs $O(NM)$ — for $N = 10^5$ and $M = 10^4$, that is 10^9 distance computations per layer. **FPS is the practical bottleneck of PointNet++-style architectures at ALS scale**, and it is why later architectures replaced it (grid pooling, serialization).

Its property, and the ALS-relevant subtlety: FPS is *density-agnostic*, spreading centroids uniformly regardless of the input distribution. That sounds desirable and has a hidden cost — the sampling *ratio* varies spatially, so a dense region is subsampled harder than a sparse one, and the amount of information each centroid summarizes varies across a single ALS sample (§20.4).

Alternatives: random sampling (fast, density-proportional, so it preserves the input distribution’s bias); grid/voxel sampling (fixed physical scale, the choice PTV2/PTV3 make); inverse-density sampling.

Grouping

For each centroid, define a neighbourhood. Two options, and PointNet++’s own reasoning for preferring one is directly the ALS argument:

- **Ball query** — all points within radius r , capped at K . **Fixed physical scale**, variable point count.
- **k -NN** — the k nearest points. Fixed point count, **variable physical scale**.

PointNet++ prefers ball query specifically because it guarantees a fixed-scale region, making local features more generalizable across varying sampling density. **That argument was made for a mildly non-uniform setting and applies with much greater force to ALS** (§20.4). If you build a PointNet++-family model for ALS, use ball query, and treat the radius schedule as a first-class hyperparameter tied to physical scales in metres.

Local PointNet

Within each group, translate coordinates to be relative to the centroid:

$$x_j^{(i)} \leftarrow x_j - x_i$$

then apply a shared MLP and max-pool. **The translation is what creates translation invariance** and what makes the learned features local-geometry descriptors rather than absolute-position descriptors.

[Case study] There is a real design question hiding here for ALS. Translating in z as well as x, y discards the point’s height above ground within the group — which is the single most informative feature you have. Options: translate in x, y only, keeping absolute or normalized z ; translate fully but append HAG as an extra input channel; or translate fully and rely on the hierarchy to recover height context. The middle option is what most forestry implementations do, and it is the one that makes the shortcut of §22.3 most available. **Worth an explicit ablation**, because it sits exactly at the tension between “give the model the informative feature” and “don’t let it shortcut.”

Density robustness: MSG and MRG

PointNet++ introduced two mechanisms explicitly to handle non-uniform density:

- **Multi-scale grouping (MSG)** — at each centroid, group at several radii, run a separate PointNet per radius, concatenate. With random input dropout during training, the network learns to weight scales according to local density.
- **Multi-resolution grouping (MRG)** — concatenate features from the previous hierarchy level with features computed directly from the raw points in the region, avoiding MSG’s cost of running large-radius PointNets at every centroid.

These mechanisms were designed for exactly the ALS problem and are usually inherited without examination. Two thoughts. First, MSG with random input dropout is essentially a density-augmentation scheme built into the architecture, which overlaps with §4.5’s Strategy 4 — worth knowing so you do not double-apply or, worse, believe you have handled density when you have only handled it at one layer. Second, MSG’s cost is why it fell out of favour, but the ALS density problem is more severe than the one it was designed for, so the cost-benefit may differ here. Re-examine rather than inherit.

The decoder

For per-point outputs, **feature propagation** layers interpolate features from the coarse set back to the fine set using inverse-distance-weighted k -NN:

$$f^{(j)} = \frac{\sum_{i=1}^k w_i f^{(i)}}{\sum_{i=1}^k w_i}, \quad w_i = \frac{1}{d(x_j, x_i)^p}$$

then concatenate with skip-connected encoder features and apply a unit PointNet. Standard U-Net logic, adapted to irregular sampling.

Note for §22.4: this decoder is precisely the structure Sonata removes, because hierarchical decoding with skip connections reintroduces access to fine-grained local geometry and thereby the geometric shortcut.

21.4 KPConv and the convolution analogy

KPConv defines a convolution on points via a set of **kernel points** in a local neighbourhood, each with a learned weight matrix; a point’s contribution is a distance-weighted interpolation over kernel points. Deformable variants learn kernel-point offsets.

Why it matters for ALS: KPConv is a genuine convolution with an explicit, interpretable spatial extent in metres, and the deformable version can adapt to local geometry. It underpins a good deal of the forestry literature (SegmentAnyTree and relatives). Its cost is that neighbourhood construction dominates, which is the same bottleneck as PointNet++.

21.5 Point Transformer V1 and V2

V1 introduces **vector attention** on point neighbourhoods. Instead of scalar dot-product attention, the attention weight is a *vector* produced by an MLP on the difference of query and key, modulated by a learned positional encoding:

$$y_i = \sum_{j \in \mathcal{N}(i)} \rho(\theta(\varphi(x_i) - \psi(x_j) + \delta)) \odot (\alpha(x_j) + \delta)$$

with $\delta = \omega(p_i - p_j)$ a learned function of the relative position. Two things to notice: the *subtraction* relation (rather than dot product) and the fact that positional information enters both the weight and the value. Vector attention gives per-channel modulation, which converges better on point data than scalar attention.

V2 adds **grouped vector attention** (sharing weight vectors across channel groups, reducing parameters and overfitting) and **partition-based pooling** — the grid pooling that PTv3 keeps.

The cost that motivated V3: explicit k -NN search and pairwise relative positional encoding *dominate runtime* — roughly 28% and 26% of PTv2’s forward time respectively. Two-thirds of the compute goes to figuring out who the neighbours are and where they are, not to modelling.

21.6 Point Transformer V3: ordering instead of searching

PTv3’s central move is worth stating precisely because it is easy to misdescribe: **it replaces neighbourhood search with neighbourhood ordering**.

Serialization

Sort the points along a **space-filling curve** — a path that visits every cell of a discretized space with strong locality, so points adjacent along the curve are usually adjacent in space. PTv3 uses four patterns: **Z-order (Morton)**, **transposed Z-order**, **Hilbert**, and **transposed Hilbert**. The sort happens once per pooling level, not per layer.

Patch attention

Group the serialized sequence into **non-overlapping patches of fixed size** by simple contiguous slicing (with padding), and apply standard dot-product attention within each patch. Because the sequence is ordered by spatial locality, a contiguous patch is approximately a spatial neighbourhood.

This is why the receptive field can grow from 16 neighbours in PTv2 to **1024 points** per attention layer: patch attention on a sorted array is a windowed attention over a regular sequence, so all the machinery of efficient transformer attention (including FlashAttention)

applies. The reported result is roughly **3× faster and 10× more memory-efficient than PTV2** at comparable or better accuracy.

PTv3’s authors are explicit that serialization is *less precise* than k -NN — a contiguous patch is not exactly a ball — and argue the accuracy gap is more than repaid by the scale it enables. This is the “scale over complexity” thesis, and it is the single most important architectural argument of the last few years in this field.

Patch interaction

Fixed non-overlapping patches would create permanent walls at patch boundaries. PTV3 breaks them by varying the ordering across layers: **shift order** (cycling through the four serialization patterns), **shuffle order** (randomly permuting which pattern is used), **shift dilation**, and **shift patch**. Points that are split across a boundary in one layer are grouped together in the next.

xCPE and block structure

Enhanced conditional positional encoding (xCPE) replaces expensive pairwise relative positional encoding with a single sparse convolution layer plus skip connection, prepended before attention. Cheap, and it injects local geometric context.

Block structure details worth knowing because they are easy to get wrong when adapting the architecture: **pre-norm**, and **Layer Normalization inside attention blocks** — but PTV3’s own ablation found **Batch Normalization is essential in the pooling path and cannot be replaced by LN**, hypothesized to stabilize point-cloud data distribution during pooling. Down-sampling is **grid pooling** inherited from PTV2, with shuffle order integrated into it.

The ALS-specific problem with serialization

Here is the issue I flagged in §20.3, now concretely. **Space-filling curves are defined on a cube.** Z-order interleaves the bits of the x , y , and z integer coordinates; Hilbert curves are constructed recursively on a cubic domain. Both assume the three axes are comparably scaled.

An ALS tile is roughly $1000 \times 1000 \times 50$ m of vegetation over arbitrary relief. Feed that to a bit-interleaving curve with a common grid size and the z bits are nearly constant across the tile, so the curve degenerates toward a 2D curve with a small vertical wobble — which may be fine, or may destroy vertical locality in exactly the dimension that matters. Feed it a per-axis normalized coordinate and the physical meaning of “patch” becomes anisotropic in an uncontrolled way.

Three things to test, and I would treat this as a headline experiment rather than a hyperparameter sweep:

1. **Anisotropic grid size for serialization** — a finer quantization in z than in x, y , so the curve resolves vertical structure at the scale it actually varies.
2. **2D serialization with z as a feature** — serialize on (x, y) only, and give the network height as an input channel. This makes the pillar interpretation explicit (§20.7) and

matches the 2.5D geometry.

3. **Patch size in metres, not points.** A patch of 1024 points spans a very different physical extent at 2 pulses/m² than at 40. Since patch size *is* the receptive field, **the effective receptive field of a PTv3 on ALS varies with acquisition density** — which is a domain-shift mechanism nobody has characterized, and which interacts directly with your leave-one-QL-out split (§19.6).

Point 3 is, I think, the most interesting open question in this chapter. It predicts a specific failure: a model trained at QL1 density and evaluated at QL2 has its receptive field silently doubled in physical extent, and vice versa. If that prediction holds, the fix is straightforward — normalize density before serialization, or scale patch size by measured density — and the measurement would be a genuine contribution.

21.7 Superpoint Transformer: partition instead of hierarchy

A different answer. **Superpoint Transformer** first partitions the cloud into geometrically homogeneous **superpoints**, builds a hierarchical superpoint graph, and applies attention over that graph. Because the partition reduces the problem size by orders of magnitude before any learning happens, the model can be tiny: **212k parameters, up to 200× more compact than comparable state of the art, trainable on a single GPU in hours, and 79.6% mIoU on DALES.**

Two reasons to take this seriously for ALS. First, **DALES is aerial lidar**, making this one of the few architectures with a published aerial number rather than an indoor one. Second, the partition step is a *learned or geometric* form of the scale-selection problem that §16.5 and §21.3 keep running into — superpoints adapt their extent to local geometry by construction, which is precisely what fixed-radius and fixed- k neighbourhoods cannot do. The project is actively developed, including a GPU-accelerated learnable partition.

The trade-off: the partition is a hard commitment made before learning, so errors in it are unrecoverable, and vegetation — being self-similar and lacking crisp geometric boundaries (§20.6) — is the hardest case for geometric partitioning. Whether superpoints work as well on canopy as on buildings and ground is an empirical question worth answering.

21.8 Comparison and recommendation

	Neighbourhood	Scale control	ALS fit	Cost
PointNet++ (ball query, MSG)	Fixed-radius balls	Explicit, in metres	Good in principle; MSG designed for density variation	FPS is the bottleneck
KPConv	Fixed-radius, kernel points	Explicit, deformable	Good; interpretable extent	Neighbour search dominates
PTv2	k -NN + vector attention	Implicit, variable scale	Weak — variable physical scale under density variation	KNN + RPE = 54% of forward time

	Neighbourhood	Scale control	ALS fit	Cost
PTv3	Serialized patches	Implicit, density-dependent	Excellent throughput; §21.6 caveats unresolved	Best in class
Sparse conv	Voxel	Fixed physical size	Robust to density by construction	Efficient, mature
SPT	Superpoints	Adaptive by construction	Published aerial results; vegetation partition untested	Extremely cheap

My recommendation for the benchmark, in priority order:

1. **PTv3 via Pointcept as the primary backbone**, with Sonata weights as the pre-trained initialization, and the §21.6 serialization questions as an explicit experiment rather than a default.
2. **Sparse-conv U-Net as the robustness reference**. Fixed physical voxel size makes it the natural control for density-driven domain shift, so the PTv3-versus-sparse-conv gap under leave-one-QL-out is directly interpretable.
3. **SPT as the efficiency-frontier comparison**, and because it has aerial numbers.
4. **Multi-band raster CNN as the honest baseline** (§20.7).
5. **PointNet++ with ball-query MSG** only if you want to test the density-robustness mechanisms specifically — it is not competitive on throughput, but MSG is the mechanism most explicitly designed for your problem, and knowing whether it helps is worth one experiment.

Further reading for Chapter 21

- Qi et al. (2017), *PointNet* and *PointNet++* — read the PointNet++ density-robustness section (MSG/MRG) carefully; it is the most ALS-relevant part of the paper and the most skipped.
- Thomas et al. (2019), *KPConv*.
- Zhao et al. (2021), *Point Transformer*; Wu et al. (2022), *Point Transformer V2*.
- Wu et al. (2024), *Point Transformer V3* — read §4.2 on serialized attention and the ablations on serialization pattern, patch interaction, and patch size. The [CVPR version](#) has the serialization figure (Fig. 3) that is the single most useful diagram for §21.6.
- [Superpoint Transformer](#) — repository and papers, including SuperCluster for panoptic segmentation.
- [Pointcept](#) — reference implementations of PTv2, PTv3, and Sonata, with the config system built for the pretrain-then-probe workflow.

Chapter 22. Self-Supervised Pretraining

22.1 Why pretrain at all

3DEP gives you effectively unlimited unlabelled point clouds and a few thousand labelled plots. That ratio is the textbook case for self-supervised pretraining, and it is the reason your project

is framed the way it is.

The closest prior work is *Advancing Aerial Laser Scanning Applications with Large-Scale Pre-Training*, which builds a 3DEP-derived corpus of over 17,000 km² and 184 billion points, sampled to maximize land-cover and terrain diversity, and pretrains with BEV-MAE. **Read it early, and read its negative results by terrain class more carefully than its positive ones** — those define the bar you have to clear and the places where pretraining did not help.

22.2 The masked-modelling design space

The MAE recipe transfers to point clouds with several free choices, each of which matters more than in images.

Masking unit. Images have pixels and obvious patches. Point clouds have no canonical unit: - *Point patches* (FPS centroids + k -NN groups) — the Point-MAE approach; natural for object-centric data. - *Voxels* — fixed physical scale. - *BEV cells / pillars* — the 2.5D-aligned choice, and the one I would default to for ALS (§20.7). - *Vertical columns* — mask everything above a ground location, forcing the model to infer canopy from surroundings. - *Height strata* — mask a vertical band, forcing inference of one canopy layer from the others. Ecologically motivated and, as far as I know, untried.

Masking ratio and schedule. Images tolerate 75%. Point clouds are already sparse, so high ratios can leave nothing to condition on. Sonata’s answer is a **progressive scheduler on mask size and ratio**, increasing difficulty over training so the model cannot settle into an easy solution early.

Reconstruction target. This is the choice with the largest effect: - *Coordinates* (Chamfer distance) — the obvious target, and the one most vulnerable to the shortcut of §22.3. - *Occupancy* — did this voxel contain points? - *Density* — how many? BEV-MAE reconstructs the average point count per masked region specifically to capture local density. - *Geometric properties* — GeoMAE predicts centroid, normal, and curvature per voxel alongside occupancy, on the argument that these yield a nontrivial task and mutually reinforce fine-grained geometric reasoning. - *Features* rather than geometry — the self-distillation route (§22.4).

[Case study] For ALS, density and occupancy targets are more ecologically meaningful than coordinates, because the *number* of returns at a given height above ground is what carries the structural information (§16.3). A model trained to reconstruct return density profiles is being trained on something close to the quantity your downstream tasks actually use. I would rank targets: density-per-stratum > occupancy > geometric properties > raw coordinates.

22.3 The geometric shortcut

This is the most important concept in the chapter.

Sonata’s finding is that existing 3D self-supervised approaches fall short when evaluated by linear probing, and diagnoses the cause as the **geometric shortcut**: representations collapse to low-level spatial features. Their framing of why this is *specific to 3D* is worth internalizing:

In images, information (colour) is organized in a consistent, structured 2D grid. In point clouds, there is no such regular structure — the information is organized via the point coordinates themselves, whose local structure and statistics vary widely by data source. A model that uses point coordinates in its operations therefore has access to information that leaks the answer.

Concretely: the coordinates fed in as positional encoding already contain much of what the pretext task asks the model to predict. The model learns to read positions rather than to understand structure. The symptom is a representation that fine-tunes acceptably but probes terribly — Sonata reports **72.5% linear probing mIoU on ScanNet, roughly 3.3× prior self-supervised methods**, against a full fine-tuning state of the art of 79.4%, which shows how large the gap had been.

Why this is worse in ALS than in indoor scans. Height above ground is *so* predictive of semantic class in vegetation that a model can reconstruct masked regions from z alone. Mask a patch of canopy; the model infers that points should exist at roughly the height of the surrounding canopy, and the reconstruction loss falls without any structural understanding. In an indoor scan, knowing a point’s height tells you a little (floor, table, ceiling). In a forest, knowing a point’s HAG tells you most of what a coarse classifier needs.

So the ALS-specific version of the geometric shortcut is: **the model learns the canopy height model and nothing else.**

22.4 Sonata’s mitigations, and how they map to ALS

Sonata addresses the shortcut through two strategies — obscuring spatial information and enhancing reliance on input features — with four concrete mechanisms. Each maps onto ALS almost directly:

1. Remove the hierarchical decoder; self-distil from encoder output. Hierarchical decoding with skip connections restores access to fine-grained geometry, reintroducing the shortcut. Removing it obscures geometric cues and increases feature channels.

ALS mapping: This aligns with §20.6’s observation that your labels are plot-level, so you do not need a per-point decoder anyway. The architectural pressure from the label structure and from the shortcut point the same direction. **Encoder-plus-pooling, no decoder, is doubly motivated for your benchmark.**

2. Progressive scheduler on mask size and ratio, increasing difficulty to force learning from input features rather than geometric cues.

ALS mapping: Directly applicable. Consider also scheduling *which* masking unit — start with BEV cells, progress to vertical strata — so the model cannot rely on one inference route.

3. Masked point jitter — stronger Gaussian noise applied to the coordinates of masked points, disrupting spatial relationships.

ALS mapping: This is the cleverest of the four and the one I would ablate first. It attacks the shortcut at its source: if masked coordinates are noisy, position cannot be read off precisely. For ALS there is a natural extension — jitter *anisotropically*, with more noise in z than in

x, y , specifically targeting the height shortcut. That is a two-line change with a plausible large effect.

4. Apply SSL losses at coarser spatial scales, plus Layer Normalization in place of Batch Normalization for cross-domain scalability, and Sinkhorn–Knopp centering for similarity matching.

ALS mapping: Coarser-scale losses are well matched to plot-level downstream tasks. The LN choice matters if you pretrain across programs (3DEP + IGN + NEON), where batch statistics differ by source.

Sonata pretrains on 140k point clouds and remains effective with as little as 1% of ScanNet, with a linear layer under 0.2% of model parameters — which is the label-efficiency property your §19.8 curves are meant to measure.

22.5 The height-normalization decision

This is where the shortcut becomes a concrete engineering choice, and there is no free answer.

Option	Pro	Con
HAG only	Removes terrain nuisance; transferable across relief	Hands the model the shortcut; discards elevation as ecological covariate
Absolute z only	Forces the model to learn terrain and vegetation jointly	Huge nuisance variable; datum artifact (§9.7); longitude proxy
Both as separate channels	Model can use each appropriately	Shortcut still available via the HAG channel
HAG + coarsened elevation	Terrain nuisance removed; elevation available as ecology, not artifact	Requires a DEM at inference
HAG with anisotropic jitter during pretraining	Directly attacks the shortcut	Adds a hyperparameter; may harm fine vertical resolution

My recommendation: HAG as the primary vertical coordinate, plus coarsened elevation (say a 1 km-smoothed DEM value) as a separate global channel, plus anisotropic masked-point jitter during pretraining, plus an explicit ablation against absolute z . The reasoning: HAG is what every engineered feature uses and what every downstream task needs; coarsened elevation gives ecology without the datum-and-longitude artifact; and jitter is the mechanism that keeps the shortcut from swallowing the pretraining signal.

22.6 Alternatives to masked modelling

Contrastive and multi-view. The classic instance-discrimination approach needs positive pairs. In ALS you have a genuinely unusual source of *real* positive pairs: **the same ground measured twice**. Sidelap in swath deliverables (§6.4), overlapping 3DEP collections from different projects and years, and NEON’s annual revisits all give two acquisitions of one footprint with different sensors, geometry, and dates.

[Case study] I flagged this in §4.5 as Strategy 3 and I want to restate it as the most promising underexplored idea in this space. A contrastive objective treating two acquisitions of the same

footprint as a positive pair *directly optimizes for the invariance your benchmark measures*. No annotation cost, no simulation assumptions, and the invariance learned is to real sensor and mission variation rather than to a hand-designed augmentation. The caveats are real — vegetation genuinely changes between epochs, so the “invariance” includes some phenological and growth signal — but the temporal gap is recorded, so it can be controlled or used as a curriculum.

Cross-modal. NEON gives co-registered hyperspectral and RGB on the same platform and day; 3DEP footprints can be paired with NAIP or Sentinel-2. Predicting spectral features from structure (or aligning the two embeddings) is a pretext with real semantic content and no geometric shortcut, because spectra are not recoverable from coordinates.

Auxiliary weak supervision. GEDI/ICESat-2 structural estimates, NEON-SD structural diversity metrics (§17.6), or engineered metrics themselves as regression targets. The last is worth considering as a *sanity pretraining* stage: a model that cannot predict h_{95} and cover from its own input has not learned to see.

22.7 A pretraining recipe

Assembling the chapter into something buildable:

1. **Corpus:** nationally stratified 3DEP per §11.7 and Chapter 24, with NEON footprints and their buffers excluded (§19.7).
2. **Preprocessing:** withheld filtered, consistent re-derived ground, HAG plus coarsened elevation, no intensity, density recorded per sample.
3. **Augmentation:** z -axis rotation and horizontal flips only (§20.2); degradation augmentation per §4.5 (pulse thinning, return truncation, near-ground blanking, scan-angle re-sampling, density modulation); **no scale normalization** (§20.6).
4. **Architecture:** PTv3 encoder, no decoder, with the §21.6 serialization variants as an experiment.
5. **Objective:** point self-distillation in the Sonata style — progressive mask scheduler, anisotropic masked-point jitter, losses at coarser scales, LN — with a density-per-stratum reconstruction auxiliary.
6. **Optional second stage:** multi-view contrastive on repeat-coverage pairs.
7. **Evaluation:** linear probe first, always. If linear probing is weak, the shortcut has won, and no amount of fine-tuning will fix the representation — it will only mask the problem.

Point 7 is the practical test. **Linear probing is the diagnostic for the geometric shortcut**, which is precisely why Sonata frames its contribution in those terms. Run it early and often; it is cheap and it is the canary.

Further reading for Chapter 22

- Wu et al. (2025), *Sonata: Self-Supervised Learning of Reliable Point Representations*, CVPR 2025, with the [project page](#) and [Meta’s accessible write-up](#) — the latter has the clearest explanation of why the geometric shortcut is 3D-specific, with the figure showing what coordinates alone reveal. Read this before designing any pretraining objective.
- [Sonata code and weights](#).

- *Advancing ALS Applications with Large-Scale Pre-Training* — the closest prior work; mine its sampling design and negative results.
- BEV-MAE, for the pillar-masking and density-reconstruction design; GeoMAE for the geometric-target alternative.
- Watch the Pointcept lineage for successors (Concerto, NeurIPS 2025; Utonia, ICML 2026) — this area moves faster than the ALS literature that will eventually apply it.

Chapter 23. Spatial Extent, Crop Size, and Stitching

23.1 The argument for choosing extent ecologically

Block size is usually chosen by GPU memory. That is backwards. **Choose it from the ecological support of the task, then make the compute work.**

The scales in play:

Phenomenon	Characteristic scale
Individual tree crown	3–20 m
Canopy gap	5–50 m
Field plot / ABA support	20–30 m (400–900 m ²)
NEON base plot	40 m
PureForest patch	50 m
Bird point count effective radius	50–200 m
Stand / management unit	100–500 m
Mammal home range	0.5–50 km
Delivered tile	1000 m

A model whose effective receptive field is smaller than the phenomenon cannot learn it. A model whose field is much larger wastes capacity on irrelevant context and, worse, has more opportunity to learn spatial position — which is the rung-1 leakage problem of §19.5 arriving through the architecture.

Note that “receptive field” is not the crop size. For PTV3 it is roughly the patch size in points converted to metres via local density (§21.6); for sparse conv it grows with layer count and voxel size; for SPT it depends on the partition and graph depth. **Compute it explicitly for your architecture and report it in metres.** I have not seen an ALS paper do this, and it is a one-paragraph calculation that would improve the literature.

23.2 The point-budget versus area trade

Three ways to define a sample, each with a distinct bias:

Fixed point count N , variable area. Batches cleanly; standard in the object-centric literature. **Biases toward dense regions** — a fixed- N crop covers less ground in dense canopy than in sparse, so the physical extent of your sample is a function of the vegetation you are trying to measure. For ecological tasks with a defined plot support, this is disqualifying: your sample geometry no longer matches your label geometry.

Fixed area, variable N . Matches the label support, which is what §18.10 demands. Requires padding and masking, or variable-length batching. The engineering cost is real and worth paying.

Fixed area, then downsample to fixed N . The pragmatic compromise: crop by area, then voxel- or Poisson-downsample to a target count. Preserves geometric correspondence with labels while giving fixed-shape batches. **This is what I would do**, with `filters.sample` for the downsampling (§14.3), and with the caveat that the downsampling ratio then varies with density — so record it, and consider it a covariate.

23.3 Density normalization

Two competing approaches, and the choice interacts with everything in Part I.

Normalize down to a common density. Thin every sample to the minimum common pulse density. Homogeneous; discards information from good collections; produces a lowest-common-denominator model. Appropriate as a baseline (§4.5, Strategy 1).

Condition on density. Keep native density, feed local point density and per-sample pulse density as inputs. Preserves information; risks out-of-distribution conditioning at inference.

A subtlety specific to PTV3. If patch size is defined in points, density normalization is not merely about information content — it also *fixes the receptive field in metres* (§21.6). That makes normalization more attractive for serialization-based architectures than for voxel-based ones, where voxel size already fixes physical scale. This is a concrete case where the density decision depends on the architecture, and it deserves to be stated rather than left as a coincidence.

My recommendation: condition on density as the primary approach, with degradation augmentation (§4.5) covering the range you will encounter, plus normalized-down as a reported baseline. And for PTV3 specifically, test density-scaled patch size as an alternative to input normalization.

23.4 Overlapping crops and inference stitching

At inference you need predictions over a region larger than one crop. Naive tiling produces **seam artifacts**: points near a crop boundary have truncated neighbourhoods, so their features are computed from partial context and their predictions are systematically worse.

The solution is overlapping inference with buffered crops, and the implementation detail that matters is how you merge.

Myria3D's approach is the one to copy, and it is already implemented: final interpolation of predictions back to the full cloud happens **by receptive field**, with multiple logits for the same point **sum-reduced before softmax**, which enables test-time shifts. In other words: run overlapping subtiles, accumulate logits per point across all crops that covered it, then normalize once. Averaging probabilities after softmax is the common alternative and is worse, because it double-counts confident-but-truncated predictions.

Buffer width should be set by receptive field, not convenience. A crop with a buffer narrower than the receptive field still has truncated context at the output region's edge. The

rule: $buffer \geq receptive\ field$, computed in metres per §23.1.

For regression and pooled-embedding tasks rather than per-point prediction, the analogous move is to compute the embedding on a buffered crop and pool only over the target region — which is exactly what §14.5’s pipeline does with its buffered read and exact crop.

23.5 The experiment I would run

Sweep block size at fixed compute, and publish the curve.

Almost nobody does this, and it is the single most useful piece of engineering knowledge a benchmark could produce for downstream users. The design:

- Block sizes spanning the range of §23.1: 25, 50, 100, 200, 400, 1000 m.
- **Fixed compute budget per configuration** — so larger blocks get proportionally fewer samples or coarser downsampling, which is the trade a practitioner actually faces.
- Evaluated on tasks spanning supports: tree detection (fine), ABA biomass (plot), bird occupancy (intermediate), mammal occupancy (coarse).
- Receptive field reported in metres for each configuration.

The prediction worth stating in advance: performance should peak where the receptive field approximately matches the task’s ecological support, with a broad plateau above and a sharp fall below. If that pattern appears, it gives practitioners a design rule. If it does not — say performance keeps improving with extent on all tasks — that is evidence that context matters more than support, which is itself important and would push toward larger crops than anyone currently uses.

Either result is publishable, which is the mark of a good experiment.

23.6 Multi-scale embeddings, revisited

§19.4 proposed concentric multi-scale pooled embeddings for evaluation. Chapter 23’s material makes the implementation concrete:

- **One buffered read** per sample centre, at the largest extent needed.
- **Concentric crops** at 30, 100, 300, 1000 m from that single read (`filters.crop` with successive polygons).
- **Coarser downsampling at larger extents** — the fine structure that matters at 30 m is irrelevant at 1 km, and this keeps compute roughly constant per scale.
- **Height-stratified pooling** within each extent (§19.3).
- **Concatenate or let the probe weight** the per-scale vectors.

The by-product is the finding I keep returning to: **which structural scale predicts which taxon**. That is an ecological result derived from an engineering necessity, and it is the kind of thing that makes a benchmark paper cited by people who do not care about benchmarks.

Further reading for Chapter 23

- [Myria3D](#) — for the receptive-field interpolation and overlapping-subtile inference of §23.4. Read the inference code even if you do not use the library.
- [Superpoint Transformer](#) for how a partition-based method handles large-scene inference differently.

- The PointNet++ and PTV3 papers again, for the receptive-field calculations of §23.1.
- On the general question of scale in ecological remote sensing, the classic reference is Wiens (1989), *Spatial scaling in ecology*, Functional Ecology — worth reading precisely because it predates all of this and frames the problem better than the ML literature does.

Bridge to Part VII

Part VI covered the models. Part VII assembles everything into a benchmark.

Chapter 24 covers corpus construction: stratified sampling (with *pacasam* as the reference implementation), deduplication across overlapping projects, the leaf-on/leaf-off split, dataset-size scaling curves, and the storage and manifest engineering that Part IV specified. Chapter 25 covers the baseline matrix and the full evaluation protocol as an executable plan. Chapter 26 covers release: licensing across 3DEP, NEON, and IGN; dataset and model cards; the versioned frozen test split; and a leaderboard protocol that does not invite test-set overfitting.

Then a closing chapter on what I think the highest-value contributions in this project are, in priority order — because the outline as it stands describes more work than one project can do well, and choosing is the last technical decision.

Part VII — Building the Benchmark

Airborne Laser Scanning for Machine Learning in Ecology

Orientation to Part VII

Everything so far has been analysis. This Part is a plan.

- **Chapter 24** covers corpus construction: stratification, deduplication, phenology, scaling curves, and the storage engineering Part IV specified.
 - **Chapter 25** is the baseline matrix and the evaluation plan as something you could put in a Gantt chart.
 - **Chapter 26** covers release: licensing across three data programs with three different regimes, dataset and model cards, the frozen test split, and a leaderboard protocol that does not invite overfitting.
 - **Chapter 27** is prioritization. The outline as it stands describes more work than one project can do well. Choosing what to drop is the last technical decision, and I would rather make the argument explicitly than leave it implicit.
-

Chapter 24. Corpus Construction

24.1 What “representative” has to mean

The claim you want to support is: *a model trained on a representative sample of 3DEP should work across the United States*. Chapter 6 established why the archive’s own spatial distribution

will not deliver that — coverage was driven by partner funding, so quality level, acquisition year, sensor, and contractor are all geographically structured (§6.1).

So representativeness has to be constructed, and against a specified target population. Three candidate targets, and they give different samples:

1. **Representative of CONUS land area.** Uniform spatial sampling. Dominated by rangeland, desert, and agriculture; forests under-represented relative to their ecological interest.
2. **Representative of CONUS ecological variation.** Stratified by ecoregion and land cover, which over-samples rare systems relative to their area. This is what *pacasam* does for France, oversampling rare classes and difficult landscapes deliberately.
3. **Representative of the deployment distribution** — where users will actually apply the model, weighted by likely use. Unknowable precisely, but forest-weighted and biased toward areas with active management or conservation interest.

My recommendation: option 2 for pretraining, with option 1 reported as a check. Ecological stratification gives the encoder exposure to the full range of structural regimes, which is what a general-purpose representation needs. But you should also report the *area-weighted* composition of your corpus so readers can see how far it departs from the landscape, and so a user in a rangeland state knows whether their system was over- or under-represented.

24.2 The stratification design

Four axes, from §6.8:

Ecoregion. EPA Level III (85 in CONUS) is the natural unit — fine enough to capture real ecological variation, coarse enough that each stratum can be populated. Level IV (over 900) is too fine for balanced sampling but useful as a reporting dimension.

Land cover. NLCD classes within ecoregion, since a single ecoregion spans forest, grassland, developed, and agricultural cover with different structural signatures.

Quality level. QL1 versus QL2 primarily, with QL0 where it exists. Sampling across QL is what makes leave-one-QL-out possible (§19.6), and if you sample only QL2 you cannot measure density transfer at all.

Phenological state. Derived from acquisition date plus ecoregion (§11.4). This is the axis most often omitted and, in deciduous regions, the one with the largest effect on the point cloud.

The combinatorial problem. 85 ecoregions $\times$ $\sim$ 8 relevant NLCD classes $\times$ 3 quality levels $\times$ 2 phenological states is over 4,000 cells, most of them empty because the archive simply does not contain, say, leaf-on QL1 coverage of a particular rangeland ecoregion. Two practical responses:

- **Hierarchical allocation.** Allocate a budget across ecoregions (proportional to $\sqrt{\text{area}}$, which compromises between equal and proportional allocation), then within ecoregion allocate across land cover, then fill from whatever QL and phenology combinations exist. Record what you could not fill.

- **Publish the realized stratum table**, including empty cells. A table of what the archive *cannot* provide is genuinely useful to the field — it tells the next person which generalization claims are unsupportable with 3DEP alone.

A cross-cutting axis worth adding: terrain complexity, as slope percentiles. §10.4 established that ground classification fails on slopes and §6.5 that vertical accuracy is only assessed on flat ground. If steep terrain is under-sampled in your corpus, your model’s worst failure mode will be invisible until deployment.

24.3 Deduplication and repeat coverage

Overlapping 3DEP projects mean the same ground appears more than once, from different years and sensors. Three distinct decisions:

For pretraining: repeat coverage is *not* duplication in the harmful sense — the two clouds are genuinely different observations. But including both without accounting doubles the weight of those locations, which are non-randomly distributed (repeat coverage concentrates in high-partner-interest areas). Either cap the number of epochs per footprint or weight inversely by coverage count.

For evaluation splits: repeat coverage is a leakage hazard. If footprint F appears as a 2016 QL2 tile in training and a 2021 QL1 tile in test, leave-one-project-out has not held out the location. **Split on footprint, not on tile or project.**

For the sensor-gap design: repeat coverage is the asset (§4.3, §19.7). Locate all footprints with two or more acquisitions by different sensors, and reserve them as a dedicated stratum. These are the natural experiments where sensor and geography separate, and they are the only places in 3DEP where they do.

Implementation: a spatial self-join on the WESM geometries gives you the overlap graph. Compute it once, store coverage count and sensor-diversity count per footprint in the manifest, and let those fields drive all three decisions.

24.4 The leaf-on / leaf-off problem

The largest ecological confound in 3DEP (§4.4, §5.5), and it requires a deliberate decision rather than a default.

Three options:

1. **Restrict to leaf-on.** Ecologically cleaner; drastically reduces available data in deciduous regions, where 3DEP’s terrain mandate favours leaf-off; and produces a geographically biased corpus (western conifer systems, where leaf-on/leaf-off matters less, would dominate).
2. **Include both, condition on state.** Feed phenological state as a covariate. Preserves data volume; requires the model to learn two sensing regimes; and at inference the user must supply the state (easy — it is a function of date and location).
3. **Include both, stratify in reporting.** As 2, plus mandatory stratified evaluation.

Recommendation: option 3. Include both, condition on state, and report every result separately for leaf-on and leaf-off. The reason is that *users will have leaf-off data* — a model

that only works leaf-on is not deployable across the eastern US — so excluding it optimizes the benchmark at the expense of the application.

But restrict the paired sensor evaluation to leaf-on (§19.7), because NEON flies at peak greenness and a leaf-off 3DEP comparison would contaminate the sensor-gap measurement with a phenology gap.

A note on derivation. Assigning phenological state from date requires an ecoregion-appropriate phenology model. Options in increasing sophistication: fixed date windows per ecoregion; a growing-degree-day model; or actual satellite phenology (MODIS/VIIRS land surface phenology, or a Sentinel-2 NDVI time series) at the tile’s location and acquisition date. **The third is the right answer and is not much harder** — it gives you a continuous greenness value at the exact acquisition date rather than a guessed category, and it handles the mixed-stand case (§17.4) where the deciduous fraction is leaf-off and the conifer fraction is not.

24.5 Corpus size and scaling curves

You do not know how much data you need, and the honest way to find out is to measure it.

Build the corpus in nested subsets — 1%, 3%, 10%, 30%, 100% — each a proper subset of the next, each independently stratified. Then:

- **Pretrain at each size** and evaluate downstream. The resulting scaling curve tells you whether more data would help, which is the single most useful piece of information for anyone extending the work.
- **Publish the subsets.** A user with limited compute can take the 3% subset and get a defensible result; a user with a cluster can take the whole thing. This is what FRACTAL’s design enables and it costs you nothing beyond a nested sampling scheme.

The prior comparison point: the 3DEP pretraining study used over 17,000 km² and 184 billion points. That is a reference scale, not a target — and notably, whether that was the right amount is exactly what a scaling curve would have told them.

Storage arithmetic, so the plan is grounded. At QL2 with ~7 points/m² delivered (§5.4), 1 km² is roughly 7 million points. Stored as float32 XYZ plus a few uint8/uint16 attributes, call it 20 bytes per point after downsampling to a working density, so ~140 MB/km² uncompressed, perhaps 40 MB compressed. 17,000 km² is then roughly 700 GB — large but entirely manageable on a single storage volume. **Compute, not storage, will be your constraint**, which is worth knowing early because it changes what you optimize.

24.6 Sample geometry for pretraining

Distinct from evaluation geometry (§19.4), because pretraining does not have a label support to match.

Recommendation: fixed-area square crops, area chosen from §23.5’s sweep, tiled with modest overlap within selected footprints. Reasoning: fixed area preserves the metric scale that §20.6 says is the signal; squares are simple and align with tiling; modest overlap (10–20%) gives more samples and some natural augmentation without much redundancy.

Avoid: fixed point counts (§23.2’s density bias); crops that straddle project boundaries (they mix acquisition regimes within one sample, which is either a useful augmentation or a confound, and you should decide deliberately rather than accidentally); crops with high water fraction unless water is a class you care about.

Buffered extraction is mandatory (§14.5) — ground classification on an unbuffered crop is the correctness bug most likely to silently corrupt your entire corpus.

24.7 The construction pipeline

Consolidating §15.6 with this chapter’s decisions:

1. **Snapshot and version WESM.** Build the project table with QL, dates, LBS version, status/reason codes.
2. **Compute the overlap graph.** Coverage count and sensor diversity per footprint.
3. **Assign phenology** from satellite land-surface phenology at each project’s acquisition dates.
4. **Spatial joins:** ecoregion, NLCD, slope statistics, distance to road, NEON site proximity.
5. **Allocate the stratified budget** hierarchically; record unfillable cells.
6. **Apply exclusions:** NEON footprints plus receptive-field buffers; evaluation-holdout footprints; projects with disqualifying WESM status.
7. **Select footprints and resolve tiles** via per-project tile indexes.
8. **Extract** with the buffered PDAL pipeline; consistent ground; HAG plus coarsened elevation; no intensity in the model path but retained in storage.
9. **Compute provenance** per sample (§11.7) and the engineered metric set (§16.8).
10. **Validate:** MSHR contact sheets; distribution checks; DTM-versus-delivered-DEM disagreement flags. **Look at the data.**
11. **Write nested subsets** to sharded storage with provenance denormalized into each shard.
12. **Publish the manifest** as a versioned artifact.

Steps 2, 3, and 10 are the ones most likely to be skipped and most likely to matter.

24.8 Reference implementations to study first

- **[pacasam](#)** — the sampler behind FRACTAL, solving exactly your problem: diverse, class-balanced selection from a national ALS holding. Read it before writing your own.
 - **[FRACTAL](#)** — the dataset paper, for the diversity-sampling rationale and the release structure.
 - **[OpenTopography’s 3DEP notebooks](#)** — working extraction code against the actual holdings.
 - **The 3DEP pretraining study** — for a nationally-scaled sampling design and its stated limitations.
-

Chapter 25. Baselines and Evaluation

25.1 The baseline matrix

Six rows, in ascending order of what they cost and descending order of how likely they are to embarrass you.

#	Model	Purpose
1	Engineered metrics (§16) + gradient boosting	The operational state of the art. If you lose here, stop and debug.
2	Multi-band raster (CHM/DTM/percentile stack) + 2D CNN	The under-reported baseline (§20.7); cheap, strong, ImageNet-pretrainable
3	Sparse-conv U-Net, from scratch	Density-robust reference (§21.8); the control for QL transfer
4	PTv3, from scratch	Does the architecture help without pretraining?
5	PTv3 + Sonata weights	Does <i>indoor</i> pretraining transfer to airborne data?
6	PTv3 + your ALS pretraining	The contribution

Row 5 deserves emphasis because it is the cheapest informative experiment in the entire project. Sonata weights are released; loading them and probing takes days, not months. **If Sonata-pretrained PTv3 substantially beats from-scratch PTv3 on your tasks, then domain-specific ALS pretraining has a high bar to clear and you learn that in week three rather than month fifteen.** If it does not transfer at all, that is itself the motivating result for your corpus.

Two more rows worth adding if budget allows:

7 | Superpoint Transformer | Efficiency frontier; published aerial numbers (§21.7) |
 8 | Optical-only (Sentinel-2 embedding) | Rung 3 of the ablation ladder (§19.5), as a model rather than a covariate |

25.2 The evaluation matrix

Every model in §25.1 $\times$ every task in §17–18 $\times$ every split in §19.6 is a large grid, and running all of it is neither possible nor necessary. A staged design:

Stage 1 — competence check. Models 1–6 on forest-structure tasks (ABA biomass, canopy height, PureForest species), spatial-block splits only. Purpose: does the representation work at all? Failure here blocks everything downstream.

Stage 2 — transfer characterization. Models 1, 3, 6 on the same tasks, across all seven splits of §19.6. Purpose: the transfer table of §8.5. This is where the sensor-gap and QL-transfer measurements happen.

Stage 3 — biodiversity probing. Models 1, 2, 6, 8 on the biodiversity tasks with the full ablation ladder including rungs 2b and 4b. Purpose: the representation-quality question that motivates the project.

Stage 4 — targeted experiments. The specific measurements this book has argued are novel: the block-size sweep (§23.5), PTv3 serialization variants (§21.6), pooling ablation (§19.3), height-normalization ablation (§22.5), and the density-scaled-patch-size test.

Stage 5 — external validation. FRACTAL, PureForest, and a ULS transfer probe (FOR-instance), to place your representation against published numbers.

Staging matters because Stage 1 can kill the project cheaply and Stage 4 is where the interesting results live. Do not run them in the opposite order.

25.3 What to fix before running anything

The reproducibility commitments, made once:

- **Manifest version pinned** per experiment, recorded in every result row.
- **Probe protocol identical** across representations: same optimizer, schedule, regularization search, seed count (§19.2).
- **Pooling fixed** across representations, with the pooling ablation run separately as its own experiment (§19.3).
- **Splits materialized as manifest fields**, not recomputed per run (§15.4).
- **Seeds:** minimum three, five preferred, with confidence intervals on differences. Many ALS-ML comparisons report single runs with differences smaller than seed variance.
- **Compute logged** per result (§19.8).

25.4 The results tables you should plan to produce

Designing the outputs backwards from what a reader needs:

Table A — Headline. Tasks $\times$ models, spatial-block split, primary metric with confidence intervals. The table everyone reads.

Table B — Transfer. Models $\times$ splits, one representative task per domain. The table that makes the generalization claim.

Table C — Ablation ladder. Tasks $\times$ rungs 1, 2, 2b, 3, 4, 4b, 5. The table that makes the *contribution* claim, and the one a skeptical reviewer will go to first.

Table D — Stratified. Headline results broken by quality level, sensor, phenology, ecoregion group, and slope class. The table that makes negative results interpretable, and which exists only because of the provenance schema.

Table E — Efficiency. Accuracy versus compute and parameter count.

Figure 1 — Scaling curves. Downstream performance versus pretraining corpus size (§24.5).

Figure 2 — Label-efficiency curves. Performance at 1%, 10%, 100% of labels (§19.8).

Figure 3 — Block-size sweep (§23.5), with receptive field in metres on the axis.

Figure 4 — Sensor gap. Track A versus Track B per task (§19.7).

Figures 3 and 4 are, I think, the two most likely to be cited by people building on this, because they answer engineering questions nobody has answered.

25.5 The honest failure modes

Four ways this benchmark could produce a misleading result, and the check for each.

Leakage through spatial autocorrelation. Check: rung 1 of the ablation ladder. If coordinates alone approach full performance, the split failed.

Leakage through repeat coverage. Check: split on footprint, verify no footprint appears in both train and test across projects (§24.3).

Sensor identity as a shortcut. Check: train a classifier to predict sensor model from the frozen embedding. If it succeeds easily, the embedding encodes acquisition rather than ecology, and the leave-one-sensor-out results will be poor for a reason you can now name. **This is a cheap and rarely-run diagnostic and I would make it standard.**

Accessibility as a shortcut. Check: rung 2b, road-distance-only baseline, on eBird, Snapshot USA, and NEON small mammals (§19.5).

Add a fifth, less obvious one: **the engineered-metric equivalence.** Check: rung 4b. If engineered metrics match the learned embedding everywhere, you have built an expensive metric calculator — a legitimate finding, but one that should reframe the paper rather than be buried.

Chapter 26. Release

26.1 Licensing: three programs, three regimes

USGS 3DEP — public domain as a US government work. No restrictions on redistribution or derivative use; attribution is customary rather than required. This is the easiest case and it is why 3DEP is a good foundation for a public benchmark.

NEON — and here is a change you need to know about, because it is recent and it inverts the situation described in most existing documentation. **As of June 30, 2026, NEON data are released under CC BY 4.0**, replacing the previous CC0 regime, and **downloads now require sign-in with a NEON account or API token**. CC BY requires attribution: appropriate credit, a link to the license, and an indication of whether changes were made, without implying endorsement. Sensitive-data protections are unchanged — federally threatened and endangered species and site-host species of concern remain protected.

Three practical consequences:

1. **Redistribution of NEON-derived samples now carries an attribution obligation.** If your benchmark ships NEON-derived crops or labels, the dataset card must carry the CC BY notice, the link, and a statement of modifications.
2. **Programmatic access requires a token.** Update to `neonUtilities` v4.0.1 (R) or `neonutilities` v2.0.1 (Python), and **do not commit the token** — this is a new footgun in any public notebook or workflow you publish.
3. **Some documentation is stale.** NEON pages describing CC0 predate the change. Verify against the current data-use policy rather than the first page a search returns.

IGN (PureForest, FRACTAL) — Licence Ouverte / Open Licence 2.0. Permissive with attribution.

Everything else. eBird has its own terms via the Cornell Lab and GBIF; iNaturalist observations carry per-observation licenses that vary and include some non-commercial terms; Snapshot USA is on Dryad under its terms; sPlotOpen has its own conditions via iDiv. **You cannot assume a uniform license across your evaluation labels**, and the correct release strategy follows from that.

26.2 The release strategy that follows

Given the license patchwork, **do not ship a monolithic dataset**. Ship three layers:

Layer 1 — the manifest and code, fully open. Sample geometries, provenance records, split assignments, engineered metrics, and the extraction pipeline. Under a permissive license (MIT or Apache-2.0 for code, CC0 or CC BY for the manifest). This layer is your actual contribution and it is unencumbered because it is *your* work describing public data.

Layer 2 — 3DEP-derived point clouds, openly redistributable. Public domain source, so you can host the crops directly. This is what makes the benchmark usable without everyone re-running extraction.

Layer 3 — label joins, by reference. For NEON, eBird, iNaturalist, and the rest, ship the *join* — sample ID to label, with the query needed to fetch the label from its source — rather than the label values where the license is restrictive or unclear. Provide a script that reconstructs the full dataset given the user’s own credentials.

This structure is more work than a single tarball and it is the only version that is both usable and licensable. It also has a side benefit: Layer 1 is independently valuable. Someone who wants to build a different benchmark on 3DEP can take your manifest and skip a year of provenance engineering.

26.3 Documentation

A dataset card covering: motivation and intended use; composition (with the realized stratum table from §24.2, including empty cells); collection process (the whole of Parts I–IV in summary); preprocessing (exact PDAL stages and parameters); known biases (the accessibility biases of §7.7 and §18.2, the leaf-off bias, the QL-geography confound, the near-ground blind zone of §3.5); recommended and discouraged uses; and licensing per layer.

A model card per released checkpoint: architecture, pretraining corpus version, objective and hyperparameters, compute used, evaluation results including the stratified table, known failure modes (steep terrain, out-of-distribution density, sparse-crowned systems), and intended-use guidance.

Per-task documentation carrying the §19.10 checklist for each task: support, detection structure, coordinate uncertainty, temporal policy, splits, known floors.

The last item is the one that distinguishes a benchmark from a leaderboard. **A task whose sensing and measurement floors are documented cannot be misread as a model failure**, which protects both you and everyone who follows.

26.4 The frozen test split

Three commitments:

1. **Version the split and freeze it.** Test-set assignments live in the manifest, are published, and do not change between releases. If you must change them, that is a new major version with a new name.
2. **Consider withholding labels for a subset.** A held-back evaluation partition with labels released only through a submission service prevents test-set overfitting more effectively than any honour system. It also costs real infrastructure effort, so decide based on whether you expect a competitive leaderboard.
3. **Publish the split *before* publishing results on it.** This is the cheapest credibility measure available, and it is nearly free if the manifest is versioned.

26.5 A leaderboard protocol that does not rot

If you run a leaderboard, four design choices determine whether it stays meaningful:

- **Report the stratified table, not one number.** A leaderboard ranked on a single pooled metric will be won by whoever overfits the dominant stratum.
- **Require compute disclosure.** Otherwise the ranking becomes a budget ranking.
- **Cap submissions per team per period.** Repeated test-set queries are gradient descent on the test set.
- **Report the random-versus-spatial-split gap** for every submission. A model with a small gap is more trustworthy than one with a slightly better spatial score and a huge gap, and surfacing this changes what people optimize.

26.6 Reproducibility artifacts

The minimum set: pinned environment (container image, plus exact PDAL and library versions); the manifest version; all configs; seeds; the extraction pipeline as executable code rather than prose; and a single script that reproduces one table end to end from a fresh environment. That last item is the one that gets skipped and the one that determines whether anyone can build on the work.

Chapter 27. Prioritization

27.1 The problem with the plan

This book describes a project substantially larger than one person can execute well. Six data sources, eight transfer axes, six baselines, five experiment stages, and a national pretraining corpus. Attempting all of it produces a thin version of everything.

So: what is actually novel here, what is merely necessary, and what should be dropped?

27.2 The genuinely novel contributions, ranked

1. **The provenance-aware corpus and manifest (Parts III–IV).** No ALS-ML dataset carries per-sample acquisition provenance at national scale. This is the contribution that makes

every other result interpretable, it is unglamorous, and it is the one I would protect first. It is also independently valuable — Layer 1 of §26.2 is useful even if the models disappoint.

2. The sensor-gap measurement (§19.7). Paired 3DEP/NEON evaluation over identical footprints with identical processing, isolating the acquisition domain gap from the ecological one. Nobody has published this for ALS, and it answers the question every downstream user has. It is also cheap relative to its value, because the infrastructure is shared with everything else.

3. Block size and receptive field, measured (§23.5, §21.6). The sweep at fixed compute, plus the specific finding about PTv3’s density-dependent receptive field in metres. Engineering knowledge the field needs and does not have.

4. The degradation-augmentation set (§4.5). Pulse-level thinning, return truncation, near-ground blanking, scan-angle resampling, intensity-correlated z bias. A few hundred lines, not assembled this way anywhere I have found, and directly targeting the deployment distribution.

5. Multi-scale structural attribution across taxa (§19.4). Which structural scale predicts which taxon, as an ecological result falling out of an engineering necessity.

6. The ALS-specific shortcut mitigations (§22.3–22.5). Anisotropic masked-point jitter, height-normalization ablation, and linear probing as the shortcut diagnostic. Small changes with a plausible large effect on whether pretraining works at all.

7. The multi-view objective on repeat coverage (§22.6). The most promising under-explored idea in the book, and the least certain.

27.3 What is necessary but not novel

The pretraining itself; the standard baselines; the forest-structure tasks; the biodiversity probing. All of this has to happen, and none of it is where the paper’s contribution lives. Budget it as infrastructure and resist the temptation to elaborate it.

27.4 What I would drop or defer

Drop: a 3DEP semantic segmentation task. Labels do not exist (§10.7), annotation would be a project in itself, and the result would be partly circular. Use FRACTAL and PureForest as external semantic benchmarks instead.

Defer: FIA. The Memorandum of Cooperation timeline is long and the current federal-staffing situation adds uncertainty (§18.8). Start the request in month one so it is available if it arrives, and do not build a critical path through it.

Defer: sPlotOpen functional traits. Run the survivability count in week one (§18.6). If it yields under a few hundred CONUS post-2012 plots, demote it to a case study and use NEON plant cover as the primary trait task.

Reframe: iNaturalist insects. Keep as an explicit noise stress test, not a headline task, paired with NEON ground beetles as the rigorous counterpart (§18.3, §18.7).

Fix: the GlobalGeoTree description (§18.5). It contains no lidar. Substitute PureForest, and optionally add the GBIF-occurrences-intersected-with-3DEP task after a coordinate-uncertainty survivability count.

Defer: the leaderboard and submission service (§26.4–26.5). Publish the frozen split and the manifest; build the service only if there is demonstrated demand.

27.5 A staged plan

Months 1–3. WESM ingest, provenance schema, extraction pipeline, validation tooling. Sonata-weights probe on a small labelled subset (§25.1 row 5 — the cheapest informative experiment). sPlotOpen survivability count. FIA MoC request submitted. Engineered-metric baseline running.

Months 4–8. Corpus construction with nested subsets. Forest-structure tasks with baselines 1–4. Stage 1 competence check. Block-size sweep begins, since it needs no pretraining.

Months 9–14. Pretraining, with the shortcut mitigations and linear probing as the running diagnostic. Scaling curves. Stage 2 transfer characterization, including the sensor-gap measurement.

Months 15–20. Biodiversity probing with the full ablation ladder. Stage 4 targeted experiments. External validation.

Months 21–24. Release engineering, documentation, writing. Reserve real time here; the three-layer release of §26.2 and the documentation of §26.3 are weeks of work, not days.

Two structural notes. The block-size sweep is early because it does not depend on pretraining and its result informs everything after. And the Sonata probe is in month one because it can change the project’s framing: if indoor pretraining transfers well, your corpus needs to justify itself against a stronger baseline than you were expecting.

27.6 The result I would most like to see

If I could extract one number from this project it would be **Figure 4: the sensor gap, per task**. Track A minus Track B, with labels and geography held fixed and processing held identical.

Because that number is what stands between an ALS foundation model and actual use. Everyone building on 3DEP has data flown by a different contractor with a different sensor at a different density than whatever the model saw, and right now nobody can tell them what that costs. A credible estimate of it — even a discouraging one — would be more useful to the field than another point of mIoU.

And it falls out of infrastructure you are building anyway, which is the definition of a contribution worth prioritizing.

Closing

Four things this book has argued, restated plainly.

A LAZ file is not a measurement. It is the fourth or fifth derived product of one, and roughly thirty parameters shaped it, of which five are recorded in the file. Knowing which of those parameters vary across your corpus, and how they correlate with geography, is the difference between a benchmark that measures ecology and one that measures processing software.

Sensor heterogeneity is collinear with everything you care about. Sensor, contractor, year, quality level, region, and land cover are entangled by construction in 3DEP. You cannot fully separate them, and the honest response is to measure the entanglement, report stratified results, and find the repeat-coverage natural experiments where the axes partly decouple.

The protocol matters more than the architecture. Spatial autocorrelation, accessibility shortcuts, and sensor identity will each produce impressive-looking results from nothing. The ablation ladder, the spatial splits, and the shortcut diagnostics are what make a number mean something — and they are cheaper than the models they evaluate.

The unglamorous artifact is the durable one. Architectures will be superseded within two years; the frontier already moves faster than the ALS literature can follow it. A provenance-aware, nationally stratified, honestly documented corpus with versioned splits will still be useful when PTV3 is a historical footnote. Build that first, and build it well.

Appendix A — Consolidated resource list

Specifications - [USGS Lidar Base Specification Online](#); [Revision History](#); [Revision Status](#) - [3DEP Quality Levels](#) - [ASPRS LAS 1.5 \(HTML\)](#); [LAS 1.4-R15 \(PDF\)](#); [ASPRS LAS GitHub](#) - [COPC specification](#); [EPT specification](#) - [NGS New Datums](#)

Data access - [WESM geopackage](#); [3DEP Spatial Metadata](#) - [AWS Open Data: USGS 3DEP](#) - [NEON data portal](#); [NEON data use policy](#) - [viewer.copc.io](#); [validate.copc.io](#)

Software - [PDAL](#) and [workshop](#); [lasR](#); [laspy](#) - [lidR](#) and [lidR book](#); [lidRmetrics](#) - [Pointcept](#); [Sonata](#); [Myria3D](#); [Superpoint Transformer](#) - [pacasam](#); [Biolith](#) - [OpenTopography 3DEP notebooks](#)

Datasets and benchmarks - [PureForest](#); [FRACTAL](#) - [NeonTreeEvaluation](#); [FOR-instance](#) - [OpenForest catalogue](#)

Key papers - [Point Transformer V3](#); [Sonata](#) - [ALS large-scale pre-training](#); [FRACTAL](#) - [Ploton et al., spatial validation](#) - [eBird Best Practices](#) - [Meier et al., NEON TOS sampling](#)

Appendices

Airborne Laser Scanning for Machine Learning in Ecology

These appendices develop three things the main text argued for but left as prose. Each is intended to be implementable as written.

Appendix B — Classification and Validity Flags: A Practical Reference

Chapter 10 covered ground-classification algorithms and Chapter 12 covered the bit-level layout. This appendix consolidates the operational question: what actually happens to produce the class codes and flags in your file, and what should your loader do about it.

B.1 The production workflow

Classification is a semi-automated pipeline run by the contractor after strip adjustment and vertical calibration. The stages, in order:

1. **Gross noise and blunder removal.** Before anything else, because a single point 500 m below ground will corrupt any surface fit. Isolated-point tests, local-minimum tests, height gating against a plausible elevation envelope.

2. **Ground classification.** The substantive step, and an *iterative surface fit* rather than a per-point decision. The dominant implementation in a 3DEP-era corpus is progressive TIN densification (Axelsson 2000, i.e. TerraScan): seed with confident low points, build a TIN, iteratively admit points whose distance and angle to the enclosing facet fall below thresholds, refine, repeat. Alternatives are hierarchical robust interpolation (SCOP++), cloth simulation, and progressive morphological filtering.

Parameters — seed spacing, iteration angle and distance thresholds, maximum building size — are tuned per project and often per terrain type *within* a project.

3. **Derived classes from the ground surface.** Water from breaklines plus low-lying flat regions; bridge decks (class 17) usually semi-manually against transportation vectors; vegetation strata (3/4/5) by height-above-ground thresholding, *if the project requires them at all*.

4. **Manual QC and editing.** A technician reviews cross-sections, hillshades, and profile views, fixes obvious failures, and re-runs affected regions with different parameters.

Stage 4 is invisible in the delivered file and is a substantial part of why vendor ground classification often outperforms a naive re-derivation. It is also unrecorded, unquantified, and variable in thoroughness between contractors — which is the argument for consistency over per-project quality (§10.10).

B.2 What the specification does and does not constrain

Constrained: the minimum set of classes that must exist; that no point remains in class 0 unless withheld; that classification is consistent across a project; that all non-withheld ground-classified points are used in DEM creation; that the delivered DEM meets NVA on flat open check points.

Unconstrained: the algorithm, its parameters, the amount of manual editing, and any assessment of ground-surface quality under vegetation on slopes.

So the variable with the largest influence on every height-above-ground feature you compute is the one the specification leaves free. This is not an oversight — 3DEP's mandate is terrain

products assessed on open ground — but it is the single most consequential fact about your corpus.

B.3 The class codes that matter for ecology

Code	Name	Notes for your pipeline
0	Never classified	Must not appear unless withheld
1	Unclassified	In much of 3DEP this is “everything above ground.” Do not read it as “unknown”
2	Ground	Your DTM source; quality varies (§10.4)
3/4/5	Low/medium/high vegetation	Optional in 3DEP; thresholds project-specific; not comparable
6	Building	Optional
7	Low noise	May or may not also be withheld
9	Water	Only <i>verified</i> where associated with a breakline
12	(Reserved)	Historically used for overage; its presence dates a file
17	Bridge deck	Matters if you compute HAG near infrastructure
18	High noise	Same caveat as class 7
20	Ignored ground	Real ground deliberately excluded from the DEM — vertical, near-vertical, or overhanging bare-earth surfaces whose inclusion would create interpolation artifacts
21	Snow	Rare, and disqualifying for canopy work if present
22	Temporal exclusion	Valid at acquisition but transient; rarely used

Two implications worth internalizing.

Class 20 means your DTM and the delivered DEM legitimately disagree. A surface built from class 2 alone omits class 20; the delivered DEM omits it too, by design. But if you re-derive ground with SMRF, your algorithm has no concept of “ignored” and will happily include those surfaces. Disagreement between your DTM and the delivered DEM concentrated on steep terrain may be class 20, not error — which is why the `dtm_disagreement_by_slope_class` field of §11.7 needs interpretation rather than thresholding.

Class 1 is not a semantic label. In projects that deliver no vegetation classes, class 1 is the residual of “not ground, not water, not noise.” Treating it as a semantic category in a segmentation task is a category error.

B.4 The four bit flags

Separate byte in PDRF 6–10; packed with classification in PDRF 0–5 (§12.3).

Flag	Meaning	3DEP status
Synthetic	Point derived rather than measured	Rare; if present, these are fabricated and are not observations
Key-point	Model key point — a thinned subset representing the surface	Flagged if computed
Withheld	Not a valid surface return	Required since LBS 2020 rev. A; supersedes classification
Overlap	Point in a sidelap region	Shall not be used in 3DEP deliverables; overage is removed at tiling

B.5 The withheld flag in detail

This is a *different axis* from classification. Classification answers *what is this point?* Withheld answers *is this a valid measurement of a surface at all?*

The current rule: the withheld bit flag shall **only** be used to identify points that cannot reasonably be interpreted as valid surface returns. Examples given by the specification: outliers, blunders, geometrically unreliable points, aerosol back-scatter, laser multi-path, airborne objects, and sensor anomalies. The flag may be used alongside other classification codes (low/high noise, for instance), but it must be used in all cases meeting those criteria.

Mapped onto Part I's physics, that list covers:

- **Airborne objects** — birds, aircraft, insects (§3.2)
- **Aerosol back-scatter** — cloud, haze, smoke returns (§1.3)
- **Laser multi-path** — reflections off water or glass arriving late, producing points below the true surface (§2.5)
- **Sensor anomalies** — including multiple-pulses-in-air range ambiguity ghosts over high relief (§2.5)
- **Geometrically unreliable points** — earlier specification wording spelled this out as points *near the extreme edge of the swath*, which is exactly why the density assessment of §5.4 excludes the outer ~5% of each swath

That last category is the important one conceptually: **withheld covers not only spurious returns but real returns whose geometry cannot be trusted.**

The revision history, which reveals the intent

Version	Rule
LBS v.2.1	“Outliers, blunders, geometrically unreliable points near the extreme edge of the swath, and any other points the data producer deems unusable” — vague, at producer discretion
2020 rev. A	Reworded to “cannot reasonably be interpreted as valid surface returns”; use became required
2021 rev. A	“only” inserted , closing the loophole of flagging inconvenient-but-valid points; proof of performance introduced
2023 rev. A	Proof-of-performance artifact must exactly match the delivered point cloud; methodology write-up required in the Lidar Mapping Report

The tightening ran in both directions. Under-flagging leaves birds in your canopy; over-flagging deletes real surface. USGS’s Lidar Error Dictionary illustrates the latter with a figure of geometrically valid points wrongly flagged withheld.

Why the flag is authoritative and the class code is not

The governing clause: use of the withheld bit flag is required for all points that are not valid surface returns, and **the flag supersedes any classification applied to these points** — provided the flag is appropriately applied, any classification, including class 0, is acceptable.

So a withheld point may carry class 0, class 1, class 7, class 18, or in principle class 2. **A filter on classification alone does not remove invalid returns.**

The failure mode is specific: withheld points are disproportionately high-altitude noise, so a pipeline ignoring the flag will occasionally see canopy heights of hundreds of metres, and every metric sensitive to the upper tail — maximum height, h_{99} , rumple, canopy-relief ratio — is corrupted at exactly the samples where it matters most.

Flagged, not deleted

Withheld points are ones the contractor *chose to leave in* the classified deliverable. Flagging is an alternative to deletion, not a record of it. Points removed outright are simply absent and leave no trace — which means the withheld fraction is a measure of *flagging practice*, not of noise burden.

Proof of performance: the MSHR

Maximum Surface Height Rasters are built from the highest non-withheld point per pixel, at twice the bare-earth DEM cell size. Properly flagged data yields a raster of uniform appearance; improperly flagged non-valid returns produce visible non-uniformity.

Steal this. Rendering MSHRs across your sampled tiles into a contact sheet is a fast, visual, whole-corpus check for the exact failure that corrupts your height statistics. It is one writ-

ers.gdal stage (dimension: Z, output_type: max, on non-withheld points) plus a montage command.

B.6 The loader policy

Stage order matters. Do not reorder.

```
pipeline = (
    reader
    # 1. Validity. Authoritative, and first.
    | pdal.Filter.expression(expression="Withheld == 0")
    # 2. Defensive noise-class exclusion, for pre-2020 projects
    #   where invalid returns may be encoded as class only.
    | pdal.Filter.expression(expression="Classification != 7 && Classification != 18")
    # 3. Your own conservative screen, for projects that flagged nothing.
    | pdal.Filter.outlier(method="statistical", mean_k=8, multiplier=3.0)
    | pdal.Filter.expression(expression="Classification != 7")
    # 4. Only now: ground, HAG, features.
    | pdal.Filter.smrf(...)
)
```

Record the count removed at each of the three stages separately. The pattern identifies which specification regime a project belongs to:

Withheld fraction	Noise-class fraction	Interpretation
> 0	any	Post-2020 practice; flag is trustworthy
0	> 0	Likely pre-2020; invalid returns encoded as class only
0	0	Either pristine, or nothing was flagged or classified — inspect before trusting
Very high (> 5%)	any	Possible over-flagging; check MSHR for deleted real surface

A caveat on the geometrically-unreliable category. Because swath-edge points may be withheld, filtering on the flag also removes some of the edge-of-swath geometry. That is correct for feature computation and it interacts with your scan-angle covariate (§2.3): the retained scan-angle distribution is truncated by a process that varies between projects. Record `scan_angle_abs_p95` *after* withheld filtering, so the covariate describes the data you actually use.

Appendix C — Stratified Allocation: An Implementable Specification

Chapter 24 argued for stratification on ecoregion $\times$ land cover $\times$ quality level $\times$ phenology and noted the combinatorial problem. This appendix specifies the allocation concretely.

C.1 The frame

Define the sampling frame as the set of candidate footprints — fixed-size square cells (say 1 km) tiled over CONUS — each carrying, from the joins of §24.7:

- `ecoregion_l3` (85 levels in CONUS)
- `nlcd_class` (dominant, plus fractions)
- `quality_level` (from WESM)
- `phenology` (continuous greenness at acquisition date, plus a derived category)
- `slope_p50`, `slope_p95`
- `coverage_count`, `sensor_diversity` (from the overlap graph, §24.3)
- `wesm_status` (meets / meets-with-variance / does-not-meet)
- `excluded` (NEON buffer, evaluation holdout)

C.2 Allocation across ecoregions

Neither equal nor proportional allocation is right. Equal allocation over-weights tiny ecoregions; proportional allocation lets the Great Basin dominate. Use **square-root allocation**, the standard compromise:

$$n_e = N \cdot \frac{\sqrt{A_e}}{\sum_{e'} \sqrt{A_{e'}}}$$

where A_e is the eligible area of ecoregion e and N the total budget. Apply floor and ceiling constraints:

$$n_e \leftarrow \text{clip}(n_e, n_{\min}, 0.05N)$$

with $n_{\min}$ set so that even the smallest ecoregion gets enough samples to be represented (a few hundred), and the ceiling preventing any single ecoregion from exceeding 5% of the corpus.

Optionally weight by an ecological-heterogeneity term — the entropy of the NLCD distribution within the ecoregion — so structurally diverse regions receive more samples than homogeneous ones of the same area.

C.3 Allocation within ecoregion

Within ecoregion e , allocate across land cover proportionally to $\sqrt{\text{area}}$ again, but with two modifications:

1. **Floor for forested classes.** NLCD 41/42/43 (deciduous/evergreen/mixed forest) and 90 (woody wetlands) get a minimum share regardless of area, because these are where the structural signal lives and where your downstream tasks are.

2. **Cap for developed and open water.** Classes 21–24 and 11 get a ceiling, since they contribute little ecological signal and can dominate in some ecoregions.

C.4 Filling QL and phenology

These are the axes you cannot control, because the archive provides what it provides. So treat them as **stratification-by-inclusion** rather than allocation:

```
for each (ecoregion, nlcd) cell with budget n:
    candidates = eligible footprints in cell
    # Partition candidates by (QL, phenology_category)
    subcells = group(candidates, by=[quality_level, phenology_cat])
    # Allocate n across non-empty subcells as evenly as possible,
    # capped by each subcell's available count
    alloc = water_fill(n, [len(s) for s in subcells])
    # Within subcell, prefer:
    #   1. wesm_status == 'meets'
    #   2. spatial dispersion (maximin distance among selected)
    #   3. slope diversity (stratify on slope_p95 quartile)
    for s, k in zip(subcells, alloc):
        select k from s by maximin_dispersion, stratified on slope quartile
    record shortfall = n - sum(alloc) # cells the archive cannot fill
```

Water-filling — allocate evenly, redistribute the excess from subcells that cannot supply their share — is the right primitive here, because most subcells are supply-limited rather than budget-limited.

Maximin dispersion within a subcell (greedily select the candidate farthest from those already chosen) prevents clustering, which matters because eligible footprints are clustered by project.

C.5 Reporting the shortfall

The output of C.4 is two tables, and **both should be published**:

Table 1 — realized composition. Counts by ecoregion $\times$ NLCD $\times$ QL $\times$ phenology.

Table 2 — shortfall. Requested minus realized per cell, with a reason code: `no_coverage`, `no_leaf_on_coverage`, `no_QL1_coverage`, `all_excluded`, `wesm_status_fail`.

Table 2 is more useful to the field than Table 1. It is a map of what generalization claims are unsupportable with 3DEP alone — and nobody has published it.

C.6 Nested subsets

For the scaling curves of §24.5, build subsets by **assigning each selected footprint a uniform random priority at selection time**, then defining the $p\%$ subset as the footprints with priority below p . This guarantees nesting (the 3% subset is a subset of the 10%) while preserving approximate stratum proportions in every subset, since the priority is independent of stratum.

Store `subset_priority` in the manifest. One float, and it makes every scaling experiment a filter rather than a re-sample.

C.7 Exclusion geometry

The leakage rule of §19.7 requires excluding NEON footprints *and their buffers* from pretraining. Concretely:

```
buffer_m = max(receptive_field_m over all architectures tested) * 2
exclusion_zone = union(NEON_flight_boxes).buffer(buffer_m)
eligible = frame.difference(exclusion_zone)
```

Two notes. The factor of two is deliberate slack, because you will change architectures and you do not want to rebuild the corpus. And the exclusion must be computed on **flight boxes**, not plot locations — NEON flight boxes are at least 10×10 km, and excluding only the plots would leave the surrounding landscape in the pretraining set, which for a model with kilometre-scale context is not an exclusion at all.

If you additionally want leave-one-ecoregion-out results for ecoregions containing NEON sites, the exclusion must extend to the whole ecoregion, which is expensive. Decide which ecoregions matter for that split before building, and record the decision in `splits`.

Appendix D — Receptive Field in Metres: Measurement and the Serialization Experiment

§21.6 claimed that PTV3’s receptive field on ALS is density-dependent, and §23.1 claimed that receptive field should be reported in metres. Neither is standard practice. This appendix specifies both.

D.1 Why the analytic estimate is not enough

For a sparse-convolutional network, the receptive field is computable in closed form: sum the dilated kernel extents through the layer stack, multiply by voxel size, and you have metres. Clean.

For PTV3 it is not, because the receptive field is defined by *patch membership in a serialized order*, and the physical extent of a patch of P points depends on the local point density after grid pooling — which varies within a sample and between acquisitions.

An analytic first approximation: at pooling stage s with grid size g_s , points are roughly one per occupied voxel. For a 2.5D surface with mean vertical occupancy $\bar{k}_s$ voxels per column, the planimetric density is $\rho_s \approx \bar{k}_s / g_s^2$ points per m^2 . A patch of P points then covers approximately

$$A_s \approx \frac{P}{\rho_s} = \frac{P g_s^2}{\bar{k}_s}, \quad \ell_s \approx \sqrt{A_s} = g_s \sqrt{P / \bar{k}_s}$$

Note that $\bar{k}_s$ — the vertical thickness of the point cloud in voxels — is a *canopy property*. So the patch extent shrinks in dense multi-layered canopy and grows over bare ground, within a single

sample. That is worth stating plainly: **the effective receptive field of a serialization-based transformer on ALS is smaller in forest than in field.**

Because the estimate depends on measurable quantities, measure them rather than assuming.

D.2 The empirical measurement protocol

A perturbation test, which is architecture-agnostic and takes an afternoon:

1. Take a representative sample (fixed area, native density).
2. Run a forward pass; record output features F_0 at every point (or at the pooled embedding, for whole-sample tasks).
3. Pick a probe point p near the sample centre.
4. Perturb p : displace it by a small amount, or delete it, or add a synthetic point at its location.
5. Re-run; record F_1 .
6. Compute per-point $|F_1 - F_0|$ and locate the set S of points whose features changed by more than a threshold (say 1% of the feature-norm distribution's median).
7. Receptive field radius = the 95th percentile of $\text{distance}(p, q)$ for q in S .
8. Repeat over ~ 100 probe points and ~ 20 samples spanning the density and canopy-cover range.

Report the result as a **distribution**, not a number, and cross-tabulate against local pulse density and canopy cover. If the claim in §21.6 is right, you will see receptive field in metres varying by a factor of two or more across the density range, and shrinking with canopy cover per §D.1.

For whole-sample pooled embeddings the same test works with the embedding as output; the “receptive field” then becomes the spatial extent over which a single point influences the pooled vector, which is the quantity that matters for §19.4’s multi-scale design.

Cost: two forward passes per probe. Trivially parallel. This is a cheap experiment that would improve the ALS literature measurably, because at present nobody reports it at all.

D.3 The serialization experiment

Four configurations, one task, one corpus. Hold everything else fixed.

Config	Serialization	Grid	Patch size	Rationale
S0	Default PTV3 (Z, TZ, H, TH), isotropic grid	g in all axes	Fixed P points	The inherited baseline
S1	Same curves, anisotropic grid	g_{xy} , with $g_z = g_{xy}/4$	Fixed P	Resolve vertical structure at the scale it varies (§20.3)

Config	Serialization	Grid	Patch size	Rationale
S2	2D serialization on (x, y) ; height as input feature	g_{xy} only	Fixed P	Explicit pillar interpretation, matching 2.5D geometry (§20.7)
S3	Default curves, density-scaled patch size	g isotropic	$P \propto \hat{\rho}$ per sample	Fixes receptive field in metres across acquisitions (§23.3)

Primary outcome: downstream task performance under **leave-one-QL-out**, because that is the split where a density-dependent receptive field should hurt. **Secondary:** in-domain performance, throughput, and the measured receptive-field distribution from §D.2.

The prediction, written down in advance: S0 shows the largest in-domain-to-QL-transfer gap, because its receptive field changes silently across the density shift. S3 shows the smallest gap at some in-domain cost. S1 and S2 improve on S0 primarily on tasks requiring vertical discrimination (species, stratification) rather than on horizontal tasks (cover, gap detection).

If that pattern holds, the recommendation writes itself and it is actionable by anyone using PTv3 on airborne data. If S0 is fine everywhere, then serialization is more robust than the geometry suggests, and that is worth knowing too — it would mean the space-filling-curve concern is theoretical rather than practical, which is a cleaner result than a hedge.

D.4 Reporting

For every architecture in the baseline matrix, report a row:

Architecture	RF (median, m)	RF (IQR, m)	RF sensitivity to density	RF sensitivity to cover
Sparse-conv U-Net, 0.5 m voxel	analytic + measured	narrow	~none by construction	~none
PTv3, $P=1024$, default	measured	wide?	to be measured	to be measured
PTv3, P density-scaled	measured	narrower?	reduced by design	to be measured
SPT	measured	partition- dependent	to be measured	to be measured

That table, plus Figure 3 of §25.4 (the block-size sweep with receptive field on the axis), is the practical engineering contribution of the architecture work. It tells a practitioner how to pick a crop size and a patch size for their own density regime — which is the question they will actually have, and which no current paper answers.